

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 12.09.2026, 18:29:28
Файлов исходного кода: 83
Суммарный объём кода: 1.16 МВ
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: seg-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sum]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борушка [id: mst-borushka]
23. 21. Алгоритм Кнута-Морриса-Патта (KMP) [id: kmp]
24. 22. Z-функция [id: string-z-func]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

Билеты (учебный контент)

- 23. src/data/tickets/ticket1_5.ts
- 24. src/data/tickets/ticket14_20.ts
- 25. src/data/tickets/ticket21_24.ts
- 26. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 27. src/components/ArticulationPointsViz.tsx
- 28. src/components/BellmanFordViz.tsx
- 29. src/components/BfsViz.tsx
- 30. src/components/ChapterImage.tsx
- 31. src/components/ChapterNav.tsx
- 32. src/components/ChapterView.tsx
- 33. src/components/DfsBridgesSimulator.tsx
- 34. src/components/DijkstraViz.tsx
- 35. src/components/DPVisualizer.tsx
- 36. src/components/EulerSimulator.tsx
- 37. src/components/ExpressQuiz.tsx
- 38. src/components/FloydViz.tsx
- 39. src/components/GraphTraversalViz.tsx
- 40. src/components/HeapViz.tsx
- 41. src/components/hints/demoKit.tsx
- 42. src/components/hints/demosExtra.tsx
- 43. src/components/hints/demosGraph.tsx
- 44. src/components/hints/demosStruct.tsx
- 45. src/components/hints/MiniDemo.tsx
- 46. src/components/hints/SimulatorModal.tsx
- 47. src/components/hints/TermPopover.tsx
- 48. src/components/JohnsonViz.tsx
- 49. src/components/KosarajuViz.tsx
- 50. src/components/KruskalSimulator.tsx
- 51. src/components/MnemonicCards.tsx
- 52. src/components/MobileToc.tsx
- 53. src/components/Navbar.tsx
- 54. src/components/PlanarityDemo.tsx
- 55. src/components/QueueViz.tsx
- 56. src/components/SalmonAutomatonWidget.tsx
- 57. src/components/SegmentTreeVisualizer.tsx

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/83: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```
```

` .github/workflows/deploy-pages.yml `

Полное содержимое после изменений

```yaml

name: Deploy app to GitHub Pages

on:

push:

branches:

- main

# Позволяет проверить Pages прямо из рабочей ветки

- "arena/\*\*"

workflow dispatch:

```
 path: ./dist
```

```
 deploy:
```

```
 name: Publish to GitHub Pages
```

```
 needs: build
```

```
 runs-on: ubuntu-latest
```

```
 environment:
```

```
 name: github-pages
```

```
 url: ${ steps.deployment.outputs.page_url }
```

```
 steps:
```

```
 - name: Deploy
```

```
 id: deployment
```

```
 uses: actions/deploy-pages@v5
```

```
....
```

```
Patch относительно `main`
```

```
````diff
```

```
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
```

```
new file mode 100644
```

```
index 00000000..17418e3
```

```
--- /dev/null
```

```
+++ b/.github/workflows/deploy-pages.yml
```

```
@@ -0,0 +1,60 @@
```

```
+name: Deploy app to GitHub Pages
```

```
+
```

```
+on:
```

```
+  push:
```

```
+    branches:
```

```
+      - main
```

```
+      # Позволяет проверить Pages прямо из рабочей ветки
```

```
+      - "arena/**"
```

```
+  workflow_dispatch:
```

```
+
```

```
+permissions:
```

```
+  contents: read
```

```
+  pages: write
```

```
+  id-token: write
```

Универсальное пособие • полный исходный код

```
-----
+   runs-on: ubuntu-latest
+   environment:
+     name: github-pages
+     url: ${ steps.deployment.outputs.page_url }
+   steps:
+     - name: Deploy
+       id: deployment
+       uses: actions/deploy-pages@v5
+   ```
```

`.github/workflows/rebuild-pdf.yml`

Полное содержимое после изменений

```yaml

name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

```
#
исходный код → full_code_all_pages.txt (script)
|
→ page-0001.png ... page-NNNN.png (
|
→ full_code_all_page
#
```

# Готовые TXT и PDF коммитятся обратно в ветку (их отда  
# промежуточные PNG сохраняются как artifacts запуска.

on:

push:

branches:

- "\*\*"

paths-ignore:

# чтобы коммит самого workflow не запускал бескон

- "public/export/\*\*"

- "\*\*/\*.md"

workflow\_dispatch:

inputs:

- ```
    for policy in /etc/ImageMagick-6/policy.xml /
      if [ -f "$policy" ]; then
        sudo sed -i 's/rights="none" pattern="\{P
      fi
    done
    convert -version
```
- name: Install dependencies
run: npm ci --no-audit --no-fund
 - name: Step 1 – код → txt
run: npm run export:txt
 - name: Step 2 – txt → png
env:
 EXPORT_DENSITY: \${github.event.inputs.density}
run: npm run export:png
 - name: Step 3 – png → pdf
run: npm run export:pdf
 - name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true
 - name: Summary
run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \`public/export/full\_code\_all\_pages`"
 echo "| \`public/export/full\_code\_all\_pages`"
 echo "| PNG-страниц | \$(ls tmp/export-pages) |"
 } >> "\$GITHUB_STEP_SUMMARY"
 - name: Upload artifacts (PDF + TXT)

```
    steps:
      - name: Checkout
      - uses: actions/checkout@v4
      + uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
      - uses: actions/setup-node@v4
      + uses: actions/setup-node@v7
        with:
          - node-version: "22"
          + node-version: "24"
            cache: npm

      - name: Install ImageMagick + DeJaVu fonts
@@ -97,7 +97,7 @@ jobs:
        } >> "$GITHUB_STEP_SUMMARY"

      - name: Upload artifacts (PDF + TXT)
      - uses: actions/upload-artifact@v4
      + uses: actions/upload-artifact@v7
        with:
          name: full-code-export
          path: |
@@ -107,7 +107,7 @@ jobs:
          retention-days: 30

      - name: Upload artifacts (PNG-страницы)
      - uses: actions/upload-artifact@v4
      + uses: actions/upload-artifact@v7
        with:
          name: full-code-pages-png
          path: tmp/export-pages/*.png
    ....
```

Полное содержимое после изменений

````xml

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
 <rect width="64" height="64" rx="16" fill="#4f46e5"/>
 <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
 <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
```

### Patch относительно `main`

````diff

```
diff --git a/public/favicon.svg b/public/favicon.svg
new file mode 100644
index 00000000..3bbba0
--- /dev/null
+++ b/public/favicon.svg
@@ -0,0 +1,5 @@
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
+  <rect width="64" height="64" rx="16" fill="#4f46e5"/>
+  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
+  <circle cx="47" cy="21" r="4" fill="#34d399"/>
+</svg>
```

`src/App.tsx`

Полное содержимое после изменений

````tsx

```
import { useCallback, useEffect, useMemo, useState } from 'react';
import { chapters } from '../data/content';
import { Navbar } from '../components/Navbar';
import { Sidebar } from '../components/Sidebar';
import { MobileToc } from '../components/MobileToc';
import { ChapterView } from '../components/ChapterView';
```



```

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
 {activeTab === "guide" ? (
 <>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />

 <div className="flex flex-col lg:flex-row max-w-7xl mx-auto" style={{ gap: 10px }}>
 /* Сайдбар только на десктопе – на мобильном скрывается */
 <div className="hidden lg:block lg:w-80 shrink-0" style={{ padding: 10px }}>
 <Sidebar selectedChapterId={activeChapterId} chapters={chapters} />
 </div>

 <main id="main-content" className="flex-1 min-w-0" style={{ padding: 10px }}>
 /* Шапка главы с быстрыми переходами */
 <div className="flex items-center justify-between" style={{ padding: 10px 0 0 0 }}>
 <div className="min-w-0">
 <div className="text-[10px] uppercase" style={{ padding: 0 10px }}>
 {activeChapter.category || "Универсальное пособие"}
 </div>
 <h2 className="text-base sm:text-xl font-medium" style={{ padding: 0 10px }}>
 {activeChapter.title}
 </h2>
 </div>
 <ChapterNav prev={prev} next={next} index={index} />
 </div>

 <div className="h-1 w-full bg-slate-800 rounded" style={{ padding: 10px 0 0 0 }}>
 <div
 className="h-full bg-gradient-to-r from-slate-800 to-slate-700"
 style={{ width: `${progress}%` }}
 />
 </div>
 </div>
 </div>
 </>
) : null}
 </Navbar>
 </div>
)

```

-----  
````  

Patch относительно `main`

````diff

diff --git a/src/App.tsx b/src/App.tsx

index 2091f33..4583756 100644

--- a/src/App.tsx

+++ b/src/App.tsx

@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";

import { ChapterNav } from "../components/ChapterNav";

import { SimulatorModal } from "../components/hints/SimulatorModal";

import { SimulatorHub } from "../components/SimulatorHub";

+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {

- const [activeTab, setActiveTab] = useState<"guide" | "simulator">();

+ const [activeTab, setActiveTab] = useState<"guide" | "simulator">();

const [activeChapterId, setActiveChapterId] = useState<string>();

const [modalVizId, setModalVizId] = useState<string>();

@@ -95,7 +96,7 @@ export default function App() {

</main>

</div>

</>

- ) : (

+ ) : activeTab === "simulator" ? (

<main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">

<SimulatorHub

onOpen={setModalVizId}

@@ -105,6 +106,8 @@ export default function App() {

</>

</main>

</div>

+ ) : (

+ <PythonCompiler />

)

```

 </div>
 <div className="min-w-0 flex-1">
 <h1 className="text-xl font-extrabold text-white">
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-mono">
 Подготовка к экзаменам: Алгоритмы, Структуры
 </p>
 </div>
 </div>

 <div className="flex items-center w-full lg:w-auto">
 >
 <button
 id="tab-guide-btn"
 onClick={() => setActiveTab('guide')}
 className={`flex-1 lg:flex-none flex items-center justify-center
 duration-200 whitespace-nowrap ${
 activeTab === 'guide'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <BookOpen className="w-4 h-4" /> Учебное пособие
 </button>
 <button
 id="tab-simulator-btn"
 onClick={() => setActiveTab('simulator')}
 className={`flex-1 lg:flex-none flex items-center justify-center
 duration-200 whitespace-nowrap ${
 activeTab === 'simulator'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <Cpu className="w-4 h-4" /> Тренажёры
 </button>
 <button
 id="tab-projects-btn"
 onClick={() => setActiveTab('projects')}
 className={`flex-1 lg:flex-none flex items-center justify-center
 duration-200 whitespace-nowrap ${
 activeTab === 'projects'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <Code className="w-4 h-4" /> Проекты
 </button>
 </div>
</div>

```

```

 disabled={busy}
 className="flex items-center justify-center
er:bg-amber-600/20 border border-amber-500/
 title="Скачать всё пособие одним автономным
 >
 {busy ? <Loader2 className="w-4 h-4 animate
 </button>
 </div>
</div>
</header>
);
};
....

```

### Patch относительно `main`

```

````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
  import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
  import { chapters } from '../data/content';
  import { downloadBookHtml } from '../utils/exportHtml'

  interface NavbarProps {
-   activeTab: 'guide' | 'simulator';
-   setActiveTab: (tab: 'guide' | 'simulator') => void;
+   activeTab: 'guide' | 'simulator' | 'compiler';
+   setActiveTab: (tab: 'guide' | 'simulator' | 'compile
  }

  export const Navbar: React.FC<NavbarProps> = ({ active
@@ -60,9 +60,20 @@ export const Navbar: React.FC<Navbar
    >

```

Универсальное пособие • полный исходный код

import { CheckCircle2, ExternalLink, Info, Loader2, Play

```
const PYODIDE_VERSION = "0.27.7";  
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyo  
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyo
```

```
const STARTER_CODE = `# Первый запуск загрузит Python в  
name = "алгоритмы"
```

```
for number in range(1, 4):  
    print(f"{number}. Учим {name}!")  
`;  
`;
```

```
type PyodideRuntime = {  
  runPythonAsync: (source: string) => Promise<unknown>;  
  setStdout: (options: { batched: (message: string) => ;  
  setStderr: (options: { batched: (message: string) => ;  
  setStdin: (options: { stdin: () => string | number | ;  
};
```

```
type PyodideModule = {  
  loadPyodide: (options: { indexURL: string }) => Promi  
};
```

```
let runtimePromise: Promise<PyodideRuntime> | null = nu
```

```
function getPythonRuntime() {  
  if (!runtimePromise) {  
    runtimePromise = import(/* @vite-ignore */ PYODIDE_I  
      (module as PyodideModule).loadPyodide({ indexURL:  
    });  
  }  
  return runtimePromise;  
}
```

```
function errorText(error: unknown) {  
  if (error instanceof Error) return error.message;  
  return String(error);  
}
```

```

        setRunning(false);
    }
}, [code, stdin, running, runtimeReady]);

const reset = () => {
    setCode(STARTER_CODE);
    setStdin("");
    setOutput("Нажмите «Запустить», чтобы увидеть резул
};

return (
    <main className="max-w-screen-2xl mx-auto px-4 sm:p
    <div className="max-w-6xl mx-auto">
        <div className="flex flex-col sm:flex-row sm:it
            <div>
                <div className="inline-flex items-center gap
                    <Terminal className="w-4 h-4" /> Боковая
                </div>
                <h2 className="text-2xl sm:text-3xl font-ex
                <p className="text-slate-400 mt-2 max-w-2xl
                    Пишите небольшой код и запускайте его пря
                    который переводит CPython в WebAssembly.
                </p>
            </div>
            <a
                href="https://pyodide.org/"
                target="_blank"
                rel="noreferrer"
                className="inline-flex items-center gap-1.5
            >
                Как это работает <ExternalLink className="w
            </a>
        </div>

        <div className="grid xl:grid-cols-[minmax(0,1.33fr)
            <section className="rounded-2xl border border
                <div className="flex flex-wrap items-center
                    <div className="flex items-center gap-2">

```

```

        -none focus:ring-2 focus:ring-inset focus:ring-slate-500
        placeholder="Напишите Python-код..."
    />

    <div className="border-t border-slate-800 border-b border-slate-800 border-l border-slate-800 border-r border-slate-800" >
        <label className="block px-4 pt-3 text-slate-500" >
            Ввод для input() • по одной строке на клавишу
        </label>
        <textarea
            id="python-stdin"
            value={stdin}
            onChange={(event) => setStdin(event.target.value)}
            spellCheck={false}
            rows={2}
            placeholder={'Например:\nАлиса'}
            className="block w-full resize-y bg-transparent border border-slate-800"
        />
    </div>
</section>

<section className="rounded-2xl border border-slate-800" >
    <div className="flex items-center justify-between" >
        <div className="flex items-center gap-2" >
            <Terminal className="w-4 h-4 text-emerald-500" />
        </div>
        <span className={`inline-flex items-center gap-2`} >
            {runtimeReady && <CheckCircle2 className="w-4 h-4 text-emerald-500" />}
            {runtimeReady ? "готов" : "не загружен"}
        </span>
    </div>
    <pre className="min-h-[360px] max-h-[560px] px-4 py-4 leading-6 text-slate-300" >
        {output}
    </pre>
    <div className="border-t border-slate-800 border-b border-slate-800 border-l border-slate-800 border-r border-slate-800" >
    </div>
</section>
</div>

```

```
+name = "алгоритмы"
+
+for number in range(1, 4):
+    print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+  runPythonAsync: (source: string) => Promise<unknown>
+  setStdout: (options: { batched: (message: string) =>
+  setStderr: (options: { batched: (message: string) =>
+  setStdin: (options: { stdin: () => string | number |
+});
+
+type PyodideModule = {
+  loadPyodide: (options: { indexURL: string }) => Prom
+});
+
+let runtimePromise: Promise<PyodideRuntime> | null = n
+
+function getPythonRuntime() {
+  if (!runtimePromise) {
+    runtimePromise = import(/* @vite-ignore */ PYODIDE
+      (module as PyodideModule).loadPyodide({ indexURL
+    });
+  }
+  return runtimePromise;
+}
+
+function errorText(error: unknown) {
+  if (error instanceof Error) return error.message;
+  return String(error);
+}
+
+export function PythonCompiler() {
+  const [code, setCode] = useState(STARTER_CODE);
+  const [stdin, setStdin] = useState("");
+  const [output, setOutput] = useState("Нажмите «Запус
+  const [running, setRunning] = useState(false);
```



```

+   setOutput("Нажмите «Запустить», чтобы увидеть резу.
+   };
+
+   return (
+     <main className="max-w-screen-2xl mx-auto px-4 sm:p
+     <div className="max-w-6xl mx-auto">
+       <div className="flex flex-col sm:flex-row sm:i
+       <div>
+         <div className="inline-flex items-center g
+           <Terminal className="w-4 h-4" /> Боковая
+         </div>
+         <h2 className="text-2xl sm:text-3xl font-e
+         <p className="text-slate-400 mt-2 max-w-2x
+           Пишите небольшой код и запускайте его пр
+           который переводит CPython в WebAssembly.
+         </p>
+       </div>
+       <a
+         href="https://pyodide.org/"
+         target="_blank"
+         rel="noreferrer"
+         className="inline-flex items-center gap-1.5
+       >
+         Как это работает <ExternalLink className="
+       </a>
+     </div>
+
+     <div className="grid xl:grid-cols-[minmax(0,1.
+       <section className="rounded-2xl border borde
+       <div className="flex flex-wrap items-cente
+         <div className="flex items-center gap-2"
+           <span className="w-2.5 h-2.5 rounded-fr
+           <span className="text-sm font-bold tex
+           <span className="text-[11px] text-slate
+         </div>
+         <div className="flex items-center gap-2"
+           <button
+             type="button"

```

```

+         </label>
+         <textarea
+             id="python-stdin"
+             value={stdin}
+             onChange={(event) => setStdin(event.target.value)}
+             spellCheck={false}
+             rows={2}
+             placeholder={'Например:\nАлиса'}
+             className="block w-full resize-y bg-tran
ceholder:text-slate-700"
+         />
+     </div>
+ </section>
+
+     <section className="rounded-2xl border border-slate-200" >
+         <div className="flex items-center justify-between" >
+             <div className="flex items-center gap-2" >
+                 <Terminal className="w-4 h-4 text-emerald-500" />
+             </div>
+             <span className={`inline-flex items-center gap-2`} >
+                 {runtimeReady && <CheckCircle2 className="text-emerald-500" />}
+                 {runtimeReady ? "готов" : "не загружен"}
+             </span>
+         </div>
+         <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300">
+             {output}
+         </pre>
+         <div className="border-t border-slate-800 p-4" >
+         </div>
+     </section>
+ </div>
+
+ <div className="mt-5 grid md:grid-cols-3 gap-3" >
+     <div className="flex gap-2 rounded-xl border border-slate-200" >
+         <Info className="w-4 h-4 shrink-0 text-indigo-500" />
+         <span>Ctrl или Cmd + Enter запускает код. l
+     </div>
+     <div className="flex gap-2 rounded-xl border

```

```

    },
  },
  server: {
    host: "0.0.0.0",
    port: 5173,
    strictPort: true,
    // предпросмотр может открываться через внешний про
    allowedHosts: true,
  },
  preview: {
    host: "0.0.0.0",
    port: 4173,
    strictPort: true,
    allowedHosts: true,
  },
});

```

Patch относительно `main`

```

````diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локал
+ // VITE_BASE можно переопределить, если репозиторий
+ base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {

```

## Универсальное пособие • полный исходный код

- Определение/правило в центре (`border-blue-500` или
  - Обязательная сетка аналогий (2 колонки: неформальн
  - "Душные" детали (код, формулы, сложные доказательства
4. **\*\*Не ломай верстку:\*\*** Не придумывай свои CSS-классы в файлах `src/data/content.ts`. Не используй чистый markd

ФАЙЛ 4/83: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 13 | РАЗМЕР: 512

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100" >
 <head>
 <meta charset="UTF-8" />
 <meta name="viewport" content="width=device-width, initial-scale=1" />
 <title> Дискретка для тупых – Интерактивный гид по дискретке
 </head>
 <body class="bg-slate-950 text-slate-100 min-h-screen" >
 <div id="root"></div>
 <script type="module" src="/src/main.tsx"></script>
 </body>
</html>
```

ФАЙЛ 5/83: metadata.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 312

```
{
 "name": "Remix Remix: ExamPrep Reader",
 "description": "Удобная web-читалка для подготовки к экзаменам",
 "requestFramePermissions": [],
 "majorCapabilities": [
 "MAJOR_CAPABILITY_SERVER_SIDE_GEMINI_API"
]
}
```

## Универсальное пособие • полный исходный код

```

 "@types/react-dom": "19.2.3",
 "@vitejs/plugin-react": "5.1.1",
 "esbuild": "^0.28.2",
 "tailwindcss": "4.1.17",
 "typescript": "5.9.3",
 "vite": "7.3.2",
 "vite-plugin-singlefile": "2.3.0"
 },
 "description": "Интерактивное пособие по алгоритмам и
}
```

-----  
ФАЙЛ 7/83: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР: 1.5 КБ

-----  
# algv0 – Универсальное пособие по алгоритмам и структурам данных

Интерактивная web-читалка (React + Vite + Tailwind) с синтаксисом Markdown и автоматическим экспортом всего исходного кода в PDF.

## Быстрый старт

```
```bash  
npm install  
npm run dev          # предпросмотр на http://0.0.0.0:5174  
npm run build        # сборка в dist/ (single-file)  
```
```

## Экспорт кода: код → txt → png → pdf

Пайплайн собирает **\*\*весь\*\*** исходный код репозитория в один файл

| Команда              | Шаг       | Результат                     |
|----------------------|-----------|-------------------------------|
| ---                  | ---       | ---                           |
| `npm run export:txt` | код → txt | `public/export/full-code.txt` |
| `npm run export:png` | txt → png | `tmp/export-pages/...         |

## Универсальное пособие • полный исходный код

- \* **\*\*Всплывающие подсказки по терминам\*\*** – текст главы а известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается карточка мини-анимацией и кнопкой «Показать симулятор» (симулятор Словарь – `src/data/glossary.ts`, мини-анимации – `src/`)
- \* **\*\*Реестр интерактивов\*\*** – `src/components/vizRegistry` и для панели под главой, и для модалки из подсказки.

## ## Структура

...

|                    |                                           |
|--------------------|-------------------------------------------|
| src/               |                                           |
| App.tsx            | – оболочка, привязка глав к визуализации  |
| components/        | – интерактивные симуляторы (Действия)     |
| data/              | – контент пособия (главы/билеты/карты)    |
| scripts/export/    | – пайплайн код → txt → png → pdf          |
| public/export/     | – сгенерированные TXT и PDF (обновляемые) |
| .github/workflows/ | – автоматизация                           |

...

## # Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструментами, поддерживаемые теги и классы.

## ## Заголовки

- \* Заголовки секций: Используют стандартный тег `

## ` и классы.
- \* Подзаголовки: Используют тег `

### `.

## ## Текст и параграфы

- \* Обычный текст содержится в тегах `

`.
- \* Блоки "Шиза" (Аналогии) и другая текстовая инфографика: параграфы `

`.

## ## Математические формулы

В приложении используется MathJax.

Универсальное пособие • полный исходный код

## Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

|                    |           |                      |
|--------------------|-----------|----------------------|
| Глава              | id        | Что делать           |
| ---                | ---       | ---                  |
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

## Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет — и

|               |                                |                                        |
|---------------|--------------------------------|----------------------------------------|
| Билет         | Тема                           | Статус                                 |
| ---           | ---                            | ---                                    |
| <b>**1**</b>  | Дерево отрезков (Segment Tree) | Главы нет. Контент вхолостую.          |
| <b>**7**</b>  | Планарность и раскраска графов | Номерной главы нет; аналогии**.        |
| <b>**11**</b> | Мосты в графах                 | Номерной главы нет; есть без аналогии. |
| <b>**13**</b> | Эйлеровы пути и циклы          | Номерной главы нет; есть без аналогии. |

Дополнительно — «осиротевшие» визуализаторы без глав (контент есть)

- \* `stack-dfs` → `StackViz.tsx` — **\*\*Стек и DFS\*\***
- \* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` — **\*\*Очередь и BFS\*\***
- \* `heap-beam-search` → `HeapViz.tsx` — **\*\*Куча и лучевой поиск\*\***
- \* `segment-trees` → `SegmentTreeVisualizer.tsx` — **\*\*Дерево отрезков\*\***
- \* `dynamic-programming` → `DPVisualizer.tsx` — **\*\*Динамическое программирование\*\***

## Уровень 3. Есть 2D, но НЕТ бытовой аналогии («сухая математика»)

|       |     |             |
|-------|-----|-------------|
| Глава | id  | Комментарий |
| ---   | --- | ---         |

## Универсальное пособие • полный исходный код

|    |                                      |   |   |   |
|----|--------------------------------------|---|---|---|
| 16 | 16. Графы. Кратчайший путь. Флойд    | - | - | - |
| 17 | 17. Графы. Алгоритм Джонсона         | - | - | - |
| 18 | 18. Графы. Остовное дерево. Краскал  | 1 | - | - |
| 19 | 19. Графы. Остовное дерево. Прима    | 1 | - | - |
| 20 | 20. Графы. Остовное дерево. Борувка  | 1 | - | - |
| 21 | 21. Префикс-функция (п), КМП         | 4 | - | - |
| 22 | 22. Z-функция                        | 4 | - | - |
| 23 | 23. Алгоритм Ахо-Корасик             | 2 | - | - |
| 24 | 24. Классы сложности, сведение задач | 4 | - | - |
| -  | Обзор: Кратчайшие пути и Графы       | - | - | - |
| -  | Алгоритмы на карте                   | - | - | - |
| -  | Эйлер: Путь ≠ Цикл                   | - | - | - |
| -  | Мосты в графах: код + визуализация   | 1 | - | - |

## ## Рекомендуемый порядок работ

1. **\*\*Вернуть потерянные билеты 1, 7, 11, 13\*\*** и подключить ``segment-trees``, ``stack-dfs``, ``queue-bfs``, ``heap-be``.  
это 5 готовых компонентов, которые сейчас просто не
2. **\*\*Дописать аналогии\*\*** в 5 главах из «Уровня 3» (по ш
3. **\*\*Добавить 2D\*\*** к 4 главам «Уровня 4» – минимум `inli`
4. **\*\*Решить судьбу `alg-map`\*\*** – раскрыть или удалить.

ФАЙЛ 9/83: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

```
{
 "compilerOptions": {
 "target": "ES2020",
 "useDefineForClassFields": true,
 "lib": ["ES2020", "DOM", "DOM.Iterable"],
 "module": "ESNext",
 "skipLibCheck": true,
```



```

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
});
```

=====

## РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

=====

-----

ФАЙЛ 11/83: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 118 | РАЗМЕР: 4.9 КБ

-----

```
import { useCallback, useEffect, useMemo, useState } from "react";
import { chapters } from "../data/content";
```

```

const progress = useMemo(() => ((index + 1) / chapter

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
 {activeTab === "guide" ? (
 <>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />

 <div className="flex flex-col lg:flex-row max-w-7xl mx-auto" >
 {/* Сайдбар только на десктопе – на мобильном скрывается */}
 <div className="hidden lg:block lg:w-80 shrink-0" >
 <Sidebar selectedChapterId={activeChapterId} />
 </div>

 <main id="main-content" className="flex-1 m-4" >
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-between" >
 <div className="min-w-0">
 <div className="text-[10px] uppercase" >
 {activeChapter.category || "Универсальное"}
 </div>
 <h2 className="text-base sm:text-xl font-medium" >
 {activeChapter.title}
 </h2>
 </div>
 <ChapterNav prev={prev} next={next} index={index} />
 </div>

 <div className="h-1 w-full bg-slate-800 rounded" />
 </main>
 </div>
 </>
) : null}
 </Navbar>
 </div>
)

```

-----  
}

-----  
ФАЙЛ 12/83: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРӨК: 124 | РАЗМЕР: 5.0 КБ  
-----

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, text";

const escapeRe = (s: string) => s.replace(/[\.*+?^${}()|]/g, "\\$&");

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
 const alternation = GLOSSARY_LABELS.map((l) => escapeRe(l));
 // границы «не буква/цифра» – \b не работает с кириллицей
 return new RegExp(`(?<![\\p{L}\\p{N}_-])(${alternation})`);
}

let cachedRegex: RegExp | null = null;
const labelToId = new Map(GLOSSARY_LABELS.map((l) => [l, `term-${l.toLowerCase().replace(/ /g, '-')}`]));

/**
 * Проходит по тексту главы и оборачивает известные термины
 * , чтобы на них кликая
```

```

 if (!id) continue;

 const usedTerm = perTerm.get(id) ?? 0;
 const usedSurface = perSurface.get(surface) ?? 0;
 if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_SURFACE) continue;
 perTerm.set(id, usedTerm + 1);
 perSurface.set(surface, usedSurface + 1);

 if (m.index > last) frag.appendChild(document.createElement("div"));
 const span = document.createElement("span");
 span.className = "term-hint";
 span.dataset.termId = id;
 span.tabIndex = 0;
 span.setAttribute("role", "button");
 span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
 span.textContent = m[1];
 frag.appendChild(span);
 last = m.index + m[1].length;
 }

 if (last === 0) continue;
 if (last < text.length) frag.appendChild(document.createElement("div"));
 textNode.parentNode?.replaceChild(frag, textNode);
}

/**
 * Навешивает разметку терминов на контейнер при смене
 *
 * Дополнительно перепроверяет разметку после каждого рендера
 * (или что-то ещё) перезаписал innerHTML главы, подсказки
 * а не пропадают до перезагрузки страницы.
 */
export function useTermHints(ref: React.RefObject<HTMLDivElement>) {
 const run = () => {
 const el = ref.current;
 if (!el) return;
 try {

```

```
}
.no-scrollbar::-webkit-scrollbar {
 display: none;
}
.no-scrollbar {
 scrollbar-width: none;
}
}

@keyframes pulse-gold {
 0%,
 100% {
 stroke: #eab308;
 stroke-width: 5;
 filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
 }
 50% {
 stroke: #fef08a;
 stroke-width: 8;
 filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
 }
}

.animate-pulse-gold {
 animation: pulse-gold 1.5s infinite;
}

@keyframes tocIn {
 from {
 transform: translateX(-100%);
 }
 to {
 transform: translateX(0);
 }
}

@keyframes hintIn {
 from {
```

```
.chapter-body {
 max-width: 100%;
 overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
 min-width: 0;
}

.chapter-body :where(pre, code) {
 white-space: pre-wrap;
 word-break: break-word;
}

.chapter-body pre {
 overflow-x: auto;
 max-width: 100%;
 font-size: clamp(11px, 2.9vw, 13px);
}

.chapter-body table {
 display: block;
 width: 100%;
 overflow-x: auto;
 border-collapse: collapse;
 font-size: clamp(11px, 3vw, 14px);
}

.chapter-body summary {
 cursor: pointer;
 padding: 0.35rem 0;
}

@media (max-width: 767px) {
 .chapter-body :where(.grid) {
 grid-template-columns: minmax(0, 1fr) !important;
 }
 .chapter-body :where(h2) {
```

```
@keyframes demoProgress {
 from {
 width: 0%;
 }
 to {
 width: 100%;
 }
}
```

---

ФАЙЛ 14/83: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

---

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
 <StrictMode>
 <App />
 </StrictMode>
);
```

---

ФАЙЛ 15/83: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

---

```
export interface Chapter {
 id: string;
 title: string;
 type: "html" | "markdown";
 content: string;
 description?: string;
```

```

import { Chapter } from "../types";

export const additionalTickets: Chapter[] = [
 {
 id: "sparse-table",
 title: "2. Двумерная разреженная матрица (Sparse Table)",
 type: "html",
 description: "Разреженная таблица для идемпотентных операций",
 category: "Продвинутые структуры",
 content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 ух отрезков: $\min(ST[L][k], ST[R - 2^k + 1][k])$
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 border border-emerald-500 shadow-md">
 Аналогия 1: Школьные линейки
 </div>
 <p class="text-slate-300 text-sm mb-4">
 зок длины 7. Ты берешь две заготовленные заготовки
 ь отрезок длины 7 (перекрывая друг друга пополам)
 ничего складывать, просто пересекаем куски.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 border border-rose-500 shadow-md">
 Аналогия 2: Ступени
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
`
 }
]

```



```
<p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 двоичная куча. Пара – точка на декартовой
 </div>
```

```
<p class="text-slate-300 text-sm mb-4">Постр
 платит $\approx n \cdot \log n$ сравнений, counting по ма
 точка вставляется спуском от корня: $x \leq \text{уз}$
 единственно. Весь процесс – в симуляторе под
 «соединить сам» – кликаешь две точки,
 ь родитель», «получился бы цикл», «равные »,
 «Миф $2k/2k+1$ » и «Поиск $\neq$ сортировка»
 ждый запрещённый ход.</p>
```

```
<div class="text-center my-6">
 <p class="text-slate-400 text-xs uppercase">
 <p class="my-1 font-bold leading-none">

 ext-2xl text-slate-300">Й
 </p>
 <p class="text-slate-300 font-mono text">
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Split – таможня
 </div>
 <p class="text-slate-300 text-sm">Sp
 х»». Спуск от корня: узел, который вместе
 скрытия – досматривается только одна ве
 </div>
```

```
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 border-rose-500 shadow-md">
```

Аналогия 2: Merge – стыковка

```

 auto [L, R] = split(t->right, x);
 t->right = L;
 return {t, R};
 } else {
 auto [L, R] = split(t->left, x);
 t->left = R;
 return {L, t};
 }
}
}

```

</div>  
</details>

<details class="bg-slate-900/40 rounded-lg l  
<summary class="p-4 font-bold text-slate  
-b border-slate-700/50">

Код: вставка, merge, erase (Скрыт

</summary>

<div class="p-5 text-sm text-slate-300  
<pre class="bg-slate-950 p-4 rounded

```

def insert(root, key, pri):
 if root is None:
 return Node(key, pri)
 if key <= root.key: # равные x — левый
 root.left = insert(root.left, key, pri)
 else:
 root.right = insert(root.right, key, pri)
 return root

```

```

for key, pri in pairs_sorted_by_pri_desc: # порядок э
 root = insert(root, key, pri)

```

<pre class="bg-slate-950 p-4 rounded

```

def merge(a, b): # все x в a < все x в b
 if not a or not b: return a or b
 if a.pri > b.pri:
 a.right = merge(a.right, b); return a
 b.left = merge(a, b.left); return b

```

```

def erase(root, key): # удалить произвольн

```

```
<div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Стопка бумаг

```
</div>
```

```
<p class="text-slate-300 text-sm">С
трогаешь — он уполз вглубь. Ничего не удаля
а холодное тонет.</p>
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
```

Аналогия 2: Тропинка через га.

```
</div>
```

```
<p class="text-slate-300 text-sm">Ч.
разрешено временно быть кривым — серия обра
ртизированной $O(\log n)$.</p>
</div>
```

```
</div>
```

```
<div class="bg-slate-900/40 rounded-lg bord
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
я. Порядок виден во вкладке «Механика».</p>
 <p class="text-slate-400 text-sm mb-1">
иза кверху. Кадр «* груз» показывает его
 <p class="text-slate-400 text-sm">• «Sp
поднимается за $n-1$ поворотов и остаётся ба
</div>
```

```
<p class="text-slate-300 text-sm mt-4"><b с
нно <code class="bg-slate-950 px-2 py-0.5 r
одиночная операция — до <code class="bg-sl
"> $O(n)$ </code>. Повторное обращение к тому ж
ald-300 border border-slate-800"> $O(1)$ </code>
```

```

 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сжатие суммы подматрицы за $O(1)$ ",
 category: "Алгоритмы матрицы",
 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 ≈ Аналогия 1: Вырезание прямоу
 </div>
 <p class="text-slate-300 text-sm mb">
 з него маленький целевой прямоугольник в цел
 ый верхний угол отрезается дважды! Поэтому
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-planar-colors",

```

```

 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
перед.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Учеба в вузе
 </div>
 <p class="text-slate-300 text-sm mb-4">
екая сортировка – это расписание, которое
 >
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
border-rose-500 shadow-md">
 Ограничения: Циклы
 </div>
 <p class="text-slate-300 text-sm mb-4">
получить опыт, нужна работа". Алгоритм выяв
 </div>
 </div>
 </div>
</section>`,
 },

```

```

 type: "html",
 description: "Рёбра, удаление которых расхреначивает",
 category: "Графы. Продвинутые",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
и fup[v] > tin[u].</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Единственный мост
 </div>
 <p class="text-slate-300 text-sm mb-4">
ЭТОТ МОСТ, ЭКОНОМИКОЙ ОБОИХ КУСКОВ ПРИДЕТ
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-articulation",
 title: "12. Графы. Точки сочленения",
 type: "html",
 description: "Вершины, удаление которых убивает связность",
 category: "Графы. Продвинутые",
 content: `

```

```

</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-indigo-400"
 <div class="bg-slate-900 p-4 rounded-lg mb-4"
 <p class="text-lg text-indigo-300 italic"
 <p class="text-xl font-mono text-white"
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 b
 rder border-emerald-500 shadow-md">
 Аналогия 1: Рисование не отрыв
 </div>
 <p class="text-slate-300 text-sm mb-4"
 е сходится нечетное число линий, значит, из
 а везде должно быть четное число (зашел-вышел)
 </div>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "pathfinding-accel",
 title: "17. Графы. Ускорения поиска",
 type: "html",
 description: "",
 category: "Графы. Пути",
 content: `
<section id="pathfinding-accel" class="mb-12 scroll-mt-12"
 <div class="flex items-center mb-6 flex-wrap gap-3"
 <span class="bg-blue-600 text-white px-4 py-1 rounded"
 <h2 class="text-2xl sm:text-3xl font-bold text-white"
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-blue-400"

```

```

 (проверяем через DSU) - берем в ответ.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Прокладка интернет
 </div>
 <p class="text-slate-300 text-sm mb-6">
 с самых дешевых дорог в стране". Он строит
 единую сеть.</p>
 </div>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "mst-prima",
 title: "19. Графы. Остовное дерево. Прима",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-6">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 которое торчит из посещенных в непосещенных
 </div>
 </div>
 <div class="grid grid-cols-1 gap-6">

```





```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-blue-400"
 <div class="bg-slate-900 p-4 rounded-lg mb-1"
 <p class="text-lg text-blue-300 italic"
 <p class="text-xl font-mono text-white"
щегося с i.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 b
rder border-emerald-500 shadow-md">
 Аналогия 1: Копипаста
 </div>
 <p class="text-slate-300 text-sm mb-1"
в тексте кричит: "Эй! Начиная с этой буквы
т "окно" [L, R] самой дальней найденной копи
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate-300"
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300"
 <pre class="bg-slate-950 p-4 rounded"
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - l]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
 </div>
 </details>
</div>
</div>
</section>`,
},

```

```
 ь отличный переводчик на английский (Сведения
 аче В, то В – это NP-Полная (сосредо
 </div>
 </div>
</div>
</section>`,
 },
];
```

---

ФАЙЛ 18/83: src/data/content\_temp.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 385 | РАЗМ

---

```
export interface Chapter {
 id: string;
 title: string;
 type: "html";
 content: string;
}

export const chapters: Chapter[] = [
 {
 id: "euler-path-vs-cycle",
 title: "Эйлер: Путь ≠ Цикл (ты путаешь!)",
 type: "html",
 content: `<<section id="euler-path-vs-cycle" class="`
 <div class="flex items-center mb-6">
 <span class="bg-indigo-600 text-white px-4 py-1`
 <h2 class="text-3xl font-bold text-white">He 6y,
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord`
 <h3 class="text-xl font-bold text-blue-400`
```



```
<p class="text-slate-300 text-sm mb-4">
 Представь, что ты альпинист. Ты спу
 Из пещеры и нет других выходов — то
 Если верёвка (ребро v-u) оборвётся
 Если бы из пещеры и был чёрный ход
</p>
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-rose-500 shadow-md">
 Аналогия: Кровеносный сосуд
</div>
<p class="text-slate-300 text-sm mb-4">
 Граф — это кровеносная система. Ребра (вершины и ребра) — это не мост, живём.
 Если же перерезать единственную артерию, то организм умрёт.
 Алгоритм DFS ищет такие «критические» ребра.
</p>
</div>
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border-rose-500 shadow-md">
 <summary class="p-4 font-bold text-slate-400">
 Полный код на C++ (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-4">
 <p>Классическая реализация — <code class="text-slate-300 text-sm">
 оходов.</p>
```

```
 <div class="bg-slate-950 p-4 rounded-lg border-rose-500 shadow-md">
 <pre class="text-xs font-mono text-slate-300">
```

```
#include <vector>
#include <algorithm>

using namespace std;
```

```

} </pre>
 </div>
 <p class="text-xs text-slate-400">Запись
 </div>
</details>
</div>
</section>`,
 },
 {
 id: "planarity-euler-formula",
 title: "Планарность: K5, K3,3 и формула Эйлера",
 type: "html",
 content: `<section id="planarity-euler-formula" cla
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Планар
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400 m

 <div class="bg-slate-900 p-4 rounded-lg mb-6">
 <p class="text-lg text-blue-300 italic m
 <p class="text-3xl font-mono text-white"
 <div class="text-sm text-slate-400 spac
 <p><span class="text-blue-400 font-l
 <p><span class="text-emerald-400 fo
 <p><span class="text-rose-400 font-l
 </div>
 </div>
 </div>

 <p class="text-slate-300 text-sm">
 Следствие: Для планарн
ode>.
 Если ребер больше – граф гарантированно
 </p>
</div>

```



order-slate-700/50">

Душный мод: Код и корень DFS (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300 space-x-4">

<p class="text-blue-400 font-bold">Особый

<p>

Для стартовой вершины обхода правил (иначе то выше неё прыгать некуда). Поэтому для неё у него **больше 1 независимого ребенка**.

</p>

<div class="bg-slate-950 p-4 rounded-lg">

<pre class="text-xs font-mono text-slate-100">

```
void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++;
 int children = 0;

 for (int to : adj[v]) {
 if (to == p) continue;
 if (visited[to]) {
 low[v] = min(low[v], tin[to]);
 } else {
 dfs(to, v);
 low[v] = min(low[v], low[to]);
 if (low[to] >= tin[v] && p != -1)
 IS_CUTPOINT(v);
 ++children;
 }
 }
 if (p == -1 && children > 1)
 IS_CUTPOINT(v);
}
```

</div>

</div>

</details>

<div class="bg-indigo-950/60 p-6 rounded-xl border">

<h4 class="text-lg font-bold text-indigo-400">



```

 ...tickets6to13,
 ...tickets14to20,
 ...tickets21to24
].filter(c => c && c.id);

// Remove old 7, 11, 12, 13: graph-planar-colors, graph
const toRemove = ["graph-planar-colors", "graph-bridges

const dedup = new Map<string, Chapter>();
merged.forEach(c => {
 if (!toRemove.includes(c.id)) {
 // We prefer the newer versions (like tickets14_20
 dedup.set(c.id, c);
 }
});

// Add the new ones
if (newChapters) {
 newChapters.forEach(c => dedup.set(c.id, c));
}

const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
 const numA = parseInt(a.title.match(/(\d+)/)?.[0] ||
 const numB = parseInt(b.title.match(/(\d+)/)?.[0] ||
 return numA - numB;
});

export const chapters: Chapter[] = finalChapters;

export const activeVizIds = [
 "euler-path-vs-cycle", "bridges-code", "planarity-euler
 "graph-dfs-bfs", "stack-dfs", "queue-bfs", "heap-beam-se
 "intro", "dijkstra", "bellman-ford", "floyd", "aho-corasi
 "mst-kruskal", "mst-prima", "mst-boruvka", "string-kmp",
 "string-z-func", "segment-trees", "sparse-table", "dynam
];

```

```

 "Волна от источника: сначала все соседи, потом со
formal:
 "Очередь (FIFO). Кладём старт, достаём вершину —
 у рёбер.",
complexity: "O(V + E)",
demo: "bfs",
simulator: "queue-bfs",
},
{
 id: "dfs",
 labels: ["DFS", "обход в глубину", "поиск в глубину"],
 title: "DFS — обход в глубину",
 short:
 "Прёшь вперёд до упора, упёрся — откатываешься на
formal:
 "Стек (или рекурсия). Из DFS растут: времена вход
complexity: "O(V + E)",
demo: "dfs",
simulator: "stack-dfs",
},
{
 id: "binsect",
 labels: ["binsect", "бинсект", "бинарный поиск", "д
 title: "Бинарный поиск (и его самозванец binsect)",
 short:
 "Бинарный поиск НАХОДИТ готовый элемент в уже отс
 о не сортирует. «binsect» из конспекта — шутлив
 quicksort.",
formal:
 "Поиск одного элемента в отсортированном массиве —
 O(n log n). Бинарный поиск сортировку не выполн
complexity: "поиск: O(log n); сортировкой не являетс
},
{
 id: "heap",
 labels: ["куча", "кучу", "кучи", "куче", "кучей", "к
 title: "Куча (priority queue)",
 short:

```

```

 id: "trie",
 labels: ["бор", "бора", "боре", "бору", "бором", "т
 title: "Бор (trie, префиксное дерево)",
 short:
 "Дерево, где путь от корня по буквам и есть слово
 formal: "Из корня по символам; в вершине хранится ф.
 complexity: "поиск слова — $O(|\text{слова}|)$ ",
 demo: "trie",
 simulator: "aho-corasick",
},
{
 id: "bfs-on-trie",
 labels: [
 "обход дерева",
 "обход бора",
 "обход в ширину по бору",
 "какой-то обход",
 "обходом дерева",
 "конечный автомат",
 "конечного автомата",
 "автомат",
 "автомата",
],
 title: "Обход бора в ширину (тот самый «какой-то об
 short:
 "В Ахо–Корасике бор обходят именно BFS: суффиксн
 о есть строго по уровням.",
 formal:
 "Кладём в очередь детей корня (их ссылка = корень
 кладём с в очередь.",
 complexity: "O(суммарной длины словаря × алфавит)",
 demo: "trie-bfs",
 simulator: "aho-corasick",
},
{
 id: "suffix-link",
 labels: ["суффиксная ссылка", "суффиксные ссылки",
 title: "Суффиксная ссылка",

```

```

 complexity: "препроцессинг $O(n)$, запрос $O(1)$ ",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
},
{
 id: "sparse-table",
 labels: [
 "разреженная таблица", "разреженная таблица", "sp",
 "разреженная", "двоичные подьёмы", "двоичный подь",
],
 title: "Разреженная таблица (метод двоичных подьёмов)",
 short:
 "Заранее считаем ответ для всех отрезков длины 1,
 м.",
 formal: "ST[i][j] = op(ST[i][j-1], ST[i+2^(j-1)][j-1])",
 complexity: "препроцессинг $O(n \log n)$, запрос $O(1)$ ",
 demo: "sparse-table",
 simulator: "sparse-table",
},
{
 id: "segment-tree",
 labels: ["дерево отрезков", "дереве отрезков", "дер"],
 title: "Дерево отрезков",
 short:
 "Матрёшка из отрезков: корень отвечает за весь ма
 товых кусков.",
 formal: "Строится за $O(n)$, запрос и обновление — $O(\log n)$ ",
 demo: "segment-tree",
 simulator: "segment-trees",
},
{
 id: "bridge",
 labels: ["мост", "моста", "мосты", "мостов", "мостов"],
 title: "Мост",
 short:
 "Ребро, после удаления которого граф разваливается",
 formal: "Ребро (u,v) — мост $\iff \text{low}[v] > \text{tin}[u]$, где u —
 родитель v, и не существует ребро в родителя — нет).",

```

```

 ассив.",
 demo: "amortized",
 },
 {
 id: "np",
 labels: [
 "NP-полная", "NP-полнота", "NP-трудная", "NP-полнота",
 "класс P", "полином", "полиномиальное",
],
 title: "Класс NP и NP-полнота",
 short:
 "NP — «решение проверить легко, найти трудно» (суть NP-полноты в NP.",
 formal: "Задача NP-полна, если она в NP и к ней полнота",
 demo: "np",
 },
 {
 id: "potential",
 labels: ["потенциал", "потенциалы", "потенциалов",
 "потенциал"],
 title: "Потенциалы (перевзвешивание Джонсона)",
 short:
 "Трюк «поднять все дороги на одинаковую высоту»: для любого ребра (u, v)
 $w'(u, v) = w(u, v) + h(u) - h(v)$, где h — потенциал, разность потенциалов
 к путей сохраняется.",
 demo: "relax",
 simulator: "johnson-algo",
 },
 {
 id: "scc",
 labels: ["компонента сильной связности", "компоненты сильной связности",
 "компоненты"],
 title: "Компоненты сильной связности",
 short:
 "Группы вершин, где из каждой можно дойти до каждой другой",
 formal: "Косарайю: DFS с сохранением порядка выхода из вершин",
 complexity: " $O(V + E)$ ",
 demo: "scc",
 simulator: "scc-kosaraju",
 },

```

```

 short:
 "Решаем задачу через ответы на её меньшие копии и
formal:
 "Нужны: оптимальная подструктура + перекрывающиеся
 ция).",
demo: "dp",
simulator: "dynamic-programming",
},
{
 id: "shortest-path",
 labels: ["кратчайший путь", "кратчайшие пути", "кра
title: "Кратчайший путь",
short:
 "Самый дешёвый маршрут из A в B. «Дешёвый» – по с
formal:
 "Дейкстра – неотрицательные веса, $O(E \log V)$. Фор
 ы, $O(V^3)$.",
demo: "relax",
simulator: "dijkstra",
},
{
 id: "negative-cycle",
 labels: ["отрицательный цикл", "отрицательного цикл
title: "Отрицательный цикл",
short:
 "Круг, пройдя по которому вы становитесь «богаче»
formal:
 "Форд–Беллман: если на V -й итерации ещё удаётся с
demo: "relax",
simulator: "bellman-ford",
},
{
 id: "greedy",
 labels: [
 "жадный алгоритм", "жадный", "жадно", "жадная",
 "самое дешёвое ребро", "самое дешёвое ребро", "са
],
title: "Жадный алгоритм",

```

```

 "Три случая: zig (родитель – корень), zig-zig (уз
 рацию.",
 demo: "amortized",
 simulator: "splay-tree",
},
{
 id: "adjacency",
 labels: ["матрица смежности", "список смежности", "
 title: "Способы хранить граф",
 short:
 "Матрица смежности – таблица «есть ли ребро», быс
 й, экономно для разреженных графов.",
 formal: "Матрица: проверка $O(1)$, память $O(V^2)$. Спис
},
{
 id: "dijkstra",
 labels: ["Дейкстра", "Дейкстры", "Дейкстре", "Дейкс
 title: "Алгоритм Дейкстры",
 short:
 "Жадный «навигатор»: всегда раскрываем ближайший
 льные.",
 formal:
 "Куча хранит пары (расстояние, вершина). Достали
 complexity: " $O(E \log V)$ с бинарной кучей",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "bellman-ford",
 labels: ["Форд-Беллман", "Форда-Беллмана", "Беллман
 title: "Алгоритм Форда–Беллмана",
 short:
 "Тупой, но честный: $V-1$ раз проходим по ВСЕМ рёбра
 ",
 formal:
 "После k -й итерации гарантированно верны все крат
 ось – есть отрицательный цикл.",
 complexity: " $O(V \cdot E)$ ",

```

```

 "Все компоненты одновременно шлют «гонца» к ближайшей вершине.
 formal:
 "Фаза: для каждой компоненты ищем минимальное исключающее
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst-boruvka",
},
{
 id: "kruskal",
 labels: ["Краскал", "Краскала", "Краскалу", "Kruskal"],
 title: "Алгоритм Краскала",
 short:
 "Сортируем все рёбра по весу и берём подряд те, которые не образуют
 formal:
 "Сортировка O(E log E) + E операций union/find. Сложность
 complexity: "O(E log E)",
 demo: "mst",
 simulator: "mst-kruskal",
},
{
 id: "components",
 labels: [
 "компонента связности", "компоненты связности", "компонент",
 "компоненте связности", "связности", "связный", "связность",
],
 title: "Компоненты связности",
 short:
 "Граф-архипелаг: острова, между которыми нет мостов.
 бой.",
 formal:
 "Считаются одним проходом: пока есть непосещённая вершина,
 complexity: "O(V + E)",
 demo: "components",
 simulator: "graph-dfs-bfs",
},
{
 id: "graph",
 labels: [

```



```

 id: "bst",
 labels: ["бинарное дерево поиска", "дерево поиска",
 title: "Бинарное дерево поиска",
 short:
 "Слева всё меньше корня, справа — всё больше. Пои
 formal:
 "Без балансировки дерево может вырождаться в списо
 complexity: "O(высоты): от $O(\log n)$ до $O(n)$ ",
 demo: "segment-tree",
},
{
 id: "inclusion-exclusion",
 labels: [
 "включений-исключений", "включения-исключения", "
 "вырезание прямоугольников",
],
 title: "Включения-исключения на префиксных суммах",
 short:
 "Чтобы получить сумму прямоугольника, берём больш
 рый отрезали дважды.",
 formal:
 "Sum(x1..x2, y1..y2) = P[x2][y2] - P[x1-1][y2] -
 complexity: "запрос $O(1)$ после $O(nm)$ предподсчёта",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
},
{
 id: "reduction",
 labels: ["сведение", "сведения", "сведении", "сведён
 title: "Сведение задач",
 short:
 "«Я не умею решать A, зато умею B». Если A можно
 formal:
 "A $\leq_p$ B: есть полиномиальная функция, переводящая
 NP-полную задачу к новой.",
 demo: "np",
},
{

```

```

 id: "zig",
 labels: ["Zig", "Zag", "зиг"],
 title: "Zig – одиночный поворот",
 short:
 "Самый простой случай: родитель x уже сидит в корн
 канчивается, если глубина была нечётной.",
 formal: "Применяется ровно один раз за всю операцию",
 demo: "zig",
 simulator: "splay-rotations",
},
{
 id: "zig-zig",
 labels: ["Zig-Zig", "зиг-зиг", "ZigZig"],
 title: "Zig-Zig – x и родитель с одной стороны",
 short:
 "Дерево вытянулось в «бамбук»: g → p → x идут в о
 м (p, x). Именно это вдвое сокращает глубину вс
 formal:
 "Если крутить наивно (два раза поднимать x), бамб
 demo: "zig-zig",
 simulator: "splay-rotations",
},
{
 id: "zig-zag",
 labels: ["Zig-Zag", "зиг-заг", "ZigZag", "зигзаг"],
 title: "Zig-Zag – x и родитель с разных сторон",
 short:
 "Узлы идут «змейкой»: p – левый сын g, а x – прав
 p и g становятся двумя детьми x.",
 formal: "Эквивалентно двойному повороту AVL-дерева",
 demo: "zig-zag",
 simulator: "splay-rotations",
},
];

/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) :

```

```

title: "Билет 14. Дейкстра",
type: "html",
category: "Графы",
content: `<section id="dijkstra-content" class="mb-
<div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Дейкстра
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 Аналогия 1: Позитивное планирование
 </h3>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 (нельзя поехать и заработать). Он всегда в
 <p class="text-xl font-mono text-white">
 <div id="slot-chapter-image-dijkstra">
 </div>
 </p>
 </div>
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Позитивное планирование
 </div>
 <p class="text-slate-300 text-sm mb-4">
 (нельзя поехать и заработать). Он всегда в
 <div id="slot-chapter-image-dijkstra">
 </div>
 </div>
 </div>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 (нельзя поехать и заработать). Он всегда в
 <p class="text-xl font-mono text-white">
 <div id="slot-chapter-image-dijkstra">
 </div>
 </p>
 </div>
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-4">
 (нельзя поехать и заработать). Он всегда в
 <div id="slot-chapter-image-dijkstra">
 </div>
 </div>
</div>
`

```

```

<div class="grid grid-cols-1 xl:grid-cols-2"
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 ♂ Аналогия 1: Параноик
 </div>
 <p class="text-slate-300 text-sm mb
еряет ВСЕ возможные дороги V-1 раз подряд.
 <div id="slot-chapter-image-bellman
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Отрицательный цикл
 </div>
 <p class="text-slate-300 text-sm mb
г станет ЕЩЕ короче – значит мы попали во в
 </div>
 </div>

 <div id="slot-bellman-viz" class="mt-8"></div>
</div>
</section>`
},
{
 id: "floyd",
 title: "Билет 16. Флойд-Уоршелл",
 type: "html",
 category: "Графы",
 content: `<section id="floyd-content" class="mb-12"
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Флойд

```

```

 },
 {
 id: "aho-corasick",
 title: "Билет 23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `<section id="aho-corasick" class="mb-12 s
<div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-indigo-600 text-white px-4 py-1 round
 <h2 class="text-2xl sm:text-3xl font-bold text-white
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-blue-400 mb-4">

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text
 <p class="text-sm text-blue-300 italic mb-2">0с
 <p class="text-lg font-mono text-white">Бор (Tr
 </div>
 <p class="text-slate-300 text-sm">Позволяет найти
 -mono text-amber-300 bg-black/30 px-1 rounded">
 y.</p>
 </div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border bo
 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">
 Аналогия 1: Паутина (Бор)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 мы двигаемся по веткам. Если нужного продолж
 ("нити страховки") в самый длинный из извест
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2

```

```

while (!q.empty()) {
 int v = q.front(); q.pop();
 for (auto const& [ch, child] : nodes[v].trie) {
 // Суффиксная ссылка ребёнка – это переход
 nodes[child].link = nodes[nodes[v].link].go

 // Если суффиксная ссылка сама является сло
 // Иначе наследуем terminal_link
 if (nodes[nodes[child].link].is_terminal) {
 nodes[child].term_link = nodes[child].l
 } else {
 nodes[child].term_link = nodes[nodes[ch
 }

 q.push(child);
 }
}

</div>
</div>
</details>
</div>
</section>`
},
{
 id: "alg-map",
 title: "Алгоритмы на карте",
 type: "html",
 category: "Графы",
 content: `<section id="alg-map-content" class="mb-1
<div class="flex items-center mb-6">
 <span class="bg-purple-600 text-white px-4 py-1
 <h2 class="text-3xl font-bold text-white">Алгори
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord

```

```

 options: [
 "Потому что Гамильтон — болезнь, и лечить её тяжко",
 "Потому что для эйлерова цикла есть простой критерий",
 "Потому что Эйлер был умнее Гамильтона"
],
 correctIndex: 1,
 explanation: "Для эйлерова цикла работает критерий Эйлера. Это NP-полная задача, которую не решают.",
},
{
 question: "Куда ведёт эйлеров путь, если нечётных вершин больше 2?",
 options: [
 "Всё ещё можно пройти по всем рёбрам ровно раз",
 "Такого графа не существует",
 "Эйлерова пути нет — нужно удалять лишние рёбра"
],
 correctIndex: 2,
 explanation: "Если нечётных вершин больше 2, эйлерова пути нет. Если ровно 2 нечётные или 0, то путь есть."
}
],
"bridges-code": [
 {
 question: "После DFS у нас low[u] = 4, а tin[v] = 2. Это мост?",
 options: [
 "Да, так как low[u] (4) > tin[v] (2)",
 "Нет, low[u] должно быть меньше или равно",
 "Только если граф связный и неориентированный"
],
 correctIndex: 0,
 explanation: "Условие моста: low[u] > tin[v]. В данном случае 4 > 2, значит, это мост."
 },
 {
 question: "Для чего нужен массив low[v]?",
 options: [
 "Для того чтобы было больше переменных в коде",
 "Чтобы знать минимальный tin предка, до которого нужно вернуться",
 "Чтобы хранить глубину рекурсии"
]
 }
]

```







```

title: "3. Декартово дерево (Treap)",
type: "html",
description: "Дерево поиска по ключу и Куча по приоритету",
category: "Продвинутые структуры",
content: `
<section id="treap" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 3
 <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">Декартово дерево (Treap)
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Описание
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Декартово дерево (Treap) — это бинарное дерево поиска, которое сочетает в себе свойства бинарного дерева поиска и кучи. Оно поддерживает операции поиска, вставки и удаления элементов за время O(log n).
 <p class="text-xl font-mono text-white">Основа: Бинарное дерево поиска + Свойства кучи.
 <p class="text-xl font-mono text-white">метода: Split и Merge.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Удар катаной (Split)
 </div>
 <p class="text-slate-300 text-sm mb-2">Представим, что мы хотим удалить элемент X из дерева. Мы делаем «разрез» по значению X. Все, кто меньше X улетают в левое поддерево, а все, кто больше X, остаются в правом.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия 2: Слияние капель (Merge)
 </div>
 <p class="text-slate-300 text-sm mb-2">После удаления элемента X, мы должны восстановить свойства бинарного дерева поиска. Мы просто смотрим на корни: у кого Y меньше, того делаем левым, а у кого Y больше, того делаем правым. Мы просто смотрим на корни: у кого Y меньше, того делаем левым, а у кого Y больше, того делаем правым.
 </div>
 </div>
 </div>
 <div class="bg-slate-900/40 rounded-lg p-6">
 <h3 class="text-xl font-bold text-blue-400">Описание
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Декартово дерево (Treap) — это бинарное дерево поиска, которое сочетает в себе свойства бинарного дерева поиска и кучи. Оно поддерживает операции поиска, вставки и удаления элементов за время O(log n).
 <p class="text-xl font-mono text-white">Основа: Бинарное дерево поиска + Свойства кучи.
 <p class="text-xl font-mono text-white">метода: Split и Merge.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Удар катаной (Split)
 </div>
 <p class="text-slate-300 text-sm mb-2">Представим, что мы хотим удалить элемент X из дерева. Мы делаем «разрез» по значению X. Все, кто меньше X улетают в левое поддерево, а все, кто больше X, остаются в правом.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия 2: Слияние капель (Merge)
 </div>
 <p class="text-slate-300 text-sm mb-2">После удаления элемента X, мы должны восстановить свойства бинарного дерева поиска. Мы просто смотрим на корни: у кого Y меньше, того делаем левым, а у кого Y больше, того делаем правым. Мы просто смотрим на корни: у кого Y меньше, того делаем левым, а у кого Y больше, того делаем правым.
 </div>
 </div>
 </div>
 </div>
</section>
`

```

```

<h3 class="text-xl font-bold text-blue-400">
<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
(Zig, Zig-Zig, Zig-Zag), гарантируя амортизацию
</div>
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: VIP-лифт
 </div>
 <p class="text-slate-300 text-sm mb-4">
вися корнем дерева (VIP-статус). Если ты член,
ка почти не требуется времени!</p>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
border-rose-500 shadow-md">
 Аналогия 2: Балансировка через бамбук
 </div>
 <p class="text-slate-300 text-sm mb-4">
и становиться кривым как бамбук. Но как толстый
</p>
 </div>
</div>
<details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300
-b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
// Zig, Zag
function rotate(x) {
 let p = x.parent;
 if (p.left === x) { // Right rotate
 p.left = x.right;

```

```

 </div>
 </div>
</section>`
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сжатие суммы подматрицы за O(1)",
 category: "Алгоритмы матрицы",
 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 ≈ Аналогия 1: Вырезание прямоу
 </div>
 <p class="text-slate-300 text-sm mb
 з него маленький целевой прямоугольник в цел
 ый верхний угол отрезается дважды! Поэтому
 </div>
 </div>
 </div>
 </div>
 </div>
</div>
</section>`

```

```

 (нельзя поехать и заработать). Он всегда в
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-2">
дный и не умеет пересматривать уже "зафиксир
 </div>
 </div>
 <div id="slot-dijkstra-viz" class="mt-8"></div>
 </div>
</div>
</section>`
 },
 {
 id: "bellman-ford",
 title: "15. Графы. Поиск кратчайшего пути. Форд-Беллман",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="bellman-ford-content" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">15.
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-rose-400/50">
 <h3 class="text-xl font-bold text-rose-400">15.1. Алгоритм Форда-Беллмана
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">Алгоритм Форда-Беллмана
 <p class="text-xl font-mono text-white">
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4

```

```

<div class="bg-slate-800 p-6 rounded-lg border-2
 <div class="absolute -top-3 left-4 bg-slate-800
 border-emerald-500 shadow-md">
 Суть фокуса (По шагам)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Главный герой – внешний цикл
 шаг class="bg-slate-900 text-pink-400"
 шу себе проходить через вершину k?»</i>
 </p>
 <ul class="text-sm text-slate-300 space-y-2">
 Шаг k=0:Шаг k=1:class="text-indigo-300"
 Шаг k=2:Шаг k=N:

```

```

<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 (проверяем через DSU) - берем в ответ.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия: Прокладка интернет
 </div>
 <p class="text-slate-300 text-sm mb-4">
с самых дешевых дорог в стране". Он строит
единую сеть.</p>
 </div>
 </div>
 </div>
 </div>
</section>`
 },
 {
 id: "mst-prima",
 title: "19. Графы. Остовное дерево. Прима",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ается с соседом.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 border-emerald-500 shadow-md">
 Аналогия: Племена
 </div>
 <p class="text-slate-300 text-sm mb-4">
 изкому племени для объединения. К вечеру пл
 ства шлют гонцов к ближайшему чужому царству.
 </div>
 </div>
 </div>
 </div>
 </section>`
 }
};
```

---

ФАЙЛ 25/83: src/data/tickets/ticket21\_24.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 262 | РАЗМ

---

```
import { Chapter } from '../types';
```

```
export const tickets21to24: Chapter[] = [
 {
```

```
 id: "string-kmp",
```

```
 title: "21. Префикс-функция (п) — Взгляд назад (КМП)",
```

```
 type: "html",
```

```
 category: "Строки",
```

```
 content: `
```

```
<section id="string-kmp" class="mb-12 scroll-mt-10">
```



акого места этого слова мы можем продолжить

</p>

</div>

</div>

<details class="bg-slate-900/40 rounded-lg

<summary class="p-4 font-bold text-slate

-b border-slate-700/50">

Пример вычисления (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300

<p>Тот же пример: <b>abcabcd</b></p>

<ul class="list-disc pl-5 space-y-2

<li><b>a</b></li> Префиксов нет. <code>

<li><b>ab</b></li> Из начала ("a")

>

<li><b>abc</b></li> Опять мимо. <code>

<li><b>abca</b></li> 0! Начало ("a"

</code></li>

<li><b>abcab</b></li> Начало ("ab")

/li>

<li><b>abcabc</b></li> Начало ("abc

e></li>

<li><b>abcabcd</b></li> "d" всё исп

"abca" vs "abcd"). Конец "abcd" не совпада

</ul>

</div>

</details>

<details class="bg-slate-900/40 rounded-lg

<summary class="p-4 font-bold text-slate

-b border-slate-700/50">

Код C++ (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300

<pre class="bg-slate-950 p-4 rounded

vector<int> pi(s.length());

for (int i = 1; i < s.length(); i++) {

int j = pi[i-1];

Тут мы берём всю строку и «прикидываю начну читать строку отсюда, насколько длинная».

Это самый быстрый способ найти `Z[i]`, считаешь Z-функцию, и там, где `Z[i]`

<!-- Аналогия 2 -->

<div class="bg-slate-900 p-6 rounded-lg">

<div class="absolute -top-3 left-4 border border-rose-500 shadow-md">

Аналогия 2: LLM Самокопираст

</div>

<p class="text-slate-300 text-sm mb">

В LLM Z-функция может применяться `[i]` (где `i` указывает назад на сама себя.

</p>

</div>

</div>

<details class="bg-slate-900/40 rounded-lg">

<summary class="p-4 font-bold text-slate-300 border border-slate-700/50">

Пример вычисления (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300">

<p>Тот же пример: **abcabcd**</p>

<ul class="list-disc pl-5 space-y-2">

<li>`Z[0]` – не считаем

<li>`Z[1]` (**b**cabcd) `[1] = 0`</li>

<li>`Z[2]` (**c**abcd) `[2] = 0`</li>

<li>`Z[3]` (**a**bcda) `[3] = 0`</li>  
? Нет, в начале 'a', тут 'd'). Совпало 3 буквы

<li>`Z[4]`, `Z[5]`, `Z[6]`</li>

</ul>

```
 <p class="text-sm text-blue-300 italic mb-2">0с
 <p class="text-lg font-mono text-white">Бор (Тр
 </div>
 <p class="text-slate-300 text-sm">Позволяет найти
 го один проход.</p>
</div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border bo
 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">
 Аналогия 1: Паутина (Бор)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 ет — мы падаем по суффиксной ссылке ("
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2
 <div class="absolute -top-3 left-4 bg-rose-900
 -500 shadow-md">
 Аналогия 2: Матёшка (Терминальные)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 нашли и "he", потому что оно спрятано внутри
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg border b
 <summary class="p-4 font-bold text-slate-400 ou
 r-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-
 <pre class="bg-slate-950 p-4 rounded overfl
int get_link(int v) {
 if (t[v].link == -1) {
 if (v == 0 || t[v].p == 0) t[v].link = 0;
```

```

 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4
rder border-emerald-500 shadow-md">
 Аналогия 1: Судoku (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb
пример сортировка чисел). Класс NP –
енную сетку (ответ), он БЫСТРО (за класс P)
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4
rder border-purple-500 shadow-md">
 Аналогия 2: Машина-переводчик
 </div>
 <p class="text-slate-300 text-sm mb
ь отличный переводчик на английский (Сведен
аче В, то В – это NP-Полная (сосредо
 </div>
 </div>
 </div>
 </section>`
 }
];

```

ФАЙЛ 26/83: src/data/tickets/ticket6\_13.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 486 | РАЗМ

```
import { Chapter } from "../../types";
```

```
export const tickets6to13: Chapter[] = [
```

```
 {
 id: "graph-dfs-bfs",
 title: "6. Графы. Представление, DFS, BFS",

```

```

{ /* DFS vs BFS */}
<div class="bg-slate-700/50 p-6 rounded-xl border-2 border-indigo-500 shadow-md">
 <h3 class="text-xl font-bold text-indigo-400">DFS vs BFS</h3>

 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <!-- Аналогия DFS -->
 <div class="bg-slate-800 p-6 rounded-lg border border-indigo-500 shadow-md">
 <div class="absolute -top-3 left-4 right-4 bottom-4">
 ♂ DFS (Поиск в глубину) = Жадный алгоритм
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Что делает: Жадный алгоритм ищет путь к целевой вершине (например, минимальную по номеру) и пробует другой путь.
 </p>
 <ul class="text-xs text-indigo-300">
 Где используется:
 Топологическая сортировка (линейное время)
 Поиск мостов и точек сочленения
 Сильно связанные компоненты Киркмана
 Поиск циклов и просто проверка

 </div>

 <!-- Аналогия BFS -->
 <div class="bg-slate-900 p-6 rounded-lg border border-emerald-500 shadow-md">
 <div class="absolute -top-3 left-4 right-4 bottom-4">
 ♂ BFS (Поиск в ширину) = Волна
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Что делает: Концентрические волны от начальной вершины. Работает через Очередь (FIFO). Продвигается до тех пор, пока не достигнет целевой вершины.
 </p>
 <ul class="text-xs text-emerald-300">
 Где используется:
 Кратчайший путь в Невзвешенном графе

 </div>
 </div>

```

```

} </pre>
 </div>
 </details>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 7
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">Графы. Планарные. Покраска
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Теорема о 4 красках
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Любой планарный граф можно раскрасить 4 цветами.
 <p class="text-xl font-mono text-white">
 ый граф можно раскрасить 4 цветами.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb-4">
 . Границы не пересекаются в одной точке по-
 ы не сливались цветами, Всегда можно всего 4
 </p>
 </div>
 </div>
 </div>
 </div>
`
 }
]

```

```

 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Разложение задач по порядку выполнения",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 перед.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Учеба в вузе
 </div>
 <p class="text-slate-300 text-sm mb-4">
 еская сортировка – это расписание, которое
 >
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Ограничения: Циклы
 </div>
 <p class="text-slate-300 text-sm mb-4">
 получить опыт, нужна работа". Алгоритм выяв

```

```

 },
 {
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая.",
 category: "Графы. Продвинутые",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-emerald-400">Макро-вершины
 <p>Макро-вершинами называются компоненты сильной связности (SCC) ориентированного графа, в котором каждая вершина принадлежит ровно одной SCC. SCC — это максимальные подграфы, в которых каждая вершина достижима из каждой другой вершины.
 </div>

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Пример: Рассмотрим граф с 4 вершинами (A, B, C, D) и следующими рёбрами: A → B, B → C, C → A, C → D. В этом графе есть две SCC: {A, B, C} и {D}.
 </div>

 <p class="text-slate-300 text-sm mb-4">
 «Это просто цикл?» – Нет!
 </p>

 <ul class="list-disc list-inside text-sm text-slate-300">
 Если у тебя есть цикл <code>A → B → C → A</code>, то это SCC.
 Если ты добавишь вершину Г с рёбрами G → A и G → B, то SCC останется той же.
 Правило: Если два цикла имеют хотя бы одну общую вершину, то они принадлежат одной SCC.

 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg">
 <h4 class="text-emerald-400">Алгоритм Косарая
 <p>Алгоритм Косарая позволяет найти все SCC в ориентированном графе за O(V + E) времени. Он основан на двух проходах по графу: первый проход — поиск компонент сильной связности, второй — топологическая сортировка графа.
 </div>
 <div class="bg-slate-800 p-6 rounded-lg">
 <h4 class="text-emerald-400">Пример работы
 <p>Рассмотрим тот же граф, что и раньше. В первом проходе мы находим SCC {A, B, C} и {D}. Во втором проходе мы топологически сортируем граф, получая порядок D, C, B, A.
 </div>
 </div>
 </div>
`
 }
],
 footer: `
<div class="text-slate-400 text-sm">
 <p>© 2024. Все права защищены. Этот документ является частью учебного пособия по алгоритмам на графах.
 </p>
</div>
`
}

```



```

 !

 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых расхреначивает",
 category: "Графы. Продвинутые",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
и fup[v] > tin[u].</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Единственный мост
 </div>
 <p class="text-slate-300 text-sm mb-4">
 этот мост, экономикой обоих кусков придет
 </div>
 </div>
 </div>
 </div>
 </div>
`

```

```
 </p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border border-slate-500 shadow-md">
 Аналогия: Главный перекресток
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Представь город, где два района соединены, но её перекроют (удалят вершину) – районы будут хрупкостью системы.
 </p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border border-emerald-500 shadow-md">
 Телеком: Центральный свитч
 </div>
 <p class="text-slate-300 text-sm mb-4">
 У тебя несколько компьютеров в одной абелем (мостом). Сами свитчи – это точки со
 </p>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
 <summary class="p-4 font-bold text-slate-400">
 Душный мод: Код и корень DFS (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-2">
 <p class="text-blue-400 font-bold">Особ
 </p>
 Для стартовой вершины обхода правил (то выше неё прыгать некуда). Поэтому для не
```

```

 Как точка сочленения рушит :
 Косарайю, свернув каждую SCC в одну супер-в
 (сделаем неориентированным), то любая
 о и есть критическая SCC! Если удалить эту
 . Одно слабое звено изолирует целые касты S

</div>
</div>
</section>`,
},
{
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-indigo-400
 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-indigo-300 italic
 <p class="text-xl font-mono text-white"
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Рисование не отрыв
 </div>
 <p class="text-slate-300 text-sm mb
 е сходится нечетное число линий, значит, из
 а везде должно быть четное число (зашел-выш

```

```

 { id: 4, x: 550, y: 150, label: "4" },
 { id: 5, x: 700, y: 50, label: "5" },
 { id: 6, x: 700, y: 250, label: "6" },
];

const edges: Edge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 },
 { u: 3, v: 4 }, // 3 and 4 are articulation points (3
 { u: 4, v: 5 },
 { u: 5, v: 6 },
 { u: 6, v: 4 },
];

// Adjacency list
const adj: number[][] = Array.from(
 { length: Object.keys(nodes).length },
 () => [],
);

edges.forEach((e) => {
 adj[e.u].push(e.v);
 adj[e.v].push(e.u);
});

interface StepInfo {
 desc: string;
 visited: Set<number>;
 ap: Set<number>;
 currentNode: number | null;
 tin: number[];
 low: number[];
}

export const ArticulationPointsViz: React.FC = () => {
 const [steps, setSteps] = useState<StepInfo[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt

```

```

 });
 } else {
 children++;
 dfs(to, v);
 currentLow[v] = Math.min(currentLow[v], currentTin[v]);

 recordStep(
 `Возврат в ${v} из ${to}. Обновляем low[${v}]`,
 v,
);

 if (currentLow[to] >= currentTin[v] && p !== ap.add(v)) {
 recordStep(
 `Найдена точка сочленения: вершина ${v}`,
 v,
);
 }
 }
}

if (p === -1 && children > 1) {
 ap.add(v);
 recordStep(
 `Корень дерева DFS ${v} имеет >1 детей. Это`,
 v,
);
} else if (p === -1) {
 recordStep(
 `Корень дерева DFS ${v} имеет <=1 ребенка, он`,
 v,
);
}
};

for (let i = 0; i < nodes.length; i++) {
 if (!visited.has(i)) {
 dfs(i);
 }
}

```

```

 <Info className="w-5 h-5 text-rose-400" />
 Моделирование поиска точек сочленения
 </h3>

 <div className="flex items-center gap-2 bg-slate-500" >
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-2 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 className="flex items-center justify-center gap-2 text-white"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-3 gap-4" >
 <div className="lg:col-span-2 relative h-96 lg:h-128" >
 <svg className="w-full h-full" viewBox="0 0 800 800" >
 {/* Edges */}
 {edges.map((edge, i) => {
 const u = nodes.find((n) => n.id === edge.u)
 const v = nodes.find((n) => n.id === edge.v)

 const isBridge =
 (edge.u === 3 && edge.v === 4) ||

```

```

return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r={r}
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-all duration-300"
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="14px"
 fontWeight="bold"
 className="select-none pointer-events-none"
 >
 {node.label}
 </text>

 {isVisited && (
 <>
 <text
 x={node.x}
 y={node.y - r - 15}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="10px"
 >
 tin:{" "}
 {currentStep.tin[node.id] >= 0
 ? currentStep.tin[node.id]
 : "-"}
 </text>
 </>
)}
 </g>
)

```

```

 <div className="flex items-center gap-2">
 <div className="w-3 h-3 rounded-full bg-rose-500">
 Посещена
 </div>
 </div>
 <div className="flex items-center gap-2">
 <div className="w-3 h-3 rounded-full bg-rose-500">
 Текущая
 </div>
 </div>
 <div className="flex items-center gap-2">
 <div className="w-8 h-1 bg-rose-500 border-1px border-slate-800">
 Мост
 </div>
 </div>
 </div>

 <div>
 <h5 className="text-xs text-slate-500 font-medium">
 ЛОГ ДЕЙСТВИЙ:
 </h5>
 <div className="space-y-2">
 {steps
 .slice(0, currentStepIndex + 1)
 .reverse()
 .map((step, idx) => {
 <div
 key={idx}
 className={`text-xs p-2 rounded $
xt-slate-500`} `>
 >
 {step.desc}
 </div>
 </div>
 })}
 </div>
 </div>
 </div>
 <div className="mt-4 shrink-0">
 <div className="w-full bg-slate-800 h-2 rounded">
 <div
 className="bg-rose-500 h-full transition"

```



```

const nodes: Node[] = [
 { id: 'S', x: 60, y: 150, name: 'База (S)' },
 { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
 { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
 { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
 { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
 { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
 { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
];

const getEdges = (cycle: boolean): Edge[] => {
 const defaultEdges = [
 { u: 'S', v: 'A', w: 4 },
 { u: 'S', v: 'B', w: 5 },
 { u: 'B', v: 'C', w: -2 },
 { u: 'A', v: 'C', w: 1 },
 { u: 'C', v: 'D', w: 3 },
 { u: 'C', v: 'E', w: 4 },
 { u: 'D', v: 'F', w: 2 },
 { u: 'E', v: 'F', w: 1 },
];
 if (cycle) {
 return [...defaultEdges, { u: 'D', v: 'A', w: -6 }];
 } else {
 return [...defaultEdges, { u: 'D', v: 'A', w: -2 }];
 }
};

const edges = getEdges(hasCycle);

const getAdjList = (ed: Edge[]) => {
 const list: { [key: string]: { v: string; w: number } } = {};
 nodes.forEach(n => (list[n.id] = []));
 ed.forEach(e => list[e.u].push({ v: e.v, w: e.w }));
 return list;
};

const adjList = getAdjList(edges);

```

```

 }

 simulationSteps.push({
 iteration: iter, checkingEdge: { u: edge.u, v: edge.v },
 distances: { ...dist }, previous: { ...prev },
 log: `Итерация ${iter}: Проверяем ${edge.u} → ${edge.v}.
 updated ? ` Релаксация успешна! dist[${edge.u}] = dist[${edge.v}] + edge.w`,
 activeCodeLine: updated ? 6 : 5,
 });
 }

 if (!anyUpdate && iter < vCount - 1) {
 simulationSteps.push({
 iteration: iter, checkingEdge: null, distances: { ...dist },
 log: ` Ранний выход после итерации ${iter}.`,
 activeCodeLine: 10,
 });
 break;
 }
}

if (simulationSteps[simulationSteps.length - 1].iteration === vCount) {
 let cycleFound = false;
 for (const edge of edges) {
 if (dist[edge.u] !== 999 && dist[edge.u] + edge.w < dist[edge.v]) {
 cycleFound = true;
 simulationSteps.push({
 iteration: vCount, checkingEdge: { u: edge.u, v: edge.v },
 distances: { ...dist }, previous: { ...prev },
 log: ` ВНИМАНИЕ! Проверка цикла: Ребро ${edge.u} → ${edge.v} обновляет ${edge.v}.`,
 activeCodeLine: 9,
 });
 break;
 }
 }
}

if (!cycleFound) {

```

```

 </div>

 <div className="flex items-center gap-3">
 <div className="bg-slate-900 p-1 rounded-lg b
 <button onClick={() => setHasCycle(false)} c
 bg-blue-600 text-white' : 'text-slate-400'}
 <button onClick={() => setHasCycle(true)} c
 r gap-1 ${hasCycle ? 'bg-rose-600 text-whit
 </div>
 <button onClick={() => { setIsPlaying(false);
 g-slate-600 text-white px-3 py-2 rounded-lg
 <button onClick={() => setIsPlaying(!isPlayin
 sition-colors text-white ${isPlaying ? 'bg-
 <button onClick={() => { setIsPlaying(false);
 ; }} className="flex items-center gap-1 bg-l
 s">⏮ar </button>
 </div>
 </div>

 <div className="flex flex-col lg:flex-row gap-6 m
 {/* Граф */}
 <div className="w-full lg:w-2/3 bg-blue-950/20
 y-center min-h-[400px]">
 <div className="absolute top-3 left-3 bg-blue
 ld shadow-sm">
 Итерация {currentStep.iteration} (⏮ar {curr
 </div>

 {currentStep.hasNegativeCycle && (
 <div className="absolute top-3 right-3 bg-r
 animate-pulse shadow-lg shadow-rose-600/50
 ▲ Отрицательный цикл!
 </div>
)}

 <svg className="w-full h-auto max-h-[500px]"
 <defs>
 <marker id="arrow-bf-normal" markerWidth=

```

```

 return (
 <g key={node.id} className="transition-
 <circle cx={node.x} cy={node.y} r={is
checkingNode ? '#7dd3fc' : '#64748b'} strokeWidth
 <text x={node.x} y={node.y + 5} fill=
 <text x={node.x} y={node.y - 28} fill=
 ight="bold" textAnchor="middle" className="
 <text x={node.x} y={node.y + 35} fill=
split(' ')[0]}</text>
 </g>
);
 }}}
 </svg>

 <div className="w-full bg-slate-950/80 p-4 ro
 <p className="font-semibold text-amber-400
 <p className="min-h-[40px] flex items-cente
 </div>
 </div>

 {/* Список смежности */}
 <div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2 text-sm font-mono t
 {Object.keys(adjList).map(u => {
 <div key={u} className="flex flex-wrap g
 -colors">
 <span className="font-bold text-white
 {adjList[u].map((edge, idx) => {
 const isChk = currentStep.checkingE
 return (
 <span key={idx} className={`px-2
edge.w < 0 ? 'bg-rose-900/80 text-rose-300
 {edge.v} ({edge.w})

)
 }
 }
)
 }
 }

```

```
 </div>
)})
 </div>
 </div>
</div>
</div>
</div>
);
}
```

---

ФАЙЛ 29/83: src/components/BfsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect, useCallback } from 'react'
import { Play, Pause, RotateCcw, StepForward } from 'lucide-react'
```

```
const NODES = [
 { id: 0, label: '0', x: 300, y: 50 },
 { id: 1, label: '1', x: 150, y: 150 },
 { id: 2, label: '2', x: 450, y: 150 },
 { id: 3, label: '3', x: 75, y: 250 },
 { id: 4, label: '4', x: 225, y: 250 },
 { id: 5, label: '5', x: 375, y: 250 },
 { id: 6, label: '6', x: 525, y: 250 },
];
```

```
const EDGES = [
 { from: 0, to: 1 }, { from: 0, to: 2 },
 { from: 1, to: 3 }, { from: 1, to: 4 },
 { from: 2, to: 5 }, { from: 2, to: 6 }
];
```

```
const ADJ: { [key: number]: number[] } = {
 0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [], 6: [],
};
```

```

 currentNode: null,
 logs: `Очередь не пуста, элементов: ${queue.length}
 });

 const curr = queue.shift()!;
 steps.push({
 activeLine: 5,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Достаем из очереди (${curr}).`
 });

 const neighbors = ADJ[curr];
 if (neighbors.length > 0) {
 for (const n of neighbors) {
 if (!visited.has(n)) {
 steps.push({
 activeLine: 7,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Рассматриваем соседа: ${n}. Он еще не
 });

 visited.add(n);
 queue.push(n);

 steps.push({
 activeLine: 9,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Помечаем ${n} как посещенную и добав.
 });
 }
 }
 } else {

```

```

useEffect(() => {
 if (!isPlaying) return;
 if (stepIdx >= STEPS.length - 1) {
 setIsPlaying(false);
 return;
 }
 const timer = setTimeout(handleNext, 1200);
 return () => clearTimeout(timer);
}, [isPlaying, stepIdx, handleNext]);

return (
 <div className="bg-slate-900 border-2 border-slate-800 p-4">
 <div className="flex flex-col md:flex-row items-center justify-between">
 <div>
 <h3 className="text-2xl font-bold text-white">
 Ал
 </h3>
 <p className="text-slate-400 text-sm mt-1">Ви
 </div>
 <div className="flex bg-slate-800 p-1.5 rounded">
 <button
 onClick={() => { setIsPlaying(false); setSt
 className="p-2 text-slate-400 hover:text-wh
 title="Сброс"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(p => !p)}
 className="p-2 text-blue-400 hover:text-bl
 title={isPlaying ? 'Пауза' : 'Воспроизведе
 >
 {isPlaying ? <Pause className="w-5 h-5" />
 </button>
 <button
 onClick={handleNext}
 disabled={stepIdx >= STEPS.length - 1}
 className="p-2 text-emerald-400 hover:text-

```

```

 if (isCurrent) {
 fill = '#3b82f6'; // blue-500
 stroke = '#60a5fa'; // blue-400
 scale = 1.3;
 } else if (isInQueue) {
 fill = '#059669'; // emerald-600
 stroke = '#34d399'; // emerald-400
 scale = 1.1;
 } else if (isVisited) {
 fill = '#0f172a'; // slate-900
 stroke = '#3b82f6'; // blue-500
 }

 return (
 <g key={n.id} className="transition-tran
 {isCurrent && {
 <circle cx={n.x} cy={n.y} r="30" fi
 }}
 <circle
 cx={n.x} cy={n.y}
 r="20"
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-colors durati
 />
 <text
 x={n.x} y={n.y}
 textAnchor="middle"
 dy=".3em"
 className="fill-white font-bold tex
 >
 {n.label}
 </text>
 </g>
);
 })))

```



```

 Состояние Очереди (Queue):
 <span className="text-xs text-slate-500"
 </div>
 <div className="flex items-center gap-2 min-h-10"
 {step.queue.length === 0 ? (
 <span className="text-slate-500 text-sm"
) : (
 step.queue.map(({qId, i}) => (
 <span key={` ${qId}-${i}`} className="text-sm"
 enter justify-center rounded-md shadow-lg
 {qId}

))
)}
 </div>
 <div className="mt-2 text-sm text-amber-300"
 ms-center justify-center">
 "{step.logs}"
 </div>
</div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 30/83: `src/components/ChapterImage.tsx`

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';
```

```
const imageMap: Record<string, { src: string; alt: string }> = {
 dijkstra: {
```

ФАЙЛ 31/83: src/components/ChapterNav.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React from "react";
import { ChevronLeft, ChevronRight } from "lucide-react";
import type { Chapter } from "../types";

interface Props {
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 compact?: boolean;
}

/**
 * Переходы «предыдущая / следующая тема» — как на Codecademy
 * Компактный вариант живёт в шапке главы, крупный — по центру
 */
export const ChapterNav: React.FC<Props> = ({ prev, next, index, total, onGo, compact }) => {
 if (compact) {
 return (
 <div className="flex items-center gap-2 text-xs">
 <button
 disabled={!prev}
 onClick={() => prev && onGo(prev.id)}
 title={prev ? prev.title : "Это первая тема"}
 className="flex items-center gap-1 px-2.5 py-1.5 rounded-sm
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 <ChevronLeft className="w-4 h-4" />
 Назад
 </button>

 {index + 1} / {total}

 </div>
);
 }
 return (
 <div className="flex items-center gap-2 text-sm">
 <button
 disabled={!prev}
 onClick={() => prev && onGo(prev.id)}
 title={prev ? prev.title : "Это первая тема"}
 className="flex items-center gap-1 px-3 py-2 rounded-sm
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 <ChevronLeft className="w-5 h-5" />
 Назад
 </button>

 {index + 1} / {total}

 </div>
);
};
```

```
 <div className="flex items-center justify-end
 text-indigo-400 mb-1">
 Следующая тема <ChevronRight className="w-3
 </div>
 <div className="text-sm font-bold text-slate-
 </button>
 })
 </nav>
};
};
```

---

ФАЙЛ 32/83: src/components/ChapterView.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useCallback, useEffect, useRef, useState } from "react";
import { Check, Download, Loader2, Maximize2, MousePointer } from "lucide-react";
import type { Chapter } from "../types";
import { useTermHints } from "../hooks/useTermHints";
import { TermPopover, type HintAnchor } from "../hints/TermHint";
import { getViz } from "../vizRegistry";
import { ChapterNav } from "../ChapterNav";
import { downloadChapterHtml } from "../utils/exportHtml";

interface Props {
 chapter: Chapter;
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 onOpenSimulator: (vizId: string) => void;
}

const isTouch = () => typeof window !== "undefined" && window.ontouchstart !== null;
```

```

 const termId = el.dataset.termId;
 if (!termId) return;
 cancelHide();
 setAnchor({ termId, rect: el.getBoundingClientRect() },
 },
 [cancelHide]
);

const handlePointerOver = (e: React.MouseEvent) => {
 if (isTouch()) return;
 const el = (e.target as HTMLElement).closest<HTMLElement>('.term');
 if (el) open(el);
};

const handlePointerOut = (e: React.MouseEvent) => {
 if (isTouch()) return;
 const el = (e.target as HTMLElement).closest<HTMLElement>('.term');
 if (el) scheduleHide();
};

const handleClick = (e: React.MouseEvent) => {
 const el = (e.target as HTMLElement).closest<HTMLElement>('.term');
 if (!el) return;
 e.preventDefault();
 if (anchor && anchor.termId === el.dataset.termId)
 else open(el);
};

const handleFocus = (e: React.FocusEvent) => {
 const el = (e.target as HTMLElement).closest<HTMLElement>('.term');
 if (el) open(el);
};

const viz = getViz(chapter.id);

return (
 <>
 <article className="chapter-body">

```

```

 <p className="mt-6 flex items-start gap-2 text-
 ed-lg px-3 py-2">
 <MousePointerClick className="w-4 h-4 shrink-0">

 Подчёркнутые термины – интерактивные: наведи
 объяснение на пальцах, мини-анимацию и кнопки

 </p>

 {viz && (
 <section id="chapter-viz" className="mt-8 scroll-mt-12">
 <div className="flex items-center justify-between">
 <h3 className="flex items-center gap-2 text-indigo-600">
 <Sparkles className="w-4 h-4 text-indigo-600" />
 Демонстрация: {viz.title}
 </h3>
 <button
 onClick={() => onOpenSimulator(chapterIndex)}
 className="flex items-center gap-1.5 text-white border-indigo-600
 ate-300 hover:text-white hover:border-indigo-600"
 >
 <Maximize2 className="w-3.5 h-3.5" /> Показать
 </button>
 </div>
 {viz.hint && <p className="text-xs text-slate-500">
 <viz.Component />
 </p>}
 </section>
)}

 <ChapterNav prev={prev} next={next} index={index} />
 </article>

 <TermPopover
 anchor={anchor}
 onClose={() => setAnchor(null)}
 onOpenSimulator={(id) => {
 setAnchor(null);
 }}
 />

```

```

{ line: ' // ЭТО МОСТ!', highlight: "normal" },
{ line: " }", highlight: "normal" },
{ line: " }", highlight: "normal" },
{ line: " }", highlight: "normal" },
{ line: "}", highlight: "normal" },
];

```

```

interface DfsStep {
 currentNode: number | null;
 visitedIndices: number[];
 tin: Record<number, number>;
 low: Record<number, number>;
 stack: number[];
 bridges: string[];
 log: string;
 activeEdge: [number, number] | null;
 backEdge: [number, number] | null;
 activeCodeLine: number[];
}

```

```

const GRAPH_NODES = [
 { id: 0, label: "A", x: 100, y: 120 },
 { id: 1, label: "B", x: 260, y: 120 },
 { id: 2, label: "C", x: 180, y: 200 },
 { id: 3, label: "D", x: 180, y: 300 },
 { id: 4, label: "E", x: 100, y: 370 },
 { id: 5, label: "F", x: 260, y: 370 },
 { id: 6, label: "G", x: 340, y: 200 }
];

```

```

const GRAPH_EDGES = [
 { u: 0, v: 1, key: "0-1" },
 { u: 1, v: 2, key: "1-2" },
 { u: 2, v: 0, key: "0-2" },
 { u: 2, v: 3, key: "2-3" },
 { u: 3, v: 4, key: "3-4" },
 { u: 4, v: 5, key: "4-5" },
 { u: 5, v: 3, key: "3-5" },

```

```

 },
 {
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:1}, sta
 activeEdge: null, backEdge: [2,0],
 log: "Видим соседа А (уже посещён)! Обратное ребро
 activeCodeLine: [7,8,9]
 },
 {
 currentNode: 3, visitedIndices: [0,1,6,2,3],
 tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3
 activeEdge: [2,3], backEdge: null,
 log: "Идём C→D. tin[D]=5, low[D]=5.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 4, visitedIndices: [0,1,6,2,3,4],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2
 activeEdge: [3,4], backEdge: null,
 log: "D→E. tin[E]=6, low[E]=6.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
 activeEdge: [4,5], backEdge: null,
 log: "E→F. tin[F]=7, low[F]=7.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
 activeEdge: null, backEdge: [5,3],
 log: "Из F видим D (посещён). Обратное ребро (F,D).
 activeCodeLine: [7,8,9]
 },
 {
 currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],

```





```

 className="transition-all duration-500"
 }}}
 {step.currentNode !== null && step.currentNode !== null ? (
 const n = GRAPH_NODES.find(x => x.id === step.currentNode.id);
 if (!n) return null;
 return <circle cx={n.x} cy={n.y} r={18} fill={step.currentNode.fill}
 >;
)}()
 {GRAPH_NODES.map(n => {
 const cur = step.currentNode === n.id;
 const vis = step.visitedIndices.includes(n.id);
 const st = step.stack.includes(n.id);
 return <g key={n.id}>
 <circle cx={n.x} cy={n.y} r={12}
 fill={cur ? "#10b981" : st ? "#064e3b" : "#f8d7da"}
 stroke={cur ? "#fff" : st ? "#34d399" : "#000"}
 strokeWidth={2.5}
 className="transition-all duration-300" />
 <text x={n.x} y={n.y+3} textAnchor="middle">
 {n.label}</text>
 {vis && (
 <div
 className="transition-all duration-300" />
 <rect x={n.x-18} y={n.y-28} width={36} height={56}
 className="transition-all duration-300" />
 <text x={n.x} y={n.y-20} textAnchor="center">
 {step.low[n.id] || "?"}</text>
 </div>
)}
 </g>
)}
)}
 </svg>
 <div className="flex items-center justify-between">
 Шаг {dfsIdx+1}/{DFS_STEPS.length}
 <div className="flex-1 mx-2 bg-slate-950 h-10 rounded">
 <div className="bg-indigo-600 h-full transition-all duration-300">
 </div>
 </div>
 </div>

```

```

 }`}>Вершина {GRAPH_NODES.find(n=>n.
 <span className="text-slate-500 m
 </div>
))
 }
 </div>
</div>

{/* Log */}
<div className="bg-indigo-950/10 border borde
 <p className="text-[11px] text-slate-300 le
 <div className="mt-2 pt-2 border-t border-s
 Мосты:</
 <span className="font-mono font-bold text
 {step.bridges.length > 0 ? step.bridges

 </div>
</div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 34/83: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect } from 'react';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
 currentNode: string | null;
 checkingEdge: { u: string; v: string } | null;
 distances: { [key: string]: number };
}

```

```

{ line: 1, text: 'function dijkstra(graph, start):'
{ line: 2, text: ' dist = {v: ∞}; dist[start] = 0'
{ line: 3, text: ' pq = PriorityQueue(); pq.push(start, 0)'
{ line: 4, text: ' while not pq.empty():' },
{ line: 5, text: ' d, u = pq.pop() // Извлекаем элемент с минимальным расстоянием'
{ line: 6, text: ' if d > dist[u]: continue'
{ line: 7, text: ' for v, weight in graph.neighbors(u):'
{ line: 8, text: ' if dist[u] + weight < dist[v]:'
{ line: 9, text: ' dist[v] = dist[u] + weight'
{ line: 10, text: ' pq.push(v, dist[v])'
{ line: 11, text: ' return dist // Все кратчайшие расстояния найдены'
};

```

```

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useState(0);
const [isPlaying, setIsPlaying] = useState(false);

```

```

useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist: Record<string, number> = { A: 0, B: 999 };
 const visited: Record<string, boolean> = { A: false, B: false };
 const prev: Record<string, string | null> = { A: null, B: null };

```

```

 simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ' Дейкстра готов к доставке. Начинаем из Пикет-Пост-Оффиса',
 activeCodeLine: 2,
 });

```

```

for (let iter = 0; iter < nodes.length; iter++) {
 let minD = 999;
 let u = null;
 for (const n of nodes) {
 if (!visited[n.id] && dist[n.id] < minD) {
 minD = dist[n.id];
 u = n.id;
 }
 }
}

```

```

simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ' Дейкстра всё доставил! Все возможные кратчайшие пути найдены! ',
 activeCodeLine: 11,
});

setSteps(simulationSteps);
}, []));

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;

if (!currentStep) return <div className="text-white">

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border-2 border-indigo-300">
 { /* Шапка симулятора */ }
 <div className="flex flex-wrap items-center justify-center gap-4">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-indigo-600 text-white rounded-md">

 </div>
 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-indigo-300">Полный код
 </p>
 </div>
 </div>
 </div>
);

```

```

<svg className="w-full h-auto max-h-[500px]"
 <defs>
 <marker id="arrow-dh-normal" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#475568" />
 </marker>
 <marker id="arrow-dh-active" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
 </marker>
 <marker id="arrow-dh-opt" markerWidth="6"
 <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
 </marker>
 </defs>
 {/* Пёпса */}
 {edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === edge.u)
 const vNode = nodes.find((n) => n.id === edge.v)
 const isChecking =
 currentStep.checkingEdge &&
 (currentStep.checkingEdge.u === edge.u &&
 currentStep.checkingEdge.v === edge.v)

 const isOptimal = currentStep.previous[edge.id] === 'optimal'

 let marker = 'url(#arrow-dh-normal)';
 if (isChecking) marker = 'url(#arrow-dh-active)';
 else if (isOptimal) marker = 'url(#arrow-dh-opt)';

 return (
 <g key={idx}>
 <line
 x1={uNode.x} y1={uNode.y}
 x2={vNode.x} y2={vNode.y}
 stroke={isChecking ? '#f59e0b' : isOptimal ? '#10b981' : '#475568'}
 strokeWidth={isChecking ? 5 : isOptimal ? 2 : 1}
 markerEnd={marker}
 className="transition-all duration-300"
 strokeDasharray={isChecking ? '4,4' : ''}
 />
 <circle

```

```

 <text x={node.x} y={node.y + 35} fill=
 {node.name.split(' ')[0]}
 </text>
 </g>
);
 }}}
</svg>
<div className="w-full bg-slate-950/80 p-4 rounded-2xl">
 <p className="font-semibold text-amber-400">
 <p className="min-h-[40px] flex items-center">
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/50 p-4 rounded-2xl">
 <h4 className="text-lg font-bold text-emerald-400">
 <div className="space-y-2.5 flex-1 overflow-y-hidden">
 {Object.keys(adjList).map((u) => {
 const isCurrentNode = currentStep.currentNode === u;
 return (
 <div key={u} className={`p-3 rounded-lg ${
 isCurrentNode ? 'bg-emerald-900/60' : 'bg-slate-300'
 }`} >
 <div className="flex items-center gap-2">
 <span className={`px-2 py-0.5 rounded ${
 isCurrentNode ? 'bg-amber-400' : 'bg-slate-300'
 }`} >
 {u}

 →
 </div>
 <div className="flex flex-wrap gap-2">
 {adjList[u].map((edge, idx) => {
 const isCheckingThis = currentStep.currentNode === edge.to;
 return (
 <span key={idx} className={`px-2 py-0.5 rounded ${
 isCheckingThis ? 'bg-amber-400' : 'bg-slate-300'
 }`} >

```

```

 isCurr ? 'bg-indigo-950/60 font
 }`}>
 <td className="py-2.5 px-3 flex
 <span className={`w-2 h-2 rounded
 {n.name}
 </td>
 <td className="py-2.5 px-3 font
 {dist === 999 ? '∞' : dist}
 </td>
 <td className="py-2.5 px-3 text
 {prev || '-'}
 </td>
 </tr>
);
 }}}
</tbody>
</table>
</div>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60
 <div>
 <h4 className="text-lg font-bold text-slate
 <div className="bg-slate-950/80 rounded-lg
 {codeLines.map((item) => {
 const isActive = currentStep.activeCode
 return (
 <div key={item.line} className={`py-1
 isActive ? 'bg-indigo-900/80 text
 }`}>
 <span className="text-slate-600 sel
 <span className="whitespace-pre fle
 {isActive && <span className="text-
 }
 </div>
)};

```

```

const [steps, setSteps] = useState<DPStep[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useSt
const [isPlaying, setIsPlaying] = useState<boolean>(f
const [speed] = useState<number>(800);
const [activeTab, setActiveTab] = useState<'table' |

useEffect(() => {
 const generatedSteps: DPStep[] = [];
 let stepId = 0;
 const memoMap: { [key: number]: number } = {};

 function simulateFib(n: number): number {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вызов fibМемо(${n}). Проверяем наличие в
 codeLine: 2,
 memoState: { ...memoMap }
 });

 if (n in memoMap) {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'memo_hit',
 desc: `< Кэш-хит! fibМемо(${n}) уже посчитано
 codeLine: 2,
 memoState: { ...memoMap }
 });
 return memoMap[n];
 }

 if (n <= 2) {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'base case',

```



```

 desc: ` Расчет завершен! fibMemo(${targetN}) = $
 codeLine: 6,
 memoState: { ...memoMap }
 });

 setSteps(generatedSteps);
 setCurrentStepIndex(0);
 setIsPlaying(false);
}, [targetN]);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length -
 timer = setTimeout(() => {
 setCurrentStepIndex(prev => prev + 1);
 }, speed);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps, speed]);

const currentStep = steps[currentStepIndex] || null;
const memoState = currentStep?.memoState || {};

return (
 <div className="bg-slate-900 rounded-2xl border border-
 /* Header & Controls */
 <div className="flex flex-col lg:flex-row lg:item
 <div>
 <div className="flex items-center gap-3 mb-2">
 <div className="p-2 bg-emerald-500/10 borde
 <RefreshCw className="w-6 h-6" />
 </div>
 <h3 className="text-2xl font-extrabold text
 </div>
 <p className="text-sm text-slate-400">
 Смотри, как таблица DP кэширует результаты

```

```

 isPlaying
 ? 'bg-amber-500 text-slate-950 hover:bg-amber-600 text-slate-950'
 : 'bg-emerald-600 text-white hover:bg-emerald-700 text-white'
)` }
 >
 {isPlaying ? <Pause className="w-4 h-4" /> : <Play className="w-4 h-4" />}
 {isPlaying ? 'Пауза' : 'Пуск'}
 </button>
 <button
 onClick={() => { setIsPlaying(false); setStepsIndex(0); }}
 disabled={currentStepIndex === steps.length - 1}
 className="p-2 text-slate-400 hover:text-slate-500"
 title="Шаг вперед"
 >
 <ChevronRight className="w-5 h-5" />
 </button>
</div>
</div>
</div>

{/* DP Table View */}
<div className="mb-8 bg-slate-950 p-4 md:p-6 rounded-lg" >
 <h4 className="text-xs font-bold uppercase tracking-wide" >DP Table View
 <div className="flex flex-wrap items-center gap-2" >
 {Array.from({ length: targetN }, (_, i) => i + 1).map((num) => {
 const isCached = num in memoState || num <= 2;
 const val = num <= 2 ? 1 : memoState[num];
 const isCurrent = currentStep?.n === num;

 return (
 <div
 key={num}
 className={`flex flex-col items-center justify-center gap-2 p-2
 ${isCurrent ? 'bg-emerald-950 border-emerald-500' : 'bg-slate-900 border-emerald-500/50'}
 ${isCached ? 'border-slate-800' : 'border-slate-800/50'}`
 >
 {num}
 {val}
 </div>
);
 })}
 </div>
</div>

```

```

 <button
 onClick={() => setActiveTab('code')}
 className={`flex items-center gap-2 px-6 py-3
 activeTab === 'code'
 ? 'border-emerald-500 text-emerald-400 bg-white'
 : 'border-transparent text-slate-400 hover:bg-slate-100'}
 >
 <Code className="w-4 h-4" />
 Интерактивный код
 </button>
 </div>

```

```

{/* Tab Content */}
<div className="min-h-[320px] flex flex-col justify-between"
 {activeTab === 'table' && (
 <div className="bg-slate-950 p-6 rounded-2xl border border-slate-800"
 <h5 className="font-bold text-white text-sm"
 {steps.length} шагов
 </h5>
 <p className="text-sm text-slate-400">
 В классической рекурсии для $N=\{targetN\}$ необходимо вычислить
 вычисленное значение в таблице, мы моментами не можем
 </p>
 <div className="p-4 bg-slate-900 rounded-xl border border-slate-800"
 Всего шагов симуляции: {steps.length}
 {steps[steps.length-1]}
 </div>
 </div>
)}

```

```

{activeTab === 'code' && (
 <div className="bg-slate-950 rounded-2xl border border-slate-800"
 <div className="bg-slate-900 px-6 py-3 border border-slate-800"
 {code}
 {steps[steps.length-1]}
 </div>
)}

```

```
 </div>
 </div>
 </div>
);
}
```

---

ФАЙЛ 36/83: src/components/EulerSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState } from "react";
import { Undo } from "lucide-react";
```

```
const C1_NODES = [
 { id: 0, label: "A", x: 190, y: 60 },
 { id: 1, label: "B", x: 280, y: 140 },
 { id: 2, label: "C", x: 100, y: 140 },
 { id: 3, label: "D", x: 280, y: 250 },
 { id: 4, label: "E", x: 100, y: 250 }
];
```

```
const C1_EDGES = [
 { u: 0, v: 1 }, { u: 0, v: 2 },
 { u: 1, v: 3 }, { u: 2, v: 4 },
 { u: 1, v: 2 },
 { u: 3, v: 4 }
];
```

```
export function EulerSimulator() {
 const [c1Path, setC1Path] = useState<number[]>([]);
 const [c1VisitedEdges, setC1VisitedEdges] = useState<
 const [c1Msg, setC1Msg] = useState("Кликни на вершину");
 const [c1Done, setC1Done] = useState(false);
```

```
 const resetC1 = () => {
 setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли
```

```

 };

 return (
 <div className="space-y-4">
 <div className="flex items-center justify-between">
 <h4 className="text-base font-bold text-white">
 <button onClick={resetC1} className="flex items
 old border border-slate-700 transition-all">
 <Undo className="h-3.5 w-3.5" /> C6poc
 </button>
 </div>

 <div className="bg-slate-950 rounded-xl p-4 borde
 -hidden">
 <div className="absolute inset-0 bg-[radial-gra
 <svg className="w-full h-[260px] z-10 relative"
 {C1_EDGES.map((e, i) => {
 const u = C1_NODES.find(n => n.id === e.u)!
 const v = C1_NODES.find(n => n.id === e.v)!
 const key = `${Math.min(e.u,e.v)}-${Math.ma
 const used = c1VisitedEdges.includes(key);
 return <line key={i} x1={u.x} y1={u.y} x2={
 stroke={used ? "#6366f1" : "#475569"}
 strokeWidth={used ? 4 : 2}
 className="transition-all duration-300" />
 }}}
 {C1_NODES.map(n => {
 const isLast = c1Path[c1Path.length-1] === n
 const visited = c1Path.includes(n.id);
 return <g key={n.id} className="cursor-poin
 {isLast && <circle cx={n.x} cy={n.y} r={1
 <circle cx={n.x} cy={n.y} r={11}
 fill={isLast ? "#6366f1" : visited ? "#
 stroke={isLast ? "#fff" : visited ? "#6
 strokeWidth={2.5}
 className="transition-all duration-200
 <text x={n.x} y={n.y+4} textAnchor="middl
 bel}</text>

```

```
const quizQuestions: Question[] = [
 {
 id: 1,
 question: 'Что мы делаем в алгоритме Краскала, если
ребро (красный) цвет?',
 options: [
 'Мы радуемся и добавляем его в минимальное остовное дерево',
 'Мы перекрашиваем их в синий цвет и делим граф по
компонентам связности',
 'Мы безжалостно ВЫБРАСЫВАЕМ это ребро, иначе получим цикл',
 'Мы вызываем алгоритм Дейкстры для поиска нового ребра'
],
 correctAnswer: 2,
 explanation: 'Верно! Если вершины уже в одном клане
и добавим ребро, это нарушит структуру дерева.'
 },
 {
 id: 2,
 question: 'Почему алгоритм Дейкстры впадает в ступор?',
 options: [
 'Потому что он написан только для положительных весов',
 'Он жадный и считает, что каждый следующий шаг делит граф по
закрытые вершины.',
 'Отрицательные ребра создают гравитационный коллапс',
 'Дейкстра просто не любил отрицательные балансы на ребрах'
],
 correctAnswer: 1,
 explanation: 'Именно! Дейкстра уверен: если ты приедешь в город
для чего нужен алгоритм Беллмана-Форда.'
 },
 {
 id: 3,
 question: 'Какова ключевая фишка алгоритма Борувки?',
 options: [
 'Он использует меньше памяти благодаря коммунистическим принципам',
 'Он умеет работать с отрицательными весами без циклов',
 'Он позволяет выполнять шаги слияния колхозов (компонент)',
 'Он был разработан в Токио и поэтому самый быстрый'
],
 },
]
```

```

const currentQ = quizQuestions[currentQIndex];

const handleSelectOption = (index: number) => {
 if (isAnswered) return;
 setSelectedOption(index);
 setIsAnswered(true);
 if (index === currentQ.correctAnswer) {
 setScore(prev => prev + 1);
 }
};

const handleNext = () => {
 if (currentQIndex + 1 < quizQuestions.length) {
 setCurrentQIndex(prev => prev + 1);
 setSelectedOption(null);
 setIsAnswered(false);
 } else {
 setShowResults(true);
 }
};

const handleRestart = () => {
 setCurrentQIndex(0);
 setSelectedOption(null);
 setIsAnswered(false);
 setScore(0);
 setShowResults(false);
};

if (showResults) {
 return (
 <div className="bg-slate-900/90 border border-slate-800 p-12">
 <div className="w-20 h-20 bg-gradient-to-tr from-indigo-500 to-purple-500 shadow-xl shadow-indigo-500/30">
 <Award className="w-10 h-10 text-white" />
 </div>
 <h3 className="text-3xl font-bold text-white mb-4">

```

```


 </div>

 <div className="mb-8">
 <h4 className="text-xl md:text-2xl font-bold text-gray-900">
 {currentQ.question}
 </h4>

 <div className="space-y-4">
 {currentQ.options.map((option, idx) => {
 const isSelected = selectedOption === idx;
 const isCorrect = idx === currentQ.correctAnswer;

 let optionStyle = "bg-slate-800/80 hover:bg-slate-800/90";
 if (isAnswered) {
 if (isCorrect) {
 optionStyle = "bg-emerald-950/80 border-emerald-500";
 } else if (isSelected) {
 optionStyle = "bg-rose-950/80 border-rose-500";
 } else {
 optionStyle = "bg-slate-800/40 border-slate-700";
 }
 }

 return (
 <div
 key={idx}
 onClick={() => handleSelectOption(idx)}
 className={"flex items-start gap-4 p-4 rounded-lg"}
 style={optionStyle}
 >
 <div className="mt-0.5 shrink-0">
 {isAnswered && isCorrect ? (
 <CheckCircle2 className="w-6 h-6 text-emerald-500" />
) : isAnswered && isSelected && !isCorrect ? (
 <XCircle className="w-6 h-6 text-rose-500" />
) : (
 <div className={"w-6 h-6 rounded-full border border-slate-700"} style={{
 background: isCorrect ? "indigo-500" : "indigo-500",
 color: isCorrect ? "white" : "white"
 }} />
)
 }
 <div>{option}</div>
 </div>
);
 })}
 </div>
 </div>

```



```
 </div>
 </div>
);
 };
};
```

---

ФАЙЛ 38/83: src/components/FloydViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect } from 'react';

interface Step {
 k: number; i: number; j: number;
 matrix: number[][]; updated: boolean;
 log: string; activeCodeLine: number;
}

export default function FloydViz() {
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);

 const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4', 'Е 5', 'Ж 6'];
 const nodesSvg = [
 { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
 { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
 { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
 { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
 { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
 { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
 { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
];

 const initialMatrix = [
 [0, 4, 999, 2, 999, 999, 999],
 [999, 0, 3, 999, 999, 3, 999],
];
}
```

```

const N = 7;
for (let k = 0; k < N; k++) {
 simulationSteps.push({
 k, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: `+ Внешний цикл. Разрешаем пути через прогн.
 activeCodeLine: 3,
 });

 for (let i = 0; i < N; i++) {
 for (let j = 0; j < N; j++) {
 if (i === j || i === k || j === k) continue;

 const oldVal = dist[i][j];
 const viaVal = dist[i][k] + dist[k][j];
 if (dist[i][k] !== 999 && dist[k][j] !== 999 &&
 dist[i][j] > viaVal) {
 dist[i][j] = viaVal;
 simulationSteps.push({
 k, i, j, matrix: dist.map(r => [...r]),
 log: ` Улучшение пути [${i}]->[${j}] через
 ${k}.`,
 activeCodeLine: 7,
 });
 }
 }
 }
}

simulationSteps.push({
 k: 7, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: ` Все пары отработаны. Алгоритм Флойда заверш.
 activeCodeLine: 8,
});

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, []);

```

```

<div className="w-full lg:w-2/3 bg-indigo-950/20 justify-center min-h-[400px]">
 <div className="absolute top-3 left-3 bg-indigo-950/20 font-bold shadow-sm">
 {currentStep.k >= 0 && currentStep.k < 7 ? currentStep.k : 0}
 </div>

 <svg className="w-full h-auto max-h-[500px]" >
 <defs>
 <marker id="arrow-fw-normal" markerWidth="10" markerHeight="10"
 <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
 <marker id="arrow-fw-active" markerWidth="10" markerHeight="10"
 <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
 <marker id="arrow-fw-via" markerWidth="6" markerHeight="6"
 <path d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></marker>
 </defs>

 {Object.keys(adjList).flatMap((uStr) => {
 const u = parseInt(uStr);
 return adjList[u].map((edge) => {
 const uNode = nodesSvg.find((n) => n.id === u);
 const vNode = nodesSvg.find((n) => n.id === edge.v);
 const isTarget = currentStep.i === u && currentStep.k === 0;
 const isVia = (currentStep.i === u && currentStep.k > 0);
 let strokeColor = '#475569'; let marker = 'arrow-fw-normal';
 if (isTarget) { strokeColor = '#10b981'; marker = 'arrow-fw-active'; }
 else if (isVia) { strokeColor = '#818cf8'; marker = 'arrow-fw-via'; }
 return (
 <g key={`_${u}-${edge.v}`}>
 <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y}
 markerEnd={marker} strokeDasharray={isTarget ? [5, 5] : []}
 strokeColor={strokeColor} strokeWidth={isTarget ? 2 : 1} />
 <circle cx={({uNode.x + vNode.x} / 2) y={({uNode.y + vNode.y} / 2)}
 fill={isTarget ? '#10b981' : '#818cf8'} stroke={isTarget ? '#10b981' : '#818cf8'}
 strokeDasharray={isTarget ? [5, 5] : []} strokeWidth={isTarget ? 2 : 1} />
 <text x={({uNode.x + vNode.x} / 2) y={({uNode.y + vNode.y} / 2)}
 fill="#818cf8" fontSize="10" fontWeight="bold" text={edge.v} />
 </g>
);
 });
 });
 }
</svg>

```

```

 }`}>
 →
 {adjList[u].map((edge, idx) => {
 const isUpd = currentStep.i === u
 const isVia = currentStep.k !== currentStep.j === edge.v);
 return (

 {edge.v} ({edge.w})

)
 })}
 </div>
);
 })}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 /* Матрица расстояний */
 <div className="w-full lg:w-1/2 bg-rose-950/10">
 <div className="flex justify-between items-center">
 <h4 className="text-lg font-bold text-rose-950">Матрица расстояний</h4>
 </div>
 <div className="overflow-x-auto">
 <table className="w-full text-center border-collapse">
 <thead><tr className="bg-rose-950/40 text-white">
 <th className="p-2 border-r border-rose-950">
 {nodeNames.map((n, idx) => <th key={idx} className="p-2 border-r border-rose-950 font-bold" : ''}`}>{n.split(' ')[0]} {n.split(' ')[1]}
 </th></tr></thead>
 <tbody className="text-xs font-mono">
 {currentStep.matrix.map((row, i) => (
 <tr key={i} className="border-b border-rose-950">
 <td className={`p-2 bg-rose-950/40 text-white`}>

```

```
 });
}
```

---

ФАЙЛ 39/83: src/components/GraphTraversalViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect, useMemo } from 'react';
import { Play, Pause, SkipForward, Undo, RefreshCw } from 'lucide-react';
```

```
type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };
```

```
const nodes: Node[] = [
 { id: 'A', label: 'A', x: 200, y: 40 },
 { id: 'B', label: 'B', x: 100, y: 120 },
 { id: 'C', label: 'C', x: 300, y: 120 },
 { id: 'D', label: 'D', x: 50, y: 200 },
 { id: 'E', label: 'E', x: 150, y: 200 },
 { id: 'F', label: 'F', x: 250, y: 200 },
 { id: 'G', label: 'G', x: 350, y: 200 },
];
```

```
const edges: Edge[] = [
 { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
 { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
 { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];
```

```
const adj: Record<string, string[]> = {
 'A': ['B', 'C'],
 'B': ['A', 'D', 'E'],
 'C': ['A', 'F', 'G'],
 'D': ['B'],
 'E': ['B'],
 'F': ['C'],
 'G': ['C'],
};
```

```
function generateBFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const q: string[] = [];

 q.push(start);
 vis.add(start);
 steps.push({ type: 'visit', node: start, desc: `Начинаем с узла ${start}.`, ray: from(vis) });

 while (q.length > 0) {
 const v = q[0];
 steps.push({ type: 'process', node: v, desc: `Обработка узла ${v}.`, q, visitedNodes: [...Array.from(vis)] });
 q.shift();

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 vis.add(u);
 q.push(u);
 steps.push({ type: 'visit', node: u, desc: `Визит узла ${u}.`, ray: from(vis), backtrack: [...q], visitedNodes: [...Array.from(vis)] });
 }
 }
 }

 steps.push({ type: 'backtrack', node: '', desc: `Завершение обхода.`, ray: from(vis) });

 return steps;
}
```

```
export function GraphTraversalViz() {
 const [mode, setMode] = useState<"dfs" | "bfs">("dfs");
 const [autoPlay, setAutoPlay] = useState(false);
 const [stepIdx, setStepIdx] = useState(0);
```

```

 "text-slate-400 hover:text-slate-300"}``>
 BFS (В ширину / Волна)
 </button>
 </div>

```

```

<div className="flex flex-col md:flex-row gap-4" style={{width: '100%'}}>
 {/* SVG Graph View */}
 <div className="bg-slate-900 border border-slate-700 p-4 min-h-[300px]">

```

```

 <svg className="w-full h-full max-w-[1000px] max-h-[600px]">
 {/* Edges */}
 {edges.map((e, i) => {
 const u = nodes.find(n => n.id === e.u);
 const v = nodes.find(n => n.id === e.v);
 // Check if edge is part of the current path
 const edgeTraversed = isVisited(e.u, e.v);

 return (
 <line key={i} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
 className={
 edgeTraversed ? 'stroke-emerald-500' : 'stroke-slate-400'
 }
 />
);
 })}

```

```

 {/* Nodes */}
 {nodes.map(n => {
 const visited = isVisited(n.id);
 const current = isCurrent(n.id);

 let fillColor = "fill-slate-400";
 let strokeColor = "stroke-slate-400";
 let textColor = "fill-slate-400";

 let radius = 18;

```

```

 if (visited) {
 fillColor = mode === 'dark' ? 'fill-emerald-500' : 'fill-slate-400';
 strokeColor = mode === 'dark' ? 'stroke-emerald-500' : 'stroke-slate-400';
 textColor = "fill-white";
 }

```

```

 {step.queueOrStack && step.queue
 <div key={idx} className={`w-100px
 ${mode === 'dfs' ? 'bg-emerald-200' : 'bg-transparent'}
 text-emerald-200`}
 style={{width: 100px, height: 10px, border: 1px solid
 ${idx === (mode === 'dfs' ? 0 : steps.length - 1) ? 'red' : 'black'}
 0_10px_rgba(245,158,11,0.2)}} : ''}
 {step.item}
 {idx === (mode === 'dfs' ? 0 : steps.length - 1) ? 'Узел: {item}' : ''}

 {step.item}

 </div>
)}
 </div>
 </div>
 </div>

 <div className="flex flex-col sm:flex-row gap-2" style={
 {/* Controls */}
 <div className="flex bg-slate-900 border border-slate-800 p-2 text-slate-400
 hover:text-white hover:bg-slate-800" style={
 {/* Controls */}
 <button onClick={() => {setAutoPlay(!autoPlay)}}
 <Undo strokeWidth={3} size={16} />
 </button>
 <button onClick={() => setAutoPlay(!autoPlay)}
 justify-center min-w-[80px] ${mode === 'dfs' ? 'text-red-500' : 'text-slate-400'}
 >
 {autoPlay ? <><Pause size={16} /> : <><Play size={16} />}
 </button>
 <button onClick={() => {setAutoPlay(!autoPlay); setStepIndex(steps.length - 1)}}
 className="p-2 text-slate-400 hover:text-white hover:bg-transparent"
 >
 <SkipForward strokeWidth={3} size={16} />
 </button>
 <button onClick={() => {setAutoPlay(!autoPlay); setStepIndex(0)}}

```



```

function siftUp(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
 let idx = a.length - 1;
 while (idx > 0) {
 const parent = Math.floor((idx - 1) / 2);
 if (a[parent].priority < a[idx].priority) {
 [a[parent], a[idx]] = [a[idx], a[parent]];
 idx = parent;
 } else break;
 }
 return a;
}

function siftDown(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
 const n = a.length;
 let idx = 0;
 while (true) {
 let largest = idx;
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < n && a[l].priority > a[largest].priority) largest = l;
 if (r < n && a[r].priority > a[largest].priority) largest = r;
 if (largest !== idx) {
 [a[largest], a[idx]] = [a[idx], a[largest]];
 idx = largest;
 } else break;
 }
 return a;
}

function heapInsert(arr: Patient[], item: Patient): Patient[] {
 return siftUp([...arr, { ...item }]);
}

// Position in tree layout
function nodePos(index: number): { x: number; y: number } {
 const level = Math.floor(Math.log2(index + 1));

```

```

const [customPriority, setCustomPriority] = useState(0)
const [log, setLog] = useState<string>('')
const [shakeEmpty, setShake] = useState(false)
const [busy, setBusy] = useState(false)
const [healingPatient, setHealing] = useState<Patient>({})

const addLog = (msg: string) => setLog(prev => [msg,
 ...prev].slice(-10))

// --- INSERT: Новый пациент поступает в приемный покой
const insertPatient = useCallback((name: string, priority: number) => {
 if (busy || heap.length >= 15) {
 addLog('⚠ Все палаты переполнены! Дождитесь выписки')
 setShake(true)
 setTimeout(() => setShake(false), 400)
 return
 }
 setBusy(true)

 const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)]
 const finalName = name || preset.name.split(' ')[0]
 const finalEmoji = preset.emoji
 const finalSymptom = preset.symptom

 const newPatient: Patient = {
 id: `p${uid++}`,
 name: finalName,
 emoji: finalEmoji,
 symptom: finalSymptom,
 priority,
 animState: 'in',
 }

 const nextHeap = heapInsert(heap, newPatient).map(x => x)
 // Mark specifically this new patient as 'in' in the heap
 const target = nextHeap.find(x => x.priority === priority)
 const marked = nextHeap.map(x => x.id === (target ? target.id : ''))

 setHeap(marked)
}, [heap, busy, setShake, setLog, setBusy, setHealing])

```

```

setTimeout(() => {
 // Step 2: Root exits the tree, tree reorganizes
 setHeap(prev => prev.map((x, i) => i === 0 ? { ..
 setTimeout(() => {
 // Run binary heap extraction
 setHeap(prev => {
 if (prev.length === 0) return [];
 const a = prev.map(x => ({ ...x }));
 a[0] = { ...a[a.length - 1] };
 a.pop();
 return siftDown(a).map(x => ({ ...x, animStat
 });

 // Patient gets healed in the bed
 setTimeout(() => {
 setHealing(prev => prev ? { ...prev, animStat
 addLog(` Успех! Пациент ${topPatient.name} п
 setTimeout(() => {
 setHealing(null);
 setBusy(false);
 }, 600);
 }, 800);

 }, 350);
 }, 650);
}, [busy, heap]);

const reset = () => {
 setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat
 setLog([' База данных скорой помощи перезагружена.
 setHealing(null);
 setTimeout(() => setHeap(INITIAL_PATIENTS.map(x =>
});

// --- Render binary tree lines ---
const edges = heap.flatMap((, idx) => {

```

Куча (Heap) автоматически перестраивает пац  
оказывался на самом верху дерева (в корне).

```

 </p>
 </div>
</div>

```

```

{/* УПРАВЛЕНИЕ */}
<div className="flex flex-col lg:flex-row items-s

```

```

 {/* Добавить случайного */}
 <button
 onClick={handleRandomInsert}
 disabled={busy || heap.length >= 15}
 className="flex-1 min-w-[150px] bg-gradient-to
 opacity-40 disabled:cursor-not-allowed text
 ve:scale-95 transition-all cursor-pointer f

```

```

 >
 Привезти по Скорой (INSERT)
 </button>

```

```

{/* Добавить кастомного */}
<form onSubmit={handleCustomInsert} className="
 <input
 type="text"
 required
 value={customName}
 onChange={e => setCustomName(e.target.value)}
 placeholder="Имя пациента"
 className="flex-1 bg-slate-800 border borde
 er-emerald-500 transition-colors placeholde

```

```

 />
 <div className="flex flex-col items-center min
 <span className="text-[9px] font-mono text-
 <input
 type="range"
 min="10"
 max="99"
 value={customPriority}

```

```

<div className={`lg:col-span-8 bg-slate-900 rounded
 -[380px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}
 <div className="absolute top-4 left-4 text-[10px] font-mono
 Сортировка по тяжести (Двоичная куча / Max-Heap)
 </div>

```

```

<div className="relative w-full h-full min-h-[380px]
 <svg className="absolute inset-0 w-full h-full
 {edges}
 </svg>

```

```

{heap.map((patient, idx) => {
 const pos = nodePos(idx);
 const isRoot = idx === 0;
 return (
 <div
 key={patient.id}
 className={`
 absolute flex flex-col items-center justify-center
 -translate-x-1/2 -translate-y-1/2 transition-all
 ${patient.animState === 'in' ? 'animate-in' : ''}
 ${patient.animState === 'winner' ? 'animate-winner' : ''}
 ${patient.animState === 'out' ? 'animate-out' : ''}
 ${isRoot ? 'bg-rose-950/80 border-rose-500' : ''}
 `}
 style={{
 left: `${pos.x}%`,
 top: `${pos.y}%`,
 width: isRoot ? 60 : 52,
 height: isRoot ? 60 : 52,
 }}
 >
 {patient.value}
 {patient.priority}
 </div>
)
})}

```

```

{/* Tooltip */}

```

```

 <Heart className="w-4 h-4 text-rose-500" />
 </div>
 </div>
 </div>
) : {
 <div className="text-center text-slate-600" >
 Ждём вас
 <p className="text-xs font-bold font-mono">Ваше имя</p>
 <p className="text-[10px] mt-1">Ждём вас</p>
 </div>
 <div className="w-full h-3 bg-gradient-to-r from-slate-500 to-slate-600" />
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-800 p-4" >
 <div className="text-[10px] font-mono text-slate-400" >
 {log.map((entry, idx) => {
 <div
 key={idx}
 className={`text-xs font-mono flex items-center justify-between p-2`}
 >
 {idx === 0 ? Ваше имя : {entry}
 </div>
 })}
 </div>
</div>
);
};

```

```

 idle: "#334155",
 idleStroke: "#475569",
 text: "#e2e8f0",
 dim: "#94a3b8",
 active: "#6366f1",
 done: "#10b981",
 warn: "#f59e0b",
 bad: "#f43f5e",
 edge: "#475569",
 };

export const Svg: React.FC<{ children: React.ReactNode;
 const paused = useContext(DemoPauseContext);
 return (
 <div className="w-full">
 <svg viewBox={`0 0 ${W} ${H}`} className="w-full h-full">
 {children}
 </svg>
 {caption && (
 <div className="text-[10px] text-slate-400 text-center">
 {caption}
 </div>
)}
 <div className="text-[9px] text-slate-600 text-center">
 {paused ? "пауза — курсор на карточке" : "наведи"}
 </div>
 </div>
);
};

export const Node: React.FC<{
 x: number;
 y: number;
 r?: number;
 fill: string;
 label?: string | number;
 stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
```

```

/** Флойд: внешний цикл по «разрешённой» промежуточной
export const FloydDemo: React.FC = () => {
 const step = useLoop(3, 1200);
 // i -> j напрямую 9; через k=1 получаем 3+4=7
 const pos: Record<string, [number, number]> = { i: [4, 9],
 const viaOpen = step >= 1;
 const relaxed = step >= 2;
 return (
 <Svg
 caption={
 step === 0
 ? "k ещё запрещена: путь i → j = 9"
 : step === 1
 ? "Разрешаем пересадку через k: 3 + 4 = 7"
 : "7 < 9 → D[i][j] = 7"
 }
 >
 <Edge a={pos.i} b={pos.j} color={relaxed ? C.bad : C.good} />
 <Edge a={pos.i} b={pos.k} color={viaOpen ? C.done : C.good} />
 <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done : C.good} />

 <text x={130} y={112} textAnchor="middle" fontSize=
 9
 />
 <text x={72} y={54} textAnchor="middle" fontSize=
 3
 />
 <text x={188} y={54} textAnchor="middle" fontSize=
 4
 />

 <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C.i} />
 <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={C.via} />
 <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={C.j} />
 </Svg>
);
};

```



```

caption={
 step === 0
 ? "Вершины – объекты (города, слова, состояния)"
 : step === 1
 ? "Рёбра – связи между ними"
 : "Вес ребра – цена перехода"
}
>
{edges.map(([u, v, w], i) => {
 const a = pos[u];
 const b = pos[v];
 return (
 <g key={i}>
 <Edge a={a} b={b} color={step >= 1 ? C.active : C.warn}>
 {step >= 2 && (
 <text
 x={({a[0] + b[0]} / 2}
 y={({a[1] + b[1]} / 2 - 4}
 textAnchor="middle"
 fontSize={10}
 fontWeight="700"
 fill={C.warn}
 >
 {w}
 </text>
)}
 </g>
);
})}
{Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={C.idle} />
))}
</Svg>
);
};

```

```

/** Сжатие координат: разрежённые числа → плотные индексы
export const CoordCompressDemo: React.FC = () => {

```

```

 <rect x={startX + i * (boxW + 4)} y={98} wi
 <text x={startX + i * (boxW + 4) + boxW / 2
 {sorted.indexOf(v)}
 </text>
 </g>
)})
</Svg>
);
};

/* ----- Splay: три случая поворота ---

type Pt = [number, number];

const SplayFrames: React.FC<{
 frames: { pos: Record<string, Pt>; edges: [string, st
 captions: string[];
 ms?: number;
}> = ({ frames, captions, ms = 1500 }) => {
 const step = useLoop(frames.length, ms);
 const f = frames[step];
 const color = (id: string) =>
 id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : i

 return (
 <Svg caption={captions[step]}>
 {f.edges.map(([a, b], i) => (
 <line
 key={i}
 x1={f.pos[a][0]}
 y1={f.pos[a][1]}
 x2={f.pos[b][0]}
 y2={f.pos[b][1]}
 stroke={C.edge}
 strokeWidth={1.8}
 style={{ transition: MOVE }}
 />
)})
 </Svg>
)
}

```

```

 captions=["Зигзаг: g влево, x вправо", "Поворот НИ
frames=[
 { pos: { g: [176, 24], p: [110, 70], x: [152, 112]
 { pos: { g: [176, 24], x: [110, 70], p: [64, 112]
 { pos: { x: [130, 30], p: [80, 100], g: [180, 100]
]}
 />
);

```

ФАЙЛ 43/83: src/components/hints/demosGraph.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

const GRAPH_POS: Record<string, [number, number]> = {
 A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],
};
const GRAPH_EDGES: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D",
];

export const BfsDemo: React.FC = () => {
 const order = [["A"], ["B", "C"], ["D"], ["E", "F"]];
 const step = useLoop(order.length + 1, 850);
 const visited = new Set(order.slice(0, step).flat());
 const wave = step > 0 && step <= order.length ? order
 return (
 <Svg caption={`Уровень ${Math.max(0, Math.min(step,
 {GRAPH_EDGES.map(([u, v], i) => {
 <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
 }}})
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k}
 fill={wave.includes(k) ? C.active : visited.h

```

```

 {edges.map(([u, v], i) => {
 <Edge key={i} a={pos[u]} b={pos[v]} color={placed[i]} />
 })}
 {nodes.map((n) => {
 const idx = placed.indexOf(n.id);
 return (
 <g key={n.id}>
 <rect x={n.x - 34} y={n.y - 12} width={68} height={24} />
 <text x={n.x} y={n.y} textAnchor="middle" dx={
 {idx >= 0 && <circle cx={n.x + 30} cy={n.y} r={10} />
 {idx >= 0 && (
 <text x={n.x + 30} y={n.y - 12} textAnchor="middle" />
)}
 } />
 </g>
);
 })}
 <text x={224} y={62} textAnchor="middle" fontSize={14} />
 <text x={224} y={74} textAnchor="middle" fontSize={14} />
 </Svg>
);
};

```

```

export const RelaxDemo: React.FC = () => {
 const step = useLoop(3, 1100);
 const dV = [12, 12, 7][step];
 const improved = step === 2;
 return (
 <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Пусть" : ""}>
 <Edge a={[60, 70]} b={[200, 70]} color={improved ? "red" : "black"} />
 <text x={130} y={54} textAnchor="middle" fontSize={14} />
 <Node x={60} y={70} r={20} fill={C.active} label="3" />
 <Node x={200} y={70} r={20} fill={improved ? C.done : C.active} label="4" />
 <text x={60} y={106} textAnchor="middle" fontSize={14} />
 <text x={200} y={106} textAnchor="middle" fontSize={14} />
 <text x={130} y={22} textAnchor="middle" fontSize={14} />
 </Svg>
);
};

```

```

 <circle key={k} cx={x} cy={y} r={12} fill="none" stroke={fill} />
) : (
 <Node key={k} x={x} y={y} label={k} fill={k} stroke={stroke} />
)
)
 </Svg>
);
};

export const MstDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [40, 35], B: [130, 22], C: [220, 45], D: [70, 10], E: [180, 80]
 };
 const edges = [
 { u: "A", v: "B", w: 2 }, { u: "B", v: "C", w: 3 },
 { u: "D", v: "E", w: 4 }, { u: "B", v: "E", w: 6 },
];
 const chosen = ["A-D", "A-B", "B-C", "D-E"];
 const step = useLoop(chosen.length + 1, 800);
 const taken = new Set(chosen.slice(0, step));
 return (
 <Svg caption={`Жадно берём самые лёгкие рёбра без циклов`} >
 {edges.map((e, i) => {
 const on = taken.has(`${e.u}-${e.v}`);
 return (
 <g key={i}>
 <Edge a={pos[e.u]} b={pos[e.v]} color={on ? "red" : "black"} />
 <text x={(pos[e.u][0] + pos[e.v][0]) / 2} y={pos[e.v][1]}
 textAnchor="middle" fontSize={9} fontWeight="bold">{e.u}-{e.v}</text>
 </g>
);
 })}
 {Object.entries(pos).map(([k, [x, y]]) => (<Node key={k} x={x} y={y} label={k} fill={k} stroke={stroke} />))}
 </Svg>
);
};
};

```

```

 <Edge a={pos[1]} b={pos[3]} /><Edge a={pos[1]} b=
 {heap.map((v, i) => {
 <Node key={i} x={pos[i][0]} y={pos[i][1]} label=
 }}}
 </Svg>
);
};

```

```

export const StackDemo: React.FC = () => {
 const states = [["A"], ["A", "B"], ["A", "B", "C"], [
 const step = useLoop(states.length, 800);
 const items = states[step];
 const popping = step >= 3;
 return (
 <Svg caption={popping ? "pop: забираем ВЕРХНИЮ таре
 <rect x={90} y={22} width={80} height={96} rx={6}
 {items.map((v, i) => {
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={94} y={100 - i * 26} width={72} height={
 fill={i === items.length - 1 ? (popping ? C
 <text x={130} y={111 - i * 26} textAnchor="mi
 </g>
 }}}
 <text x={130} y={16} textAnchor="middle" fontSize=
 </Svg>
);
};

```

```

export const QueueDemo: React.FC = () => {
 const states = [["A", "B"], ["A", "B", "C"], ["B", "C
 const step = useLoop(states.length, 800);
 const items = states[step];
 return (
 <Svg caption="pop_front слева, push_back справа – к
 <defs>
 <marker id="mdArrow" markerWidth="6" markerHeight=
 <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
 </marker>
 </defs>
 <rect x={90} y={22} width={80} height={96} rx={6}
 {items.map((v, i) => {
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={94} y={100 - i * 26} width={72} height={
 fill={i === items.length - 1 ? (popping ? C
 <text x={130} y={111 - i * 26} textAnchor="mi
 </g>
 }}}
 <text x={130} y={16} textAnchor="middle" fontSize=
 </Svg>
);
};

```

```

 { id: 2, x: 188, y: 62, ch: "b", parent: 0 },
 { id: 3, x: 42, y: 104, ch: "b", parent: 1 },
 { id: 4, x: 102, y: 104, ch: "c", parent: 1 },
 { id: 5, x: 188, y: 104, ch: "c", parent: 2 },
];

export const TrieDemo: React.FC = () => {
 const step = useLoop(TRIE_NODES.length + 1, 700);
 return (
 <Svg caption="Путь от корня по буквам = слово; общие

 {TRIE_NODES.filter((n) => n.parent !== null).map(
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]} }
 {TRIE_NODES.map((n) => (<Node key={n.id} x={n.x} :
 </Svg>
);
};

export const TrieBfsDemo: React.FC = () => {
 const levels = [[0], [1, 2], [3, 4, 5]];
 const step = useLoop(levels.length + 1, 950);
 const doneIds = new Set(levels.slice(0, step).flat());
 const wave = step > 0 && step <= levels.length ? levels[step - 1] : [];
 return (
 <Svg caption={`BFS по бору: уровень ${Math.max(0, M
 {TRIE_NODES.filter((n) => n.parent !== null).map(
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]} }
 {TRIE_NODES.map((n) => (
 <Node key={n.id} x={n.x} y={n.y} label={n.ch}
 fill={wave.includes(n.id) ? C.active : doneId
)} }
 <text x={6} y={12} fontSize={9} fill={C.dim}>очер
 </Svg>
);
};

```

```

 {nodes.map((n, i) => (
 <g key={i}>
 <rect x={n.x - n.w / 2} y={n.y - 10} width={n.w} height={n.h}
 fill={n.lvl === step ? C.active : C.idle} stroke={C.idleStroke} style={{ transition: 'fill 0.2s' }} />
 <text x={n.x} y={n.y} textAnchor="middle" domClass={n.domClass} />
 </g>
))}
 </Svg>
);
};

```

```

export const PrefixSumDemo: React.FC = () => {
 const a = [3, 1, 4, 1, 5, 9];
 const pref = a.reduce<number[]>((acc, v) => [...acc, v], []);
 const step = useLoop(4, 1200);
 const l = 1, r = 3;
 const show = step >= 1;
 return (
 <Svg caption={show ? `a[${l}..${r}] = P[${r + 1}] -` : ''}
 o префиксные суммы P">
 <text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
 {a.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 39} y={24} width={34} height={12}
 fill={show && i >= l && i <= r ? C.active : C.idle} stroke={C.idleStroke} style={{ transition: 'fill 0.2s' }} />
 <text x={37 + i * 39} y={36} textAnchor="middle" />
 </g>
))}
 <text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
 {pref.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 33} y={78} width={29} height={12}
 fill={show && i === l ? C.bad : C.idle} stroke={C.idleStroke} style={{ transition: 'fill 0.2s' }} />
 <text x={34 + i * 33} y={90} textAnchor="middle" />
 </g>
))}
 </Svg>
);
};

```



```

 </Svg>
);
};

```

```

export const NpDemo: React.FC = () => {
 const step = useLoop(2, 1400);
 return (
 <Svg caption={step === 0 ? "Найти решение – долго (1000 итераций)" : "Найти решение – быстро (10 итераций)"}>
 <ellipse cx={130} cy={64} rx={112} ry={48} fill="#333" />
 <ellipse cx={92} cy={64} rx={52} ry={32} fill="#333" />
 <text x={92} y={64} textAnchor="middle" dominantBaseline="middle" font-size="12">
 1000 итераций
 </text>
 <text x={206} y={34} textAnchor="middle" font-size="12">
 10 итераций
 </text>
 <circle cx={196} cy={84} r={13} fill={step === 1 ? "#333" : "#ccc"} />
 <text x={196} y={84} textAnchor="middle" dominantBaseline="middle" font-size="12">
 {step === 1 ? "10 итераций" : "1000 итераций"}
 </text>
 <text x={130} y={124} textAnchor="middle" font-size="12">
 Найти решение – долго (1000 итераций)
 </text>
 </Svg>
);
};

```

```

export const PrefixFunctionDemo: React.FC = () => {
 const s = "abacaba";
 const pi = [0, 0, 1, 0, 1, 2, 3];
 const step = useLoop(s.length + 1, 700);
 const i = Math.max(0, step - 1);
 const len = step > 0 ? pi[i] : 0;
 return (
 <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс "${s[i-len+1:i+1]}"` : "π[0] = 0: префикс """}>
 {s.split("").map((ch, idx) => (
 <g key={idx}>
 <rect x={18 + idx * 33} y={26} width={28} height={20}
 fill={step > 0 && (idx < len || (idx > i - 1)) ? "#333" : "#ccc"}
 style={{ transition: "fill .25s" }} />
 <text x={32 + idx * 33} y={40} textAnchor="middle" font-size="12">
 {ch}
 </text>
 <text x={32 + idx * 33} y={70} textAnchor="middle" font-size="12">
 {idx <= i && step > 0 ? C.warn : C.idle}
 </text>
 </g>
))}
 </Svg>
);
};

```

---

ФАЙЛ 45/83: src/components/hints/MiniDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState } from "react";
import { DemoPauseContext } from "../demoKit";
import { ArticulationDemo, BfsDemo, BridgeDemo, DfsDemo,
import {
 AmortizedDemo, DsuDemo, HeapDemo, NpDemo, PrefixSumDemo,
 SparseTableDemo, StackDemo, SuffixLinkDemo, TrieBfsDemo,
} from "../demosStruct";
import {
 ComponentsDemo, CoordCompressDemo, FloydDemo, GraphBfsDemo,
 ZigDemo, ZigZagDemo, ZigZigDemo,
} from "../demosExtra";
```

```
export type DemoKind =
 | "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"
 | "suffix-link" | "toposort" | "relax" | "prefix-sum"
 | "segment-tree" | "bridge" | "articulation" | "mst"
 | "prefix-function" | "dp" | "floyd" | "components" |
 | "zig" | "zig-zig" | "zig-zag";
```

```
const REGISTRY: Record<DemoKind, React.FC> = {
 bfs: BfsDemo,
 dfs: DfsDemo,
 heap: HeapDemo,
 stack: StackDemo,
 queue: QueueDemo,
 dsu: DsuDemo,
 trie: TrieDemo,
 "trie-bfs": TrieBfsDemo,
 "suffix-link": SuffixLinkDemo,
 toposort: ToposortDemo,
 relax: RelaxDemo,
 "prefix-sum": PrefixSumDemo,
```

```
import React, { useEffect } from "react";
import { createPortal } from "react-dom";
import { X } from "lucide-react";
import { getViz } from "../vizRegistry";

interface Props {
 vizId: string | null;
 onClose: () => void;
}

/** Модальное окно с полноценным симулятором – вызывает
export const SimulatorModal: React.FC<Props> = ({ vizId
 useEffect(() => {
 if (!vizId) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 document.body.style.overflow = "hidden";
 return () => {
 window.removeEventListener("keydown", onKey);
 document.body.style.overflow = "";
 };
 }, [vizId, onClose]);

 const viz = getViz(vizId ?? undefined);
 if (!vizId || !viz) return null;
 const Component = viz.Component;

 return createPortal(
 <div className="fixed inset-0 z-[110] flex items-en
 <div className="absolute inset-0 bg-slate-950/80
 <div className="relative w-full sm:max-w-5xl max-l
 ded-2xl shadow-2xl flex flex-col">
```

```

}

interface Props {
 anchor: HintAnchor | null;
 onClose: () => void;
 onOpenSimulator: (vizId: string) => void;
 onCardEnter: () => void;
 onCardLeave: () => void;
}

const CARD_W = 320;
const GAP = 10;

export const TermPopover: React.FC<Props> = ({ anchor,
 const cardRef = useRef<HTMLDivElement>(null);
 const [pos, setPos] = useState<{ top: number; left: number}>({
 top: 0, left: 0 });

 useEffect(() => {
 if (!anchor) {
 setPos(null);
 return;
 }
 const vw = window.innerWidth;
 const vh = window.innerHeight;
 const width = Math.min(CARD_W, vw - 16);
 const height = cardRef.current?.offsetHeight ?? 300;

 let left = anchor.rect.left + anchor.rect.width / 2;
 left = Math.max(8, Math.min(left, vw - width - 8));

 const below = anchor.rect.top < height + GAP + 8;
 const top = below ? anchor.rect.bottom + GAP : anchor.rect.top - height - GAP;

 setPos({ top: Math.max(8, Math.min(top, vh - height - 8)), left });
 }, [anchor]);

 useEffect(() => {
 if (!anchor) return;

```

```

 >
 <X className="w-4 h-4" />
 </button>
 </div>

 <div className="p-3 space-y-2.5 max-h-[60vh] overflow-hidden" >
 <p className="text-[13px] leading-relaxed text-gray-600">
 {entry.demo && <MiniDemo kind={entry.demo} />}
 {entry.formal && (
 <p className="text-[11.5px] leading-relaxed text-gray-600">
 {entry.formal}
 </p>
)}
 {entry.complexity && (
 <div className="text-[11px] font-mono text-gray-600">
 Сложность: {entry.complexity}
 </div>
)}
 {viz && simulatorId && (
 <button
 onClick={() => onOpenSimulator(simulatorId)}
 className="w-full flex items-center justify-center text-gray-600 font-medium rounded-lg py-2 transition-colors"
 >
 <Play className="w-3.5 h-3.5" /> Показать
 </button>
)}
 </div>
 </div>,
 document.body
);
};

```

```

 if (id === 'B') return -2;
 if (id === 'C') return -1;
 }
 return null;
};

const getEdgeColorClass = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "opacity-0";
 if (step === 1) return "stroke-amber-500 stroke-dasharray-4";
 if (step === 2) return "stroke-pink-500 stroke-dasharray-4";
 return "stroke-slate-700 stroke-dasharray-4";
 }

 if (e.id === 'AB') {
 if (step === 4) return "stroke-amber-400 stroke-dasharray-4";
 if (step > 4) return "stroke-emerald-500";
 }
 if (e.id === 'BC') {
 if (step === 5) return "stroke-amber-400 stroke-dasharray-4";
 if (step > 5) return "stroke-emerald-500";
 }
 if (e.id === 'CA') {
 if (step === 6) return "stroke-amber-400 stroke-dasharray-4";
 if (step > 6) return "stroke-emerald-500";
 }

 if (step >= 4) return "stroke-slate-600"; // otherwise
 return "stroke-slate-500";
};

const getMarkerUrl = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "";
 if (step === 1) return "url(#arrow-amber)";
 if (step === 2) return "url(#arrow-pink)";
 return "url(#arrow-slate-dim)";
 }
};

```

```

};

const isEdgeTextEmerald = (e: any) => {
 if (e.id === 'AB' && step > 4) return true;
 if (e.id === 'BC' && step > 5) return true;
 if (e.id === 'CA' && step > 6) return true;
 return false;
};

const getDesc = () => {
 switch (step) {
 case 0: return (
 <div>
 Исходный граф. Имеется ребро
 Если мы запустим быстрый алгоритм Д
 </div>
);
 case 1: return (
 <div>
 Шаг 1: Точка отсчета.

 Добавляем фиктивную вершину S. Соед
ми.
 Это наша база для расчета потенциал
 </div>
);
 case 2: return (
 <div>
 Шаг 2: Ищем потенциалы $h(v)$.
 Запускаем медленный, но мощный алгор
ных вершин:
 <span className="text-pink-300 ml-1
 </div>
);
 case 3: return (
 <div>
 Шаг 3: Магическая формула.
 Всем ребрам графа назначаем новый в
 <div className="text-center font-mo

```

```
<h4 className="text-sm md:text-base font-bo
 <span className="w-2.5 h-2.5 bg-amber-4
 Визуализация: Перевзвешивание алгоритмо
 </h4>
```

```
<div className="flex flex-col md:flex-row g
 <div className="bg-[#0f172a] flex-1 min
 nter p-4">
```

```
 <svg width="400" height="300" classs
 <defs>
 <marker id="arrow-slate" vie
 start-reverse">
 <path d="M 0 0 L 10 5 L
 </marker>
 <marker id="arrow-slate-dim
 uto-start-reverse">
 <path d="M 0 0 L 10 5 L
 </marker>
 <marker id="arrow-amber" vie
 start-reverse">
 <path d="M 0 0 L 10 5 L
 </marker>
 <marker id="arrow-pink" vie
 tart-reverse">
 <path d="M 0 0 L 10 5 L
 </marker>
 <marker id="arrow-emerald"
 o-start-reverse">
 <path d="M 0 0 L 10 5 L
 </marker>
 </defs>
```

```
 {/* Edges */}
 {edges.map((e, i) => {
 const u = nodes.find(n => n
 const v = nodes.find(n => n

 const isS = e.u === 'S';
```



```

<g key={n.id} className={
 <circle
 cx={n.x} cy={n.y}
 className={`stroke-1
 ${n.id === 'S'
 ? 'fill-[#202020]
 : 'fill-slate-500
 />
 <text x={n.x} y={n.y}
 -white">
 {n.label}
 </text>

 {pot !== null && (
 <g transform={`
fade-in zoom-in duration-500">
 <rect x="-100
k-500 stroke-1" />
 <text x="0"
font-bold fill-pink-400">
 {pot}
 </text>
 </g>
)}
</g>
)
}}
</svg>
</div>

```

```

 <div className="w-full md:w-[320px] flex
 { /* Log Box */
 <div className="bg-[#0f172a] p-5 rounded
1 justify-center leading-relaxed text-sm text-white
 {getDesc()}
 </div>

 { /* Controls */

```

```
import React, { useState, useEffect } from "react";
import {
 Play,
 Pause,
 RotateCcw,
 Layers,
} from "lucide-react";

interface GNode {
 id: number;
 x: number;
 y: number;
 label: string;
}

interface GEdge {
 u: number;
 v: number;
}

const initialNodes: GNode[] = [
 { id: 0, x: 150, y: 100, label: "0" },
 { id: 1, x: 300, y: 100, label: "1" },
 { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
 { id: 3, x: 450, y: 160, label: "3" },
 { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];

const initialEdges: GEdge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 }, // Connection edge
 { u: 3, v: 4 },
 { u: 4, v: 3 },
];
```

```

 transposed,
 sccMap: { ...sccMap },
 currentSccId,
 activeEdges: transposed
 ? initialEdges.map((e) => ({ u: e.v, v: e.u }
 : [...initialEdges],
));
 });
};

recordStep(
 "init",
 "Инициализация алгоритма Косарайю. Начинаем первый",
 null,
 false,
 -1,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
initialEdges.forEach((e) => adj[e.u].push(e.v));

// DFS 1: post-order list
const dfs1 = (u: number) => {
 visited.add(u);
 recordStep(
 "dfs1",
 `DFS 1: зашли в вершину ${u}, помечаем посещение`,
 u,
 false,
 -1,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs1(to);
 }
 }
}

```

```

 recordStep(
 "dfs2",
 `DFS 2: заходим в ${u} на транспонированном графе
 u,
 true,
 currentScc,
);

 for (const to of adjT[u]) {
 if (!visited.has(to)) {
 dfs2(to, currentScc);
 }
 }
 };

// Run DFS2 using the order (reversed from back to front)
for (let i = order.length - 1; i >= 0; i--) {
 const u = order[i];
 if (!visited.has(u)) {
 sccId++;
 recordStep(
 "dfs2",
 `DFS 2: берем следующую непосещенную из стека
 u,
 true,
 sccId,
);
 dfs2(u, sccId);
 } else {
 recordStep(
 "dfs2",
 `DFS 2: вершина ${u} из стека уже покрашена в
 u,
 true,
 sccId,
);
 }
}
}

```

```

const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
};

const getScColor = (id: number) => {
 if (id === 1) return "fill-emerald-600 stroke-emera
 if (id === 2) return "fill-indigo-600 stroke-indigo
 return "fill-slate-800 stroke-slate-500";
};

return (
 <div className="bg-slate-900 rounded-xl border bord
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Layers className="w-5 h-5 text-emerald-400"
 Визуализатор Алгоритма Косарайю (Поиск SCC)
 </h3>

 <div className="flex items-center gap-2 bg-slat
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 d-500 text-white"
 >
 {isPlaying ? {
 <Pause className="w-4 h-4 ml-1" />
 } : {
 <Play className="w-4 h-4 ml-1" />
 }}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 className="flex items-center justify-center
 ors"
 >

```

```

 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#666666" />
 </marker>
</defs>

```

```

{ /* Edges */ }
{initialEdges.map((edge, idx) => {
 const u = initialNodes.find((n) => n.id === edge.u);
 const v = initialNodes.find((n) => n.id === edge.v);

 // If transposed, draw reversed coordinates
 const x1 = step.transposed ? v.x : u.x;
 const y1 = step.transposed ? v.y : u.y;
 const x2 = step.transposed ? u.x : v.x;
 const y2 = step.transposed ? u.y : v.y;

 const isSccEdge =
 step.sccMap[edge.u] &&
 step.sccMap[edge.v] &&
 step.sccMap[edge.u] === step.sccMap[edge.v];
 let stroke = "#475569";
 let markerId = "arrow";

 if (step.transposed) {
 stroke = isSccEdge ? "#818cf8" : "#4f46e5";
 markerId = "arrow-transposed";
 } else {
 if (
 step.currentNode === edge.u ||
 step.currentNode === edge.v
) {
 stroke = "#10b981";
 markerId = "arrow-active";
 }
 }
}
}

```

```

return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r="20"
 className={`transition-all duration
 strokeWidth="3"
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="13px"
 fontWeight="bold"
 className="select-none"
 >
 {node.label}
 </text>

 {sccId && (
 <text
 x={node.x}
 y={node.y - 28}
 textAnchor="middle"
 fill="#10b981"
 fontSize="10px"
 fontWeight="bold"
 >
 SCC #{sccId}
 </text>
)}
 </g>
);
}}

```

```

 >
 {val}
 </div>
))
 })
</div>
</div>

{/* List of actions log */}
<div>

 ЛОГ ОПЕРАЦИЙ:

 <div className="space-y-1.5 max-h-[150px]">
 {steps
 .slice(0, currentStepIdx + 1)
 .reverse()
 .slice(0, 5)
 .map((s, idx) => {
 <div
 key={idx}
 className={`text-[11px] p-1.5 rounded`
 >
 {s.desc}
 </div>
)}}
 </div>
 </div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200">

 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1.5">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev)

```



```

 id: string;
 u: number;
 v: number;
 weight: number;
 status: 'pending' | 'inspecting' | 'included' | 'reje
 color: string;
}

```

```

interface StepLog {
 title: string;
 description: string;
 type: 'info' | 'success' | 'warning' | 'error';
}

```

// 9 Vertices layout

```

const initialVertices: Vertex[] = [
 { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
 { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
 { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
 { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
 { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
 { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
 { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
 { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
 { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

```

// 12 Edges connecting the 9 vertices

```

const initialEdges: Edge[] = [
 { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
 { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
 { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
 { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
 { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
 { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
 { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
 { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
 { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
];

```

```

 title: 'Успех: Ребро A-B в осто́ве',
 description: 'Вершины A and B окрашены в красный цвет',
 type: 'success'
 }, ...prev));
},
// Step 3: Inspect D-E (3)
() => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Шаг 2: Берем второе ребро D-E (вес 3)',
 description: 'Оно соединяет две другие бесцветные вершины',
 type: 'info'
 }, ...prev]));
},
// Step 4: Include D-E
() => {
 setVertices(prev => prev.map(v => v.id === 3 || v.id === 4 ? {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-E в осто́ве',
 description: 'Вершины D и E окрашены в синий цвет',
 type: 'success'
 }, ...prev]));
},
// Step 5: Inspect B-C (4)
() => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Берем ребро B-C (вес 4)',
 description: 'Оно соединяет красную вершину B и синюю вершину C',
 type: 'info'
 }, ...prev]));
},
// Step 6: Include B-C
() => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{

```

```

 title: 'Отказ: Цикл обнаружен!',
 description: 'Мы выбрасываем ребро B-E. Иначе по
 type: 'error'
 }, ...prev]));
},
// Step 11: Inspect E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Берем ребро E-F (вес 7)',
 description: 'Оно соединяет фиолетовую вершину
 type: 'info'
 }, ...prev]));
},
// Step 12: Include E-F
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро E-F в осто́ве',
 description: 'Вершина F окрашена в фиолетовый ц
 type: 'success'
 }, ...prev]));
},
// Step 13: Inspect D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Берем ребро D-G (вес 8)',
 description: 'Оно соединяет фиолетовую вершину
 type: 'info'
 }, ...prev]));
},
// Step 14: Include D-G
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{

```

```

 description: 'Вершина H перекрашена в фиолетовый',
 type: 'success'
 }, ...prev]);
},
// Step 19: Inspect E-H (11) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Берем ребро E-H (вес 11)',
 description: 'Оба конца E и H уже фиолетовые. Соединяет их!',
 type: 'warning'
 }, ...prev]));
},
// Step 20: Reject E-H
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро E-H.',
 type: 'error'
 }, ...prev]));
},
// Step 21: Inspect H-I (12)
() => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Берем ребро H-I (вес 12)',
 description: 'Соединяет фиолетовую H и последнюю I',
 type: 'info'
 }, ...prev]));
},
// Step 22: Include H-I
() => {
 setVertices(prev => prev.map(v => v.id === 8 ? {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Успех: MST построено Краскалом!',
 description: 'Все 9 вершин окрашены в единый фиолетовый цвет'
 }

```

```

 type: 'success'
 }, ...prev]);
},
// Step 4: Pop A-D (5)
() => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
 description: 'Плесень расширяется в сторону верш
 type: 'info'
 }, ...prev]);
},
// Step 5: Include A-D, add A's edges to heap
() => {
 setVertices(prev => prev.map(v => v.id === 0 ? {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 eap' } : e));
 setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D
 setLogs(prev => [{
 title: 'Успех: Плесень поглотила вершину A',
 description: 'Новое ребро A-B(2) добавлено в куч
 type: 'success'
 }, ...prev]);
},
// Step 6: Pop A-B (2)
() => {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
 description: 'Плесень стремительно бежит к верши
 type: 'info'
 }, ...prev]);
},
// Step 7: Include A-B, add B's edges
() => {
 setVertices(prev => prev.map(v => v.id === 1 ? {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 eap' } : e));

```

```

() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setHeapQueue(['E-F (7)', 'D-G (8)', 'C-F (9)', 'E
 setLogs(prev => [{
 title: 'Отказ: Игнорируем внутреннее ребро',
 description: 'Ребро B-E отброшено. Плесень не р
 type: 'error'
 }, ...prev]));
},
// Step 12: Pop E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
 description: 'Плесень из центра Токио (E) тянет
 type: 'info'
 }, ...prev]));
},
// Step 13: Include E-F, add F's edges
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 eap' } : e));
 setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
 setLogs(prev => [{
 title: 'Успех: Вершина F захвачена',
 description: 'Ребро F-I(13) добавлено в кучу. Р
 type: 'success'
 }, ...prev]));
},
// Step 14: Pop D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
 description: 'Плесень расширяется вниз к G.',
 type: 'info'
 }, ...prev]));
},

```

```

 type: 'info'
 }, ...prev]);
},
// Step 19: Include G-H, add H's edges
() => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 eap' } : e));
 setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I
 setLogs(prev => [{
 title: 'Успех: Вершина H захвачена',
 description: 'В кучу добавлено H-I(12).',
 type: 'success'
 }, ...prev]);
},
// Step 20: Pop E-H (11) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Куча выдает E-H (вес 11)',
 description: 'Обе вершины E и H уже в плесени.',
 type: 'warning'
 }, ...prev]);
},
// Step 21: Reject E-H
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setHeapQueue(['H-I (12)', 'F-I (13)']));
 setLogs(prev => [{
 title: 'Отказ: Игнорируем E-H',
 description: 'Выбрасываем из кучи.',
 type: 'error'
 }, ...prev]);
},
// Step 22: Pop H-I (12)
() => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{

```

```

 description: 'Каждая из 9 деревень одновременно
 А выбирает A-B(2), D выбирает D-E(3), а G вы
 !',
 type: 'info'
 }, ...prev]);
},
// Step 2: Concurrently merge into Kolkhozes (Component)
() => {
 // Concurrently build chosen roads. Merge into:
 // Component 1: {A, B, C} (colored red)
 // Component 2: {D, E, F, G, H, I} (colored gold/
 setVertices(prev => prev.map(v =>
 [0, 1, 2].includes(v.id) ? { ...v, color: 'red'
 }));
 setEdges(prev => prev.map(e => {
 if (['0-1', '1-2'].includes(e.id)) {
 return { ...e, status: 'included', color: 'red'
 }
 if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
 return { ...e, status: 'included', color: 'blue'
 }
 return e;
 }));
 setLogs(prev => [{
 title: 'Слияние: Деревни объединились в колхозы
 description: 'Все выбранные дороги построены ра
 оз {D, E, F, G, H, I}.',
 type: 'success'
 }, ...prev]);
},
// Step 3: Five-Year Plan 2 (Inspecting bridge between
() => {
 // Now both kolkhozes look at all outgoing roads
 // Outgoing roads between {A, B, C} and {D, E, F,
 // A-D (5), B-E (6), C-F (9).
 // Cheapest is A-D (5).
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{

```



```

 setStepIndex(stepIndex + 1);
 }
};

const handleReset = {algo: 'kruskal' | 'prim' | 'boru
 setActiveAlgo(algo);
 setVertices(initialVertices);
 setEdges(initialEdges);
 setStepIndex(0);
 setHeapQueue([]);
 if (algo === 'kruskal') {
 setLogs([
 {
 title: 'Введение в раскраску (Краскал)',
 description: 'Представь, что все 9 вершин гра
 type: 'info',
 },
]));
 } else if (algo === 'prim') {
 setLogs([
 {
 title: 'Введение в Плесень (Прима)',
 description: 'Логика «Плесени»: выбираем стар
 едь (Heap) и захватываем соседей!',
 type: 'info',
 },
]));
 } else {
 setLogs([
 {
 title: 'Введение в Коллективизацию (Борувка)',
 description: 'Логика «Борувки»: Каждая из 9 д
 чтобы объединиться в колхоз.',
 type: 'info',
 },
]));
 }
};

```

```

 Супер-Интерактив 3 в 1

 <h3 className="text-2xl font-bold text-wh
 </div>
 <p className="text-slate-400 text-sm">
 Наблюдай за работой Краскала (Раскраска),
 </p>
 </div>

 {/* Algorithm Tabs */}
 <div className="flex flex-wrap items-center g
 <div className="flex items-center bg-slate-
 <button
 onClick={() => handleReset('kruskal')}
 className={
 "flex items-center gap-1.5 px-3 py-2
 +
 (activeAlgo === 'kruskal'
 ? 'bg-indigo-600 text-white shadow-
 : 'text-slate-400 hover:text-slate-
)
 >
 <GitGraph className="w-3.5 h-3.5" />
 Краскал (Билет 18)
 </button>
 <button
 onClick={() => handleReset('prim')}
 className={
 "flex items-center gap-1.5 px-3 py-2
 +
 (activeAlgo === 'prim'
 ? 'bg-emerald-600 text-white shadow
 : 'text-slate-400 hover:text-slate-
)
 >
 <Layers className="w-3.5 h-3.5" />
 Прима (Билет 19)
 </button>

```

```

 >
 <SkipForward className="w-4 h-4" />
 {stepIndex === 0 ? 'Начать' : stepIndex}
 </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-3" >
 {/* Graph Canvas */}
 <div className="lg:col-span-2 bg-slate-950 rounded-3xl p-4 active min-h-[480px] overflow-hidden">

 {/* Legend */}
 <div className="absolute top-4 left-4 bg-slate-900 p-4 flex flex-wrap flex-center gap-3 text-xs">
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-white"></div>
 {activeAlgo === 'kruskal' ? (
 <>
 <div className="flex items-center gap-1">
 <div className="w-3 h-3 rounded-full bg-white"></div>
 </div>
 <div className="flex items-center gap-1">
 <div className="w-3 h-3 rounded-full bg-white"></div>
 </div>
 <div className="flex items-center gap-1">
 <div className="w-3 h-3 rounded-full bg-white"></div>
 </div>
 </>
) : activeAlgo === 'prim' ? (
 <>
 <div className="flex items-center gap-1">
 <div className="w-3 h-1 bg-sky-600"></div>
 </div>
 <div className="flex items-center gap-1">
 <div className="w-3 h-3 rounded-full bg-white"></div>
 </div>
 </>
) : null}
 </div>
 </div>
 </div>
</div>

```

```

 y2={v.y + "%"}
 stroke={strokeColor}
 strokeWidth={isInspecting ? 4.5 : 1}
 strokeDasharray={isRejected ? '4,4' : ''}
 className={"transition-all duration-300"}
 />
 <text
 x={({u.x + v.x} / 2 + "%")}
 y={({u.y + v.y} / 2 + "%")}
 fill={isInspecting ? '#f59e0b' : '#94a3b8'}
 fontSize="4.5"
 fontFamily="monospace"
 fontWeight="bold"
 textAnchor="middle"
 dominantBaseline="middle"
 className="select-none filter drop-shadow"
 dy="-1.5"
 >
 {edge.weight}
 </text>
 </g>
);
}
</svg>

{/* HTML Overlay for Vertices */}
<div className="absolute inset-0 w-full h-full" >
 {vertices.map(vertex => (
 <div
 key={vertex.id}
 style={{ top: vertex.y + "%", left: vertex.x + "%", width: 100px, height: 100px }}
 className={"absolute -translate-x-1/2 -translate-y-1/2 font-bold text-base border-2 transition-all duration-300"}
 id={vertex.id}
 >
 {vertex.label}
 {vertex.id === 4 && activeAlgo === 'path' ? 'Path' : ''}
 </div>
))}
</div>

```

```

 <span key={idx} className={"px-2 py-2
0 shadow-md animate-pulse" : "bg-slate-800
 {item}

))
 })
 </div>
</div>
})

<div className="flex-1 p-4 overflow-y-auto"
 {logs.map((log, idx) => {
 <div
 key={idx}
 className={
 "p-4 rounded-xl border transition-a
 {log.type === 'success'
 ? 'bg-emerald-950/40 border-emera
 : log.type === 'warning'
 ? 'bg-amber-950/40 border-amber-5
 : log.type === 'error'
 ? 'bg-rose-950/40 border-rose-500
 : 'bg-slate-900/60 border-slate-7
 }
 }
 >
 <div className="flex items-center gap-2"
 {log.type === 'success' && <CheckCi
 {log.type === 'warning' && <HelpCir
 {log.type === 'error' && <XCircle c
 {log.type === 'info' && <Play class
 <h5 className="font-bold text-sm le
 </div>
 <p className="text-xs text-slate-300
 </div>
)})
</div>
</div>
</div>

```

```

<div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-mono">
 <Clock className="w-3.5 h-3.5 text-indigo-400" />
 </p>
 <p className="text-2xl font-black text-white">Clock</p>
 <p className="text-slate-400 text-xs font-mono">
 <Award className="w-3.5 h-3.5 text-indigo-400" />
 </p>
 <p className="text-2xl font-black text-white">Award</p>
 <p className="text-slate-400 text-xs font-mono">
 <Code className="w-3.5 h-3.5 text-indigo-400" />
 </p>
 <p className="text-2xl font-black text-white">Code</p>
</div>
<div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-mono">
 <Code className="w-3.5 h-3.5 text-indigo-400" />
 </p>
 <p className="text-2xl font-black text-white">Code</p>
 <p className="text-slate-400 text-xs font-mono">
 <Award className="w-3.5 h-3.5 text-indigo-400" />
 </p>
 <p className="text-2xl font-black text-white">Award</p>
 <p className="text-slate-400 text-xs font-mono">
 <Clock className="w-3.5 h-3.5 text-indigo-400" />
 </p>
 <p className="text-2xl font-black text-white">Clock</p>
</div>
<div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-mono">
 <Code className="w-3.5 h-3.5 text-indigo-400" />
 </p>
 <pre className="text-xs text-emerald-400 font-mono">
 80 flex-1">

```

```
{`# 1. Инициализируем DSU`}
```

```
def find(i):
```

```
 if parent[i] == i: return i
```

```
 parent[i] = find(parent[i])
```

```
 return parent[i]
```

```
def union(i, j):
```

```
 r_i, r_j = find(i), find(j)
```

```
 if r_i != r_j:
```

```
 parent[r_i] = r_j
```

```

 </div>
 <div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-1">
 <p className="text-slate-400 text-xs font-medium">
 <Code className="w-3.5 h-3.5 text-emerald-400">
 </p>
 <pre className="text-xs text-emerald-400 font-mono">
180 flex-1">
{`import heapq

def prim(graph, start):
 visited = [False] * len(graph)
 heap = [(0, start, -1)] # (weight, node, parent)
 mst = []

 while heap:
 w, u, prev = heapq.heappop(heap)
 if visited[u]: continue
 visited[u] = True
 if prev != -1: mst.append((prev, u, w))

 for next_w, v in graph[u]:
 if not visited[v]:
 heapq.heappush(heap, (next_w, v, u))
 return mst`}
 </pre>
 </div>
 </div>
 </div>
 </div>
}}

{activeCheatTab === 'boruvka' && (
 <div className="grid grid-cols-1 lg:grid-cols-2">
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-1">
 <p className="text-slate-400 text-xs font-medium">
 <Clock className="w-3.5 h-3.5 text-rose-500">
 </p>

```

```
 if bridge != -1 and union(bridge[0], bridge[1]) {
 mst.append(bridge)` }
 </pre>
 </div>
</div>
</div>
 })
</div>
</div>
};
```

---

ФАЙЛ 51/83: src/components/MnemonicCards.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';

const cards = [
 {
 img: dijkstraImg,
 alt: 'Добрый робот Дейкстра',
 title: 'Добрый (Дейкстра)',
 desc: 'Быстрый, жадный, но работает только с',
 highlight: 'положительными',
 highlightColor: 'text-emerald-400',
 rest: 'весами. Подсунь отрицательное ребро – сломает',
 complexity: 'O((V+E) log V)',
 border: 'border-blue-500/30 hover:border-blue-400/60',
 titleColor: 'text-blue-400',
 badge: 'text-blue-400/70 bg-blue-950/40',
 },
 {
```



```
 <p className="text-slate-400 text-sm">
 {c.desc}
 </p>
 <div className={`mt-3 text-xs font-mono px-1`>
 </div>
 </div>
)})
 </div>
);
}
```

---

ФАЙЛ 52/83: src/components/MobileToc.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useEffect, useState } from "react";
import { List, X } from "lucide-react";
import { Sidebar } from "../Sidebar";

interface Props {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
 /** Заголовок текущей темы — показываем в кнопке, что */
 currentTitle: string;
 index: number;
 total: number;
}

/**
 * Мобильное содержание: липкая панель снизу-сверху + в
 * На десктопе не рендерится (там обычный сайдбар).
 */
export const MobileToc: React.FC<Props> = ({ selectedChapterId,
 const [open, setOpen] = useState(false);

 useEffect(() => {
```

```

 <button
 onClick={() => setOpen(false)}
 className="p-2 -mr-2 text-slate-400 hover:text-white"
 aria-label="Заккрыть содержание"
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 <div className="flex-1 min-h-0">
 <Sidebar
 selectedChapterId={selectedChapterId}
 setSelectedChapterId={setSelectedChapterId}
 onNavigate={() => setOpen(false)}
 />
 </div>
 </div>
 </div>
)
</>
);
};
```

---

ФАЙЛ 53/83: src/components/Navbar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, Cpu } from 'lucide-react';
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';

interface NavbarProps {
 activeTab: 'guide' | 'simulator';
 setActiveTab: (tab: 'guide' | 'simulator') => void;
}
```

```

 uration-200 whitespace-nowrap ${
 activeTab === 'guide'
 ? 'bg-indigo-600 text-white shadow-lg s
 : 'text-slate-400 hover:text-white'
 }` }
 >
 <BookOpen className="w-4 h-4" /> Учебное по
</button>
<button
 id="tab-simulator-btn"
 onClick={() => setActiveTab('simulator')}
 className={`flex-1 lg:flex-none flex items-
 uration-200 whitespace-nowrap ${
 activeTab === 'simulator'
 ? 'bg-indigo-600 text-white shadow-lg s
 : 'text-slate-400 hover:text-white'
 }` }
>
 <Cpu className="w-4 h-4" /> Тренажёры
</button>
<a
 id="download-pdf-btn"
 href="/export/full_code_all_pages.pdf"
 download="full_code_all_pages.pdf"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
 over:bg-emerald-600/20 border border-emeral
 title="Скачать полный PDF сборник всех стра
>
 <FileDown className="w-4 h-4" /> Скачать PD

<a
 id="download-txt-btn"
 href="/export/full_code_all_pages.txt"
 download="full_code_all_pages.txt"
 target="_blank"
 rel="noreferrer"

```

```

00/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
<svg className="w-full h-full absolute inset-0"
 {(() => {
 const pts = [
 {id:0,x:160,y:40},
 {id:1,x:270,y:100},
 {id:2,x:230,y:230},
 {id:3,x:90,y:230},
 {id:4,x:50,y:100}
];
 const edges = [
 [0,1],[0,2],[0,3],[0,4],
 [1,2],[1,3],[1,4],
 [2,3],[2,4],
 [3,4]
];
 function findPt(id:number) { return pts.find(p => p.id === id); }
 function intersect(a:number,b:number,c:number,d:number) {
 if (a===c||a===d||b===c||b===d) return false;
 const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
 function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y);
 }
 const crosses: number[] = [];
 for(let i=0;i<edges.length;i++) {
 for(let j=i+1;j<edges.length;j++) {
 if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1])) {
 if (!crosses.includes(i)) crosses.push(i);
 if (!crosses.includes(j)) crosses.push(j);
 }
 }
 }
 const lines = []; const circles = [];
 for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
 if (!crosses.includes(i)) lines.push([p1.x,p1.y,p2.x,p2.y]);
 else circles.push([p1.x,p1.y,p2.x,p2.y]);
 }
 })}

```

```

const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
if (!p1||!p2||!p3||!p4) return false;
function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number)
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
}
return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)>
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y);
}
const crosses:number[]=[];
for(let i=0;i<edges.length;i++) for(let j=i+1;j<edges.length;j++)
 if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1]))
 if (!crosses.includes(i)) crosses.push(i);
 if (!crosses.includes(j)) crosses.push(j);
}
const lines = []; const circles = [];
for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
 const xing=crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x} x2={p2.x} y1={p1.y} y2={p2.y}
 stroke={xing?"#f43f5e":"#4f46e5"} stroke-width={2}/>);
}
const labels = ["\u04141","\u04142","\u04143","\u04144"];
for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" stroke="#4f46e5" stroke-width={2}/>
 <text x={p.x} y={p.y+3} textAnchor="middle">{labels[p.id]}</text>
 </g>);
}
return <>{lines}{circles}</>;
})();
</svg>
<div className="absolute bottom-2 left-2 right-2 z-20">
 V=6, E=9 {'>'} 2V-4 = 8 (двудольный) {'-->'}
</div>
</div>
</div>

```

```

 { name: 'Бизнесмен', emoji: ' ', color: 'from-slate-500',
 tartan: ' ',
 },
];

```

```
let counter = 3;
```

```

export const QueueViz: React.FC = () => {
 const [queue, setQueue] = useState<Customer[]>([
 { id: 'q1', name: 'Студент Вася', emoji: ' ', color: 'from-slate-500',
 state: 'голода после матана!', animState: 'idle' },
 { id: 'q2', name: 'Кодер Семён', emoji: ' ', color: 'from-slate-500',
 state: 'ищу ИИ за порцию борща!', animState: 'idle' },
 { id: 'q3', name: 'Кот Борис', emoji: ' ', color: 'from-slate-500',
 state: 'ез сметаны, плиз.', animState: 'idle' },
]);

```

```

 const [servedHistory, setServedHistory] = useState<string[]>([]);
 const [isServing, setIsServing] = useState(false);
 const [shakeEmpty, setShake] = useState(false);
 const [log, setLog] = useState<string[]>([]);

```

```
const addLog = (msg: string) => setLog(prev => [...prev, msg]);
```

```
// ENQUEUE: Встать в конец очереди (хвост / Tail)
```

```

const enqueue = useCallback(() => {
 if (isServing) return;
 if (queue.length >= 6) {
 addLog('⚠ Хвост очереди уже на улице, повару тяжело!');
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }

```

```

 const preset = PRESETS[counter % PRESETS.length];
 counter++;

```

```

 const newGuest: Customer = {
 id: Date.now().toString(),

```

```

 setIsServing(false);
 addLog(` ${headGuest.name} поел и ушел сытым.
 }, 400);
 }, 900);
}, [queue, isServing]));

const reset = () => {
 counter = 3;
 setQueue([
 { id: 'r1', name: 'Студент Вася', emoji: ' ', color: 'red', text: 'Я
т голода после матана!', animState: 'in' },
 { id: 'r2', name: 'Кодер Семён', emoji: ' ', color: 'blue', text: 'Я
апишу ИИ за порцию борща!', animState: 'in' },
 { id: 'r3', name: 'Кот Борис', emoji: ' ', color: 'green', text: 'Я
без сметаны, плиз.', animState: 'in' },
]);
 setServedHistory([]);
 setIsServing(false);
 setLog([' Столовая сброшена. Снова голодная толпа!']);
 setTimeout(() => {
 setQueue(prev => prev.map(c => ({ ...c, animState: 'out' })), 500);
 });

 return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-indigo-950/40 border border-indigo-400">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-indigo-400 text-2xl">ПРАВИЛО</h3>
 <p className="text-xs text-slate-300 mt-1 leading-4">
 В очереди за супом всё честно. Кто первый встал, тот и ест.

 Новые гости встают строго в конец очереди.

 Здесь нет обхода и блата – только строгий порядок.
 </p>
 </div>
 </div>
 </div>
);
};

```

```

<div className="absolute top-4 left-4 text-[10px] font-mono">
 Очередь голодных гостей (Буфер FIFO / BFS)
</div>

```

```

<div className="flex-1 flex flex-col justify-between">
 <div className="flex items-end justify-between">

```

```

 {/* ПОВАРИХА И КОТЕЛ С СУПОМ */}
 <div className="flex flex-col items-center">
 <div className="relative">
 <div className="absolute -top-10 left-10">
 <div className="w-1.5 h-6 bg-slate-500">
 <div className="w-2 h-4 bg-slate-400">
 </div>
 </div>

 </div>
 <div className="text-center">

 ПОВАР

 </div>
 </div>
 </div>

```

```

 {/* РАЗДЕЛИТЕЛЬНАЯ СТРЕЛКА */}
 <div className="flex flex-col items-center">

 </div>

```

```

 {/* СПИСОК ОЧЕРЕДИ */}
 <div className="flex-1 flex items-end gap-2">
 {queue.length === 0 ? (
 <div className="flex-1 flex flex-col justify-between">

 <div>
 <p className="text-xs font-bold font-mono">
 <p className="text-[10px]">Все сыты

```



```

 <span className={`
 absolute -bottom-2 text-[
 ${isHead ? 'bg-amber-500' :
 `}>
 {isHead && isTail ? 'h+t'

 })
 </div>
 </div>
);
})
})
</div>

</div>
</div>

{ /* Пол кухни */ }
<div className="w-full h-3 bg-gradient-to-r f
</div>

{ /* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */ }
<div className="md:col-span-3 bg-slate-900 round
 <div className="absolute top-4 left-4 text-[1
 Сытые гости (Архив FIFO)
 </div>

 <div className="absolute top-4 right-4 flex i
 order-emerald-800/80 text-[9px] font-mono">
 <Sparkles className="w-3.5 h-3.5 animate-pu
 СЫТЫ
 </div>

 <div className="flex-1 flex flex-col justify-
 {servedHistory.length === 0 ? (
 <div className="flex-1 flex flex-col item

 <p className="text-xs font-bold font-mo

```

---

ФАЙЛ 56/83: src/components/SalmonAutomatonWidget.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

---

```
import React, { useState, useEffect } from 'react';
import { Play, RotateCcw, Plus, Volume2, Info } from 'lucide-react';

interface TrieNode {
 id: number;
 char: string;
 parent: number;
 children: { [key: string]: number };
 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
 depth: number;
 x?: number;
 y?: number;
}

export const SalmonAutomatonWidget: React.FC = () => {
 const [words, setWords] = useState<string[]>(['CAR', 'FISH', 'SALMON']);
 const [newWord, setNewWord] = useState('');
 const [nodes, setNodes] = useState<{ [key: number]: TrieNode }>({});
 const [simulationStep, setSimulationStep] = useState<number>(0);
 const [bfsQueue, setBfsQueue] = useState<number[]>([]);
 const [currentNode, setCurrentNode] = useState<number>(0);
 const [speechText, setSpeechText] = useState<string>('Привет! Я Эхо-Лосось Сеня! Введи слова в словарь и попробуй!');
 const [isTalking, setIsTalking] = useState<boolean>(false);
 const [isBlinking, setIsBlinking] = useState<boolean>(false);
```

```

 words: [],
 depth: newNodes[curr].depth + 1
 };
 }
 curr = newNodes[curr].children[c];
}
if (!newNodes[curr].words.includes(word)) {
 newNodes[curr].words.push(word);
 newNodes[curr].is_terminal = true;
}
});

let currentX = 0;
const paddingX = 70;
const paddingY = 80;

const layoutDFS = (u: number, depth: number) => {
 const childKeys = Object.values(newNodes[u].children);
 newNodes[u].y = 40 + depth * paddingY;

 if (childKeys.length === 0) {
 newNodes[u].x = 40 + currentX * paddingX;
 currentX++;
 } else {
 let leftX = -1;
 let rightX = -1;
 childKeys.forEach(v => {
 layoutDFS(v, depth + 1);
 if (leftX === -1) leftX = newNodes[v].x!;
 rightX = newNodes[v].x!;
 });
 newNodes[u].x = leftX === -1 ? 40 + currentX * paddingX : (leftX + rightX) / 2;
 if (leftX === -1) currentX++;
 }
};

layoutDFS(0, 0);

```

```
// Запуск BFS
const startBFS = () => {
 const rootChildren = Object.values(nodes[0].children);
 const queue = [...rootChildren];
 const updatedNodes = { ...nodes };
 rootChildren.forEach((childId) => {
 updatedNodes[childId].fail = 0;
 });

 setNodes(updatedNodes);
 setBfsQueue(queue);
 setSimulationStep(1);
 setCurrentNode(null);
 setSpeechText(
 'Начинаем обход в ширину (BFS)! Все вершины на главной
 очереди суффикса просто нет. Жми «Следующий шаг»!'
);
};

// Один шаг BFS
const stepBFS = () => {
 if (bfsQueue.length === 0) {
 setSimulationStep(2);
 setCurrentNode(null);
 setSpeechText(
 'Ура! Очередь пуста! Мы успешно построили полную таблицу
 переходов!'
);
 return;
 }

 const r = bfsQueue[0];
 const newQueue = bfsQueue.slice(1);
 const updatedNodes = { ...nodes };
 const rNode = updatedNodes[r];
 const childrenIds = Object.values(rNode.children);

 let logs = `Рассматриваем вершину #${r} (${rNode.children}`;
```

```

 Интерактивный конструктор: Говорящий Эхо-
 </h4>
 <p className="text-sm text-slate-400">
 Построение дерева и расчет суффиксных ссы
 </p>
</div>
</div>
<div className="flex items-center gap-2">
 <button
 onClick={() => buildInitialTrie(words)}
 className="flex items-center gap-1 bg-slate
ransition-colors"
 title="Сбросить автомат"
 >
 <RotateCcw className="w-4 h-4" /> Сбросить
 </button>
</div>
</div>

/* Верхняя панель: Говорящий лосось и диалоговое
<div className="grid grid-cols-1 md:grid-cols-3 g
 /* Аватар Лосося (Сеня) с анимацией моргания и
 <div className="flex flex-col items-center just
 <div className="w-40 h-40 relative flex items
 full border border-blue-500/30 shadow-inner
 /* SVG Лосося */
 <svg
 viewBox="0 0 200 200"
 className={`w-36 h-36 transition-transform
 isTalking ? 'scale-105 drop-shadow-[0_0_
]`
 >
 /* Тело лосося */
 <ellipse cx="100" cy="100" rx="70" ry="45
 <path d="M 40 85 Q 100 55 160 85 Q 100 115
 /* Хвост */
 <path
```

```

 <div className="absolute -top-1 right-4 text-slate-800">
 <div className="absolute top-8 right-1 text-slate-800">
 </div>

 Эхо-Лосось Сеня

 </div>

```

```

{ /* Пузырь с текстом (Баббл) */ }
<div className="md:col-span-2 bg-slate-800 border border-slate-800 rounded-tween min-h-[140px]">
 { /* Треугольник баббла */ }
 <div className="absolute -left-3 top-1/2 -translate-x-3 translate-y-1/2 bg-slate-800 border-b-[12px] border-b-transparent">

```

```

 <div className="flex items-center space-x-2 text-slate-800">
 <Volume2 className={`w-4 h-4 ${isTalking ? 'animate-pulse' : ''}` />
 Прямое вещание из аквариума:
 </div>

```

```

 <p className="text-slate-200 text-base leading-relaxed">
 {speechText}
 </p>

```

```

 <div className="mt-4 pt-3 border-t border-slate-800">
 <div className="text-xs text-slate-400 flex items-center">
 <Info className="w-3.5 h-3.5 text-blue-400" />
 Статус: {simulationStep === 0 ? 'Бор готов к работе!' : 'Бор занят'}
 </div>

```

```

 <div className="flex gap-2">
 {simulationStep === 0 && (
 <button
 onClick={startBFS}
 className="bg-emerald-600 hover:bg-emerald-700 text-white px-2 py-1 shadow-lg shadow-emerald-900/40 transition-colors">

```

```

 if (n.y && n.y > maxY) maxY = n.y;
 });
 const svgWidth = Math.max(maxX + 80, 400);
 const svgHeight = Math.max(maxY + 60, 200);

 return (
 <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`}
 <defs>
 <marker id="arrowhead" markerWidth="6"
 <polygon points="0 0, 6 3, 0 6" fill=
 </marker>
 <marker id="fail-arrow" markerWidth="6"
 <polygon points="0 0, 6 3, 0 6" fill=
 </marker>
 </defs>

 {/* Draw Edges */}
 {Object.values(nodes).map(node => {
 if (node.id === 0) return null;
 const parent = nodes[node.parent];
 if (!parent.x || !parent.y || !node.x ||

 const isCurrent = currentNode === node.

 return (
 <g key={`edge-${node.id}`}>
 <line
 x1={parent.x}
 y1={parent.y + 16}
 x2={node.x}
 y2={node.y - 16}
 stroke={isCurrent ? '#60a5fa' : '#ccc'}
 strokeWidth="2"
 markerEnd="url(#arrowhead)"
 />
 <text
 x={(parent.x + node.x) / 2 - 10}
 y={(parent.y + node.y) / 2}

```

```

{Object.values(nodes).map(node => {
 if (!node.x || !node.y) return null;
 const isRoot = node.id === 0;
 const isCurrent = currentNode === node;
 const isInQueue = bfsQueue.includes(node);

 let fill = '#1e293b';
 let stroke = '#475569';

 if (isCurrent) {
 fill = '#2563eb';
 stroke = '#60a5fa';
 } else if (isInQueue) {
 fill = '#b45309';
 stroke = '#fbbf24';
 } else if (node.is_terminal) {
 fill = '#059669';
 stroke = '#34d399';
 }

 return (
 <g key={`node-${node.id}`}>
 <circle
 cx={node.x}
 cy={node.y}
 r="16"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 />
 <text
 x={node.x}
 y={node.y + 4}
 fill="white"
 fontSize={isRoot ? "10" : "12"}
 fontWeight="bold"
 textAnchor="middle"
 />
 >

```



```

 title="Удалить"
 >
 ×
 </button>

 }}}
</div>
</div>

<form onSubmit={handleAddWord} className="flex flex-col gap-2" >
 <input
 type="text"
 value={newWord}
 onChange={(e) => setNewWord(e.target.value)}
 placeholder="НОВОЕ СЛОВО..."
 maxLength={8}
 className="bg-slate-800 border border-slate-700 outline-none focus:border-blue-500"
 />
 <button
 type="submit"
 className="bg-slate-700 hover:bg-slate-600 transition-colors"
 >
 <Plus className="w-3.5 h-3.5" /> Добавить
 </button>
</form>
</div>

{/* Таблица Вершин и суффиксных ссылок */}
<div className="lg:col-span-2 bg-slate-900/50 p-4" >
 <h5 className="text-white font-bold mb-3 flex" >

 Таблица переходов и fail-

 Всего вершин: {Object.keys(nodes).length}

 </h5>
</div>

```

```

 : isInQueue
 ? 'bg-amber-400'
 : 'bg-slate-600'
 `}
 >
 #{node.id}
 {node.id === 0 && <span className=
</td>
<td className="py-2.5 px-3">
 <span className="bg-slate-800 b
 {node.char}

</td>
<td className="py-2.5 px-3 text-s
 {node.id === 0 ? '-' : `#${node
</td>
<td className="py-2.5 px-3">
 {node.id === 0 ? {
 <span className="text-slate-5
 } : {
 <span className="bg-indigo-95
 #{node.fail}

 }}
</td>
<td className="py-2.5 px-3">
 {node.id === 0 || node.term_lin
 <span className="text-slate-5
 } : {
 <span className="bg-rose-950 l
0 transition-colors cursor-help" title={`Cк
 #{node.term_link}

 }}
</td>
<td className="py-2.5 px-3 text-s
 {Object.keys(node.children).len
 ? 'ø'

```

```

 codeLine: number;
 treeState: Record<number, number>; // node index
 highlightNodes: number[]; // current nodes
 mergeChildren?: [number, number]; // pair being merged
 arrayHighlight?: [number, number]; // [L, R] range
 result?: number; // partial result
}

```

// ===== PURE LOGIC: build tree =====

```

function buildTreeFull(arr: number[]): Record<number, number> {
 const n = arr.length;
 const tree: Record<number, number> = {};
 function go(v: number, l: number, r: number) {
 if (l === r) { tree[v] = arr[l]; return; }
 const m = (l + r) >> 1;
 go(2 * v, l, m);
 go(2 * v + 1, m + 1, r);
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 }
 go(1, 0, n - 1);
 return tree;
}

```

// ===== STEP GENERATORS =====

// 1. Build steps (recursive top-down)

```

function genBuildSteps(arr: number[]): Step[] {
 const n = arr.length;
 const steps: Step[] = [];
 let id = 0;
 const tree: Record<number, number> = {};

```

```

 const CODE = [
 "build(v, l, r) {", // 1
 " if (l === r) {", // 2
 " tree[v] = A[l]; return", // 3
 " }", // 4
 " mid = (l + r) >> 1", // 5

```

```

let id = 0;

function go(v: number, l: number, r: number, ql: number, qr: number) {
 if (ql > r || qr < l) {
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 2, treeState: tree, highlightNodes: [v], result: 0,
 });
 return 0;
 }
 if (ql <= l && r <= qr) {
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 4, treeState: tree, highlightNodes: [v], result: tree[v],
 });
 return tree[v];
 }
 const m = (l + r) >> 1;
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 6, treeState: tree, highlightNodes: [v], result: 0,
 });
 const left = go(2 * v, l, m, ql, qr);
 const right = go(2 * v + 1, m + 1, r, ql, qr);
 const res = left + right;
 steps.push({ id: id++, desc: `Возврат в узел ${v}:`,
 codeLine: 8, treeState: tree, highlightNodes: [v], result: res,
 rangeChildren: [2 * v, 2 * v + 1], arrayHighlight: [2 * v, 2 * v + 1],
 });
 return res;
}
const ans = go(1, 0, n - 1, qL, qR);
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}]:`,
 codeLine: 10, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: ans,
});
return steps;
}

```

// 3. Bottom-Up Query steps (iterative on implicit tree)

```

function genQueryBottomUpSteps(arr: number[], qL: number, qR: number) {
 const n = arr.length;
 // Build iterative tree: leaves at [n..2n-1]
 const tree: Record<number, number> = {};
 for (let i = 0; i < n; i++) tree[n + i] = arr[i];
 for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i] + tree[2 * i + 1]);

 const steps: Step[] = [];

```

```
function buildVisualTree(treeState: Record<number, number>) {
 if (l === r) return { v, l, r, val: treeState[v] ?? 0 };
 const m = (l + r) >> 1;
 return { v, l, r, val: treeState[v] ?? null, left: buildVisualTree(
 treeState, v + 1, m + 1, r) };
}
```

// ===== CODE SNIPPETS =====

```
const BUILD_CODE = [
 "build(v, l, r) {",
 " if (l === r) { tree[v] = A[l]; return; }",
 " // -----",
 " // -----",
 " mid = (l + r) >> 1;",
 " build(2*v, l, mid);",
 " build(2*v+1, mid+1, r);",
 " tree[v] = tree[2*v] + tree[2*v+1];",
 "}"
];
```

```
const QUERY_TD_CODE = [
 "query(v, l, r, ql, qr) {",
 " if (ql > r || qr < l) return 0;",
 " // -----",
 " if (ql <= l && r <= qr) return tree[v];",
 " // -----",
 " mid = (l + r) >> 1;",
 " // спускаемся в обоих детей",
 " return query(left) + query(right);",
 "}"
];
```

```
const QUERY_BU_CODE = [
 "queryBU(ql, qr) {",
 " res = 0;",
 " l = ql + n; r = qr + n + 1;",
 " // -----",
 " while (l < r) {",
 " if (l & 1) res += tree[l++];",
 " l = l >> 1; r = r >> 1;
 " }
];
```

```

if (!p) return null;
const { step, idx, setIdx, playing, setPlaying, speed
const n = arr.length;

const vTree = useMemo(() => buildVisualTree(step.tree

const renderNode = useCallback((nd: VNode): React.Rea
 const isHigh = step.highlightNodes.includes(nd.v);
 const isChild = step.mergeChildren?.includes(nd.v);
 const has = nd.val !== null;

 let cls = 'bg-slate-800 border-slate-700 text-slate
 if (isHigh) cls = `${clr.ring} text-white ring-2 sc
 else if (isChild) cls = `${clr.childRing} text-ambe
 else if (has) cls = `${clr.filled} text-slate-200`;

 return (
 <div key={nd.v} className="flex flex-col items-ce
 <div className={`flex flex-col items-center jus
 ${cls}`}>
 <span className="text-[7px] opacity-60 font-m
 [{nd.l
 <span className="text-xs font-extrabold font-
 </div>
 {(nd.left || nd.right) && (
 <div className="flex flex-col items-center mt
 <div className="w-px h-2 bg-slate-700" />
 <div className="flex justify-around w-full
 {nd.left && renderNode(nd.left)}
 {nd.right && renderNode(nd.right)}
 </div>
 </div>
 </div>
)}
 </div>
);
}, [step, clr]);

return (

```

```

 : 'bg-amber-600/20 border-l-2 border-amber-600'

 </div>
);
 }}}
</pre>

```

```

{/* Description + result */}
<div className="px-3 py-2.5 bg-slate-900/60 border-t-1 border-slate-800" >
 <span className={`mt-0.5 inline-block w-2 h-2 rounded-full`} style={{
 background-color: {step.result !== undefined && {
 : color === 'emerald' ? 'text-emerald-300' : 'text-slate-400'
 }}
 }}>
 {step.desc}
 {step.result !== undefined && {
 <span className={`ml-auto shrink-0 font-mono text-sm`} style={{
 : color === 'emerald' ? 'text-emerald-300' : 'text-slate-400'
 }}>
 = {step.result}

 }}
</div>

```

```

{/* Controls */}
<div className="px-3 py-2.5 bg-slate-950 border-t-1 border-slate-800" >
 <div className="flex items-center bg-slate-900/60 border-t-1 border-slate-800" >
 <button onClick={() => { setPlaying(false); setSpeed(1200); }}>
 <RotateCcw className="w-4 h-4 text-slate-400" />
 </button>
 <button onClick={() => { setPlaying(false); setSpeed(1200); }}>
 <RotateCcw className="w-4 h-4 text-slate-400" />
 </button>
 <button onClick={() => setPlaying(!playing)}>
 <Play className="w-4 h-4 text-slate-400" />
 </button>
 <button onClick={() => { setPlaying(false); setSpeed(1200); }}>
 <RotateCcw className="w-4 h-4 text-slate-400" />
 </button>
 <button onClick={() => { setPlaying(false); setSpeed(1200); }}>
 <RotateCcw className="w-4 h-4 text-slate-400" />
 </button>
 </div>
 <div>
 <select value={speed} onChange={e => setSpeed(Number(e.target.value))}>
 <option value={1200}>Медл.</option>
 </select>
 </div>

```

```

 const size = arr.length;
 const a = Array.from({ length: size }, () => Math.floor(Math.random() * 100));
 setArr(a);
 setCustomInput(a.join(', '));
 };

 const buildSteps = useMemo(() => genBuildSteps(arr), [arr]);
 const queryTDSteps = useMemo(() => genQueryTopDownSteps(arr), [arr]);
 const queryBUSteps = useMemo(() => genQueryBottomUpSteps(arr), [arr]);

 const totalSum = arr.reduce((a, b) => a + b, 0);

 return (
 <div className="bg-slate-900 rounded-2xl border border-indigo-500" >
 {/* ---- TOP BAR: array editor + query range ---- */}
 <div className="mb-6 space-y-4">
 <div className="flex flex-col lg:flex-row gap-4">
 {/* Array input */}
 <form onSubmit={handleApply} className="flex flex-col">
 <div className="flex items-center justify-between">
 <label className="text-[10px] font-bold uppercase">Введите массив чисел</label>
 <button type="button" onClick={randomize} className="text-[10px] font-bold">Случайно</button>
 </div>
 <div className="flex items-center">
 <input type="text" value={customInput} onChange={e => setCustomInput(e.target.value)} className="flex-1 bg-slate-900 border border-indigo-500 focus:border-indigo-500 placeholder="5, 8, 3, 12, 7, 2" />
 <button type="submit" className="bg-indigo-500 text-white px-4 py-2 font-medium">Применить</button>
 </div>
 </form>
 <div className="flex flex-wrap gap-1.5 pt-1">
 {arr.map((v, i) => (

```



```

 Запрос: sum(A[{qL}..{qR}]) = {arr.slice(q
 </div>
 </div>
</div>
</div>

{/* ---- TABS ---- */}
<div className="flex flex-wrap items-center gap-1" >
 <button onClick={() => setView('build')} className="
 -colors ${view === 'build' ? 'border-indigo-500
 -slate-200'}`">
 <Hammer className="w-3.5 h-3.5" /> Построить
 </button>
 <button onClick={() => setView('queries')} class
 on-colors ${view === 'queries' ? 'border-emerg
 er:text-slate-200'}`">
 <Search className="w-3.5 h-3.5" /> Запрос: св
 </button>
 <button onClick={() => setView('theory')} class
 n-colors ${view === 'theory' ? 'border-blue-500
 te-200'}`">
 <Info className="w-3.5 h-3.5" /> Почему / Спра
 </button>
</div>

{/* ---- TAB CONTENT ---- */}
{view === 'build' && (
 <SimPanel
 key={`build-${arr.join('-')}`}
 title="Построение дерева (рекурсия)"
 icon=" "
 steps={buildSteps}
 code={BUILD_CODE}
 color="indigo"
 arr={arr}
 />
)}

```

```

 <div className="text-indigo-300">→ Левый:
 <div className="text-emerald-300">→ Правый:
h - 1) >> 1)} эл.)</div>
 <div className="text-slate-500 mt-2">Всего
 } внутренних.</div>
</div>
</div>

```

```

{ /* Comparison cards */ }
<div className="grid grid-cols-1 xl:grid-cols-2">
 <div className="bg-slate-950 p-5 rounded-xl">
 <div className="absolute -top-3 left-4 bg-emerald-500
e border border-emerald-500 shadow-md">
 ↑ Запрос Сверху-вниз (Рекурсия)
 </div>
 <p className="text-xs text-slate-300 mb-3">
 Начинаем с корня. Если отрезок узла полн
 наче спускаемся в обоих детей.
 </p>
 <div className="space-y-1.5 text-xs">
 <div className="flex justify-between border-b border-slate-700">
 0(
 <div className="flex justify-between border-b border-slate-700">
 0(
 <div className="flex justify-between">✓ легко</div>
 </div>
 </div>
 </div>
 <div className="bg-slate-950 p-5 rounded-xl">
 <div className="absolute -top-3 left-4 bg-amber-500
rder border border-amber-500 shadow-md">
 ↑ Запрос Снизу-вверх (Итерация)
 </div>
 <p className="text-xs text-slate-300 mb-3">
 Стартуем с двух указателей (l и r) на л
 ём его левого соседа. Поднимаемся пока l {"
 </p>
 <div className="space-y-1.5 text-xs">

```



```

 }}
 <span className="font-semibold text
 <ChevronRight
 className={`w-4 h-4 shrink-0 mt-0
 isSelected ? "text-indigo-400"
 }`
 />
 </button>
);
 }}}
</div>
</div>
)})
</div>

<div className="shrink-0 m-3 mt-0 bg-gradient-to-l
] text-slate-300 space-y-1.5 hidden lg:block">
 <div className="font-bold text-indigo-400 flex .
 <Sparkles className="w-3.5 h-3.5" /> Как поль
 </div>
 <p className="leading-relaxed">
 Подчёркнутые термины в тексте раскрываются по
 симулятора.
 </p>
 <p className="text-slate-400 italic">Стрелки ←
 </div>
</aside>
);
};

```

ФАЙЛ 59/83: src/components/Simulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, Ar

```

```

// Heap state
const [heap, setHeap] = useState<Hypothesis[]>([
 { id: '1', text: 'Кандидат: "Привет, я искусственный"', },
 { id: '2', text: 'Кандидат: "Привет, я испек вкусный"', },
 { id: '3', text: 'Кандидат: "Привет, я испанский ин"', },
]);
const [newCustomText, setNewCustomText] = useState('');
const [newCustomProb, setNewCustomProb] = useState(0.1);
const [heapMessage, setHeapMessage] = useState<string>('');

// Stack handlers
const addPlate = () => {
 const presets = [
 { name: 'Сковорода от омлета', icon: '🥞' },
 { name: 'Кастрюля от супа', icon: '🍲' },
 { name: 'Чашка от кофе', icon: '☕' },
 { name: 'Миска с остатками салата', icon: '🥗' },
 { name: 'Тарелка от тако', icon: '🌮' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPlate = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setStack(prev => [...prev, newPlate]);
 setPopMessage(`Положили поверх стопки: ${newPlate.name} 🍽️`);
};

const popPlate = () => {
 if (stack.length === 0) {
 setPopMessage("⚠️ Стопка пуста! Все тарелки вымыты");
 return;
 }
 const topPlate = stack[stack.length - 1];
 setStack(prev => prev.slice(0, prev.length - 1));
 setPopMessage(`Вымыта верхняя тарелка (LIFO): ${topPlate.name} 🍽️`);
};

```

```

const resetQueue = () => {
 setQueue([
 { id: '1', name: 'Студент Вася', icon: ' ' },
 { id: '2', name: 'Голодный кодер', icon: ' ' },
 { id: '3', name: 'Кот Борис', icon: ' ' },
]);
 setDequeueMessage(" Очередь восстановлена.");
};

// Heap handlers
const addHypothesis = (e: React.FormEvent) => {
 e.preventDefault();
 if (!newCustomText.trim()) return;
 const item: Hypothesis = {
 id: Date.now().toString(),
 text: `Кандидат: "${newCustomText}"`,
 probability: Number(newCustomProb)
 };
 setHeap(prev => [...prev, item]);
 setNewCustomText('');
 setHeapMessage(` Добавлена гипотеза с вероятностью`);
};

const popHeap = () => {
 if (heap.length === 0) {
 setHeapMessage("⚠ Куча пуста! Нет кандидатов для");
 return;
 }
 // Find highest probability
 let maxIdx = 0;
 for (let i = 1; i < heap.length; i++) {
 if (heap[i].probability > heap[maxIdx].probability) {
 maxIdx = i;
 }
 }
 const best = heap[maxIdx];
 setHeap(prev => prev.filter((_, idx) => idx !== maxIdx));
 setHeapMessage(` Выбран самый "тяжелый" кандидат:

```

```

 <div className="text-left">
 <div className="font-bold text-sm text-wh
 <div className="text-xs opacity-80">LIFO
 </div>
 </div>
 <span className="text-xs bg-blue-500/20 text-l
 DFS

 </button>

 <button
 onClick={() => setActiveTab('queue')}
 className={`flex items-center justify-between
 activeTab === 'queue'
 ? 'bg-indigo-600/20 border-indigo-500 tex
 : 'bg-slate-800/60 border-slate-700/60 te
 }`}
 >
 <div className="flex items-center gap-3">
 <AlignLeft className={`w-5 h-5 ${activeTab :
 <div className="text-left">
 <div className="font-bold text-sm text-wh
 <div className="text-xs opacity-80">FIFO
 </div>
 </div>
 </div>
 <span className="text-xs bg-indigo-500/20 tex
 BFS

 </button>

 <button
 onClick={() => setActiveTab('heap')}
 className={`flex items-center justify-between
 activeTab === 'heap'
 ? 'bg-emerald-600/20 border-emerald-500 t
 : 'bg-slate-800/60 border-slate-700/60 te
 }`}
 >

```

```

border-blue-500/40 px-4 py-2.5 rounded-xl
>
 Помыть верхнюю
</button>
<button
 onClick={resetStack}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
>
 <RotateCcw className="w-5 h-5" />
</button>
</div>
</div>

{popMessage && {
 <div className="bg-blue-950/60 border borde
imate-fade-in">
 <span className="inline-block w-2 h-2 roun
 {popMessage}
 </div>
}}

{/* Visualization */}
<div className="bg-slate-950/60 p-6 rounded-2
>
 {stack.length === 0 ? {
 <div className="text-center py-12 text-sl
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Стон
 <p className="text-xs mt-1">Добавьте гр
 </div>
) : {
 <div className="w-full max-w-sm flex flex
 <div className="absolute top-0 text-xs
full flex items-center gap-1">
 ВЕРШИНА СТЕКА (TOP)
 <ArrowDown className="w-3.5 h-3.5 ani

```



```

 { /* QUEUE TAB */
 {activeTab === 'queue' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Симуляция очереди за супом
 <span className="text-xs font-mono bg-in
 /span>
 </h3>
 <p className="text-slate-400 text-xs mt-1
 Кто первым пришел, тот первым получил с
 </p>
 </div>
 <div className="flex flex-wrap items-center
 <button
 onClick={addPerson}
 className="flex-1 md:flex-none flex ite
 -2.5 rounded-xl text-sm font-bold shadow-lg
 >
 <Plus className="w-4 h-4" /> Встать в о
 </button>
 <button
 onClick={servePerson}
 className="flex-1 md:flex-none flex ite
 er border-indigo-500/40 px-4 py-2.5 rounded
 >
 Налить суп первому
 </button>
 <button
 onClick={resetQueue}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
 tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>

```

```

 isFirst
 ? 'bg-gradient-to-b from-indigo-500 to-slate-900'
 : 'bg-slate-900/90 border-slate-900'
 }` }
 >
 {isFirst && {
 <span className="absolute -top-10 left-1/2 transform translate-x-50%
se tracking-wider shadow">
 Первый (Head)

 }}
 {

 {person.name}

 {isFirst ? 'Получает сун' : 'Входит'}

 </div>
);
 }}}

 <div className="flex flex-col items-center justify-center gap-6 min-w-[120px] h-[120px]">
 <Plus className="w-6 h-6 mb-1" />

 </div>
 </div>
)}
 <div className="mt-6 text-xs text-slate-500">
 ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • ИЛИ
 </div>
</div>
</div>
})

```

```

<div className="md:col-span-6">
 <label className="block text-xs font-bold">
 <input
 type="text"
 value={newCustomText}
 onChange={e => setNewCustomText(e.target.value)}
 placeholder="например: 'Привет, я лучший!'"
 className="w-full bg-slate-900 border border-emerald-500 transition-colors"
 />
 </div>
<div className="md:col-span-3">
 <label className="block text-xs font-bold">
 Вероятность:
 </label>
 <input
 type="range"
 min="0.01"
 max="0.99"
 step="0.01"
 value={newCustomProb}
 onChange={e => setNewCustomProb(Number(e.target.value))}
 className="w-full accent-emerald-500 cursor-pointer"
 />
</div>
<div className="md:col-span-3">
 <button
 type="submit"
 disabled={!newCustomText.trim()}
 className="w-full flex items-center justify-center gap-2 border border-emerald-500/40 disabled:opacity-50 disabled:cursor-not-allowed"
 >
 <Plus className="w-4 h-4" /> Добавить в список
 </button>
</div>
</form>

```

```

 <span className={`flex items-center
 isTop ? 'bg-emerald-500 text-
 }`}>
 #{index + 1}

 <div>
 <div className={`font-bold te
 {item.text}
 </div>
 {isTop && (
 <span className="text-[10px
-0.5 rounded inline-block mt-1">
 Вершина Кучи (Root) • Буд

)}
 </div>
 </div>

 <div className="flex items-center
 /* Custom progress bar */
 <div className="hidden sm:block
 <div
 className={`h-full rounded-
 style={{ width: `${percent}%
 ></div>
 </div>
 <span className={`font-mono font
 isTop ? 'text-emerald-400 tex
 }`}>
 {percent}%

 </div>
</div>
);
 }}}}
</div>
))
<div className="mt-6 text-xs text-slate-500

```

```

 const all = Object.entries(VIZ_REGISTRY).map(([id,
 id,
 entry,
 chapterTitle: titleById.get(id),
]));
 const q = query.trim().toLowerCase();
 if (!q) return all;
 return all.filter(
 (i) =>
 i.entry.title.toLowerCase().includes(q) ||
 (i.entry.hint ?? "").toLowerCase().includes(q)
 (i.chapterTitle ?? "").toLowerCase().includes(q)
);
 }, [query]);

return (
 <div>
 <div className="mb-6">
 <h2 className="text-xl sm:text-2xl font-extrabold">
 <p className="text-sm text-slate-400 leading-relaxed">
 Все интерактивные демонстрации в одном месте.
 здесь их можно открыть напрямую.
 </p>
 </div>

 <div className="relative mb-6 max-w-md">
 <Search className="w-4 h-4 text-slate-500 absolute">
 <input
 value={query}
 onChange={(e) => setQuery(e.target.value)}
 placeholder="Поиск тренажёра..."
 className="w-full bg-slate-900 border border-
 focus:outline-none focus:border-indigo-500
 />
 {query && (
 <button
 onClick={() => setQuery("")}
 className="absolute right-2 top-1/2 -transl

```

```
 </div>
 </div>
)))
 </div>
</div>
);
};
```

---

ФАЙЛ 61/83: src/components/SplayPlayground.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState } from "react";
import { SplayRotationsViz } from "../SplayRotationsViz";
import SplayWalkViz from "../SplayWalkViz";
import { SplayRotationSandbox } from "../SplayRotationSandbox";

/**
 * Страница splay – три вкладки по нарастанию сложности
 * 1. «Механика» – покадровый разбор Zig / Zig-Zig / Zig-Zag
 * (прицел → поворот, призраки старых позиций, обход)
 * 2. «Большое дерево» – те же случаи внутри настоящего
 * поворота – кадры «прицел → отстёгиваем среднее по
 * 3. «Собери сам» – песочница: подними узел кликами,
 */
type Tab = "cases" | "big" | "play";

export const SplayPlayground: React.FC<{ defaultTab?: Tab }> = ({
 defaultTab,
}) => {
 const [tab, setTab] = useState<Tab>(defaultTab);

 return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border">
 <div className="flex flex-wrap items-center justify-center">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-indigo-600 text-white">


```

```
export default SplayPlayground;
```

---

ФАЙЛ 62/83: src/components/SplayRotationSandbox.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useCallback, useMemo, useState } from "react";
import { CheckCircle2, Dices, Eye, EyeOff, RotateCcw, Undo } from "lucide-react";
```

```
/**
```

```
 * Песочница поворотов.
```

```
 *
```

```
 * Здесь ничего не подсказывается: дано настоящее дерево.
```

```
 * поднять отмеченный узел в корень. Клик по узлу = один шаг.
```

```
 * Порядок поворотов выбираешь сам; правильный ли он пока неясно.
```

```
 * только после того, как узел окажется наверху (или после сброса).
```

```
 */
```

```
export interface TNode {
```

```
 key: number;
```

```
 left: TNode | null;
```

```
 right: TNode | null;
```

```
}
```

```
const leaf = (key: number): TNode => ({ key, left: null, right: null });
```

```
const clone = (t: TNode | null): TNode | null =>
```

```
 t ? { key: t.key, left: clone(t.left), right: clone(t.right) } : null;
```

```
/** Заготовки: линия (zig-zig), змейка (zig-zag) и просека (zig-zag-zig-zag)
```

```
export const PRESETS: { name: string; hintCase: string; target: number; }[] =
```

```
{
```

```
 name: "Линия влево",
```

```
 hintCase: "Zig-Zig",
```

```
 target: 1,
```

```
export const findPath = (root: TNode | null, key: number): TNode[] => {
 const path: TNode[] = [];
 let cur = root;
 while (cur) {
 path.push(cur);
 if (key === cur.key) return path;
 cur = key < cur.key ? cur.left : cur.right;
 }
 return [];
};
```

```
/** Поднять узел key на один уровень (поворот с родителем)
export function rotateUp(root: TNode, key: number): TNode {
 const path = findPath(root, key);
 if (path.length < 2) return root;

 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const gg = path.length >= 3 ? path[path.length - 3] : null;

 if (p.left === x) {
 p.left = x.right;
 x.right = p;
 } else {
 p.right = x.left;
 x.left = p;
 }

 if (!gg) return x;
 if (gg.left === p) gg.left = x;
 else gg.right = x;
 return root;
}
```

```
/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number): TNode {
 const path = findPath(root, key);
```



```

const rows = Math.max(1, Math.max(...nodes.map((n) =>
const colW = 46;
const rowH = 62;
return {
 nodes: nodes.map((n) => ({ ...n, x: n.x * colW + co
 edges,
 width: cols * colW,
 height: rows * rowH + 20,
});
}

```

/\* ----- КОМПОНЕНТ -----

```

export const SplayRotationSandbox: React.FC = () => {
 const [presetIdx, setPresetIdx] = useState(0);
 const preset = PRESETS[presetIdx];

 const [tree, setTree] = useState<TNode>(() => preset.t
 const [history, setHistory] = useState<TNode[]>([]);
 const [moves, setMoves] = useState<{ node: number; co
 const [showAnswer, setShowAnswer] = useState(false);

 const target = preset.target;
 const solved = tree.key === target;

 const reset = useCallback(
 (idx = presetIdx) => {
 setPresetIdx(idx);
 setTree(PRESETS[idx].build());
 setHistory([]);
 setMoves([]);
 setShowAnswer(false);
 },
 [presetIdx]
);

 const { nodes, edges, width, height } = useMemo(() =>
 const path = useMemo(() => new Set(findPath(tree, tar

```

```

 >
 <Dices className="w-3.5 h-3.5" /> Другое дере
 </button>
</div>

```

```

<div className="flex gap-2 mb-3 overflow-x-auto n
 {PRESETS.map((p, i) => {
 <button
 key={p.name}
 onClick={() => reset(i)}
 className={`px-3 py-1.5 rounded-lg text-[11p
 i === presetIdx ? "bg-indigo-600 text-whi
 }`}
 >
 {p.name}
 </button>
)})}
</div>

```

```

<div className="bg-slate-900 rounded-xl border bo
 <svg
 viewBox={`0 0 ${width} ${height}`}
 className="h-auto block mx-auto"
 style={{ minWidth: Math.min(width * 1.5, 420)
 role="img"
 >
 {edges.map(([a, b], i) => {
 const na = nodes.find((n) => n.key === a);
 const nb = nodes.find((n) => n.key === b);
 if (!na || !nb) return null;
 const onPath = path.has(a) && path.has(b);
 return (
 <line
 key={i}
 x1={na.x}
 y1={na.y}
 x2={nb.x}
 y2={nb.y}

```

```

<div className="flex flex-wrap items-center gap-2"
 <button
 onClick={undo}
 disabled={history.length === 0}
 className="flex items-center gap-1.5 px-3 py-1
 -white disabled:opacity-40 text-xs font-bol
 >
 <Undo2 className="w-3.5 h-3.5" /> ОТМЕНИТЬ
 </button>
 <button
 onClick={() => reset()}
 className="flex items-center gap-1.5 px-3 py-1
 ext-xs font-bold transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Заново
 </button>
 <button
 onClick={() => setShowAnswer((s) => !s)}
 className={`flex items-center gap-1.5 px-3 py-1
 showAnswer
 ? "bg-amber-600/20 border-amber-500 text-
 : "bg-slate-800 border-slate-700 text-sla
 }`}
 >
 {showAnswer ? <EyeOff className="w-3.5 h-3.5"
 {showAnswer ? "Скрыть разбор" : "Сдаться / по
 </button>

 <span className="text-[11px] text-slate-500 ml-
 поворотов: {moves.length} • минимум: {minimal

</div>

```

```

{showAnswer && !solved && expected && (
 <div className="mt-3 text-[12px] bg-amber-950/40
 Сейчас splay крутил бы узел{" "}
 {expect
 </div>

```

ФАЙЛ 63/83: src/components/SplayRotationsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useEffect, useMemo, useState } from "react";
import { Gauge, Pause, Play, RotateCcw } from "lucide-react";
```

```
/**
```

```
 * Разбор трёх случаев операции Splay: Zig, Zig-Zig, Zig-Zag.
```

```
 */
```

```
 * Что сделано ради понятности:
```

```
 * * у узлов настоящие числа (20 / 40 / 60), а не абстрактные;
```

```
 * * сразу видно, что дерево остаётся деревом поиска;
```

```
 * * под деревом печатается обход слева направо: он ОДНОЗНАЧЕН;
```

```
 * * это и есть доказательство, что поворот ничего не ломает;
```

```
 * * кадр «прицел» ничего не двигает, только подсвечивает;
```

```
 * * на кадре «результат» остаются призраки – пунктирные линии
```

```
 * * местах и стрелки «откуда → куда»;
```

```
 * * по умолчанию анимация НЕ крутится сама: пользователь
```

```
 * * идёт в своём темпе. Автопрокрутка включается кнопкой
```

```
 */
```

```
type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
```

```
type Pos = Partial<Record<NodeId, [number, number]>>;
```

```
interface Frame {
```

```
 pos: Pos;
```

```
 edges: [NodeId, NodeId][];
```

```
 kind: "start" | "aim" | "result";
```

```
 title: string;
```

```
 text: string;
```

```
 /** Пара, которую крутим на этом шаге: кадр-«прицел». */
```

```
 focus?: [NodeId, NodeId];
```

```
 /** Кто переехал на этом кадре – рисуем призрак и стрелку. */
```

```
 moved?: NodeId[];
```

```
 ms: number;
```

```

 kind: "start",
 title: "Было: 20 висит под корнем 40",
 text: "Нам нужен ключ 20, а он на глубине 1. Деду",
 ms: RESULT_MS,
 },
 {
 ...ZIG_BEFORE,
 kind: "aim",
 title: "Прицел: крутим ребро 40 – 20",
 text: "Ничего пока не двигается. Смотри на подсве",
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [160, 50], A: [80, 130], p: [245, 130],
 edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
 kind: "result",
 title: "Стало: 20 – корень",
 text: "40 стало правым сыном двадцатки. Поддерев",
 ева от 40 – это одно и то же место.",
 moved: ["x", "p", "B"],
 ms: RESULT_MS,
 },
],
};

/* ----- Zig-Zig -----
const ZZ_S0 = {
 pos: { g: [255, 40], D: [325, 112], p: [160, 112], C:
 edges: [["g", "p"], ["g", "D"], ["p", "x"], ["p", "C"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZZ_S1 = {
 pos: { p: [165, 40], x: [85, 112], C: [212, 112], g:
 edges: [["p", "x"], ["p", "g"], ["x", "A"], ["x", "B"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZIG: Case = {

```

```

 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [100, 45], A: [45, 118], p: [188, 118],
 edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
 kind: "result",
 title: "Стало: 20 в корне, бамбук стал кустом",
 text: "Сравни с первым кадром: А и В были на глуб
 дерево, а не только достаёт ключ.",
 moved: ["x", "p", "A", "B"],
 ms: RESULT_MS,
 },
],
};

```

/\* ----- Zig-Zag -----

```

const ZG_S0 = {
 pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:
 edges: [["g", "p"], ["g", "D"], ["p", "A"], ["p", "x"]
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZG_S1 = {
 pos: { g: [258, 40], D: [328, 112], x: [148, 112], p:
 edges: [["g", "x"], ["g", "D"], ["x", "p"], ["x", "C"]
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZIGZAG: Case = {
 key: "zig-zag",
 label: "Zig-Zag",
 when: "x и p — дети с РАЗНЫХ сторон (змейка)",
 rotations: "2 поворота, первым — нижний",
 keys: { x: 40, p: 20, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
 ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
 frames: [
 {
 ...ZG_S0,

```



```

 strokeWidth={hot ? 5 : 2}
 opacity={focus && !hot ? 0.25 : 1}
 style={{ transition: "all 1200ms cubic-bezier(0.4, 0, 0.2, 1)" }}
 />
);
 }
}

```

```

{Object.keys(pos) as NodeId[]}.map((id) => {
 const p = pos[id];
 if (!p) return null;
 const sub = isSubtree(id);
 const c = COLOR[sub ? "sub" : id];
 const r = sub ? 15 : 20;
 const hot = Boolean(focus && focus.includes(id));
 return (
 <g
 key={id}
 style={{
 transform: `translate(${p[0]}px, ${p[1]}px)`,
 transition: MOVE,
 opacity: dim(id) ? 0.28 : 1,
 }}
 >
 {hot && <circle r={r + 7} fill="none" stroke="black" />}
 <circle r={r} fill={c.fill} stroke={c.stroke} />
 <text
 textAnchor="middle"
 dominantBaseline="central"
 fontSize={sub ? 12 : 15}
 fontWeight="700"
 fill={sub ? "#94a3b8" : "#fff"}
 >
 {sub ? id : keys[id]}
 </text>
 {!sub && (
 <text textAnchor="middle" y={-r - 7} font-size={10}
 >{id}</text>
)}
 </g>
);
}

```



```

 onClick={() => setCaseIdx(i)}
 className={`px-3 sm:px-4 py-2 rounded-lg text-
 i === caseIdx ? "bg-indigo-600 text-white" : "bg-white text-indigo-600"
 }`}
 >
 {c.label}
 </button>
)})
 </div>

```

```

<div className="mb-3 text-xs sm:text-[13px] text-slate-400">
 Корр.
 ({active.rotations})
</div>

```

```

{/* шаг + пояснение стоят НАД картинкой: сначала шаг, потом пояснение */}
<div className="mb-3 rounded-xl border border-slate-200 p-4">
 <div className="flex items-center gap-2 mb-1 flex-wrap">
 <span
 className={`text-[10px] font-bold uppercase
 current.kind === "aim"
 ? "bg-amber-500/20 text-amber-300"
 : current.kind === "start"
 ? "bg-slate-700 text-slate-300"
 : "bg-indigo-500/20 text-indigo-300"
 }`}
 >
 {current.kind === "aim" ? "прицел" : current.kind}

 Шаг {idx + 1} из {total}. {current.title}

 </div>
 <p className="text-[13px] text-slate-400 leading-relaxed">
 {current.description}
 </p>
</div>

```

```

<div className="bg-slate-900 rounded-xl border border-slate-800 p-4">
 <Tree frame={current} prev={prevPos} keys={activeKeys}

```

```

 setFrame((f) => (f - 1 + total) % total);
 }}
 disabled={idx === 0}
 className="px-3 py-2 rounded-lg bg-slate-800
 d: hover:text-slate-300 text-xs font-bold tr
>
 ← Назад
</button>
<button
 onClick={() => {
 setPlaying(false);
 setFrame((f) => (f + 1) % total);
 }}
 className="px-4 py-2 rounded-lg bg-indigo-600
 w-indigo-900/40"
>
 {last ? "⏮ Сначала" : "Дальше →"}
</button>
<button
 onClick={() => setPlaying((p) => !p)}
 className="flex items-center gap-1.5 px-3 py-1
 ext-xs font-bold transition-colors"
>
 {playing ? <Pause className="w-3.5 h-3.5" />
 {playing ? "Стоп" : "Авто"}}
</button>
<button
 onClick={() => setSlow((s) => !s)}
 title="Автопрокрутка ещё медленнее"
 className={`flex items-center gap-1.5 px-3 py-1
 slow
 ? "bg-emerald-600/20 border-emerald-500 tr
 : "bg-slate-800 border-slate-700 text-sla
 }`}
>
 <Gauge className="w-3.5 h-3.5" /> {slow ? "0.1
</button>
<button
```

```

*
* Проблема, которую решает: на игрушечных A/B/C-схемах
* «двух зигов подряд», а мгновенный splay в большом де
* стало». Здесь каждый ОДИНОЧНЫЙ поворот – атомарный ш
* поворот → текст, какие рёбра развернулись и какие по
* Порядок поворотов виден явно: Zig-Zig – ВЕРХНЕЕ ребр
* НИЖНЕЕ первым.
*
* Раскладка – слоты in-order: ключи не двигаются по гор
* поворот меняет только глубину. Поэтому «что переехал
*
* Чистые функции (splaySteps, rotateUp, collectEdges) :
*/

export interface SN {
 key: number;
 left: SN | null;
 right: SN | null;
 parent: SN | null;
}

export function cloneWithParents(t: SN | null): SN | null {
 if (!t) return null;
 const n: SN = { key: t.key, left: null, right: null, parent: null };
 const l = t.left ? cloneWithParents(t.left) : null;
 const r = t.right ? cloneWithParents(t.right) : null;
 if (l) {
 n.left = l;
 l.parent = n;
 }
 if (r) {
 n.right = r;
 r.parent = n;
 }
 return n;
}

export function bstInsert(root: SN | null, key: number) {

```

```

 if (g.left === p) g.left = child;
 else g.right = child;
}

/** Рёбра в направленном виде "p→c": для diff между шагами
export function collectEdges(t: SN | null): string[] {
 const es: string[] = [];
 const walk = (n: SN | null) => {
 if (!n) return;
 if (n.left) es.push(`${n.key}→${n.left.key}`);
 if (n.right) es.push(`${n.key}→${n.right.key}`);
 walk(n.left);
 walk(n.right);
 };
 walk(t);
 return es.sort();
}

const diffEdges = (before: string[], after: string[]):
 const rm = before.filter((e) => !after.includes(e));
 const ad = after.filter((e) => !before.includes(e));
 const parts: string[] = [];
 for (const a of ad) {
 const [p, c] = a.split("→");
 if (rm.includes(`${c}→${p}`)) parts.push(`ребро ${p}→${c}`);
 else parts.push(`поддерево ${c} переехало к ${p}`);
 }
 return parts.join("; ");
};

export type SplayStepKind = "search" | "aim" | "cut" | "rebalance";
export interface SplayStep {
 kind: SplayStepKind;
 root: SN | null;
 aimEdge?: [number, number];
 /** Шаг "cut": ребро, по которому отстёгивается средний узел
 cutEdge?: [number, number];
 cutLabel?: string;
}

```

```

 ? `Спуск по ключу ${key}: путь ${pathKeys.join(
 : `Ключа ${key} нет. Спуск уперся в ${last.key}
 rotCount: 0,
 });

// --- подъём ---
let rotCount = 0;
let target: SN = x as SN;
const fixRoot = () => {
 while (root && root.parent) root = root.parent;
};
/** Один поворот = два атомарных кадра: cut (что пере
const emitRotation = (child: SN, label: string) => {
 const p = child.parent!;
 const mid = p.left === child ? child.right : child.
 steps.push({
 kind: "cut",
 root: cloneWithParents(root)!,
 cutEdge: [child.key, mid ? mid.key : child.key],
 cutLabel: mid ? `поддерево ${mid.key}` : "пустое
 caseLabel: label,
 note: mid
 ? `Отстёгиваем среднее поддерево ${mid.key}: он
 сторону к ${p.key}.`
 : `Среднего поддерева нет: ${child.key} поднима
 rotCount,
 });
const before = collectEdges(root);
rotateUp(child);
fixRoot();
rotCount += 1;
const diff = diffEdges(before, collectEdges(root));
steps.push({
 kind: "rot",
 root: cloneWithParents(root)!,
 caseLabel: label,
 note: mid
 ? `Поворот: ${child.key} встал над ${p.key}. Сп

```

```

 caseLabel: "Zig-Zig",
 note: `Теперь обычный зиг: ребро ${target.parent.key}
 rotCount,
 });
 emitRotation(target, "Zig-Zig");
} else {
 const side = `${g.key} → ${pIsLeft ? "влево" : "вправо"}
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge: [p.key, target.key],
 caseLabel: "Zig-Zag",
 note: `Zig-Zag (змейка: ${side}). Порядок ОБРАТНЫЙ
 rotCount,
 });
 emitRotation(target, "Zig-Zag");
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge: [target.parent.key, target.key],
 caseLabel: "Zig-Zag",
 note: `Второе ребро: ${target.parent.key}—${target.key}
 rotCount,
 });
 emitRotation(target, "Zig-Zag");
}
}

steps.push({
 kind: "done",
 root: cloneWithParents(root)!,
 note: ` ${target.key} в корне за ${rotCount} ${rotCount}
 rotCount,
});
return { steps, found };
}

/* ----- служебное -----

```

```

const SLOT_W = 88;
const LEVEL_H = 76;

const layout = (root: SN | null) => {
 const pos = new Map<number, { px: number; py: number }>();
 let slot = 0;
 const walk = (n: SN | null, d: number) => {
 if (!n) return;
 walk(n.left, d + 1);
 pos.set(n.key, { px: 56 + slot * SLOT_W, py: 40 + d * LEVEL_H });
 slot += 1;
 walk(n.right, d + 1);
 };
 walk(root, 0);
 return pos;
};

/** Врезка «механика одного поворота»: как вправляют выключатель
const MechanismInset: React.FC<{ step: SplayStep | null }> = ({ step }) => {
 const isCut = step?.kind === "cut";
 const isRot = step?.kind === "rot";
 return (
 <div className="bg-slate-950/80 rounded-lg border border-slate-700 p-4">
 <h4 className="text-xs font-bold text-slate-300 mb-2">Механика одного поворота
 <p className="text-[10px] text-slate-500 mb-2">Каждый поворот выключателя требует
 <svg viewBox="0 0 340 190" className="w-full">
 <g transform="translate(10,6)">
 <text y="10" fill="#94a3b8" fontSize="11" font-weight="bold">1
 <line x1="55" y1="46" x2="30" y2="86" stroke="black" stroke-width="2"/>
 <line x1="55" y1="46" x2="80" y2="86" stroke="black" stroke-width="2"/>
 <line x1="30" y1="86" x2="12" y2="126" stroke="black" stroke-width="2"/>
 <line x1="30" y1="86" x2="48" y2="126" stroke="black" stroke-width="2"/>
 <circle cx="55" cy="46" r="15" fill="#1e293b" stroke="black" stroke-width="2"/>
 <text x="55" y="50" textAnchor="middle" fill="white">1
 <circle cx="30" cy="86" r="15" fill="#78350f" stroke="black" stroke-width="2"/>
 <text x="30" y="90" textAnchor="middle" fill="white">2
 <circle cx="12" cy="126" r="13" fill="#0f172a" stroke="black" stroke-width="2"/>
 <text x="12" y="130" textAnchor="middle" fill="white">3
 </g>
 </svg>
 </div>
);
};

```

```

const SplayWalkViz: React.FC = () => {
 const initial = useMemo(buildInitial, []);
 const [steps, setSteps] = useState<SplayStep[] | null>();
 const [idx, setIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [input, setInput] = useState("");
 const [treeVers, setTreeVers] = useState(0);
 const baseRef = useRef<SN>(initial);
 const timer = useRef<ReturnType<typeof setTimeout> | null>();

 const runKey = (key: number, isInsert = false) => {
 const { steps: st } = splaySteps(baseRef.current, key);
 setSteps(st);
 setIdx(0);
 setPlaying(true);
 };

 useEffect(() => {
 if (!playing || !steps) return;
 if (idx >= steps.length - 1) {
 setPlaying(false);
 // фиксируем результат в базовом дереве
 baseRef.current = steps[steps.length - 1].root ?? null;
 setTreeVers((v) => v + 1);
 return;
 }
 timer.current = setTimeout(() => setIdx((v) => v + 1), 1000);
 return () => {
 if (timer.current) clearTimeout(timer.current);
 };
 }, [playing, idx, steps]);

 const step = steps ? steps[Math.min(idx, steps.length - 1)] : null;
 const viewRoot = step ? step.root : baseRef.current;
 const pos = useMemo(() => layout(viewRoot), [viewRoot]);
 const nodes = useMemo(() => [...pos.entries()].map(([id, node]) => node), [pos]);
 const edges = useMemo(() => {

```



```

 setInput("");
 };

const stepColor = (k: number): string => {
 if (!step) return "#0f172a";
 if (step.kind === "search" && step.pathKeys?.includes(k)) return "#0f172a";
 if (step.aimEdge && step.aimEdge.includes(k)) return "#0f172a";
 return "#0f172a";
};

const strokeColor = (k: number): string => {
 if (!step) return "#64748b";
 if (step.kind !== "search" && step.pathKeys?.includes(k)) return "#64748b";
 if (step.aimEdge?.includes(k)) return "#f59e0b";
 if (step.pathKeys?.includes(k)) return "#38bdf8";
 return "#64748b";
};

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border" >
 {/* Wanka */}
 <div className="flex flex-wrap items-center justify-center" >
 <div className="flex items-center gap-3" >
 <div className="p-2.5 bg-indigo-600 text-white" >

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white" >
 <p className="text-sm text-indigo-300" >Клики
 </p>
 </div>
 </div>
 <div className="flex flex-wrap items-center gap-3" >
 <input
 value={input}
 onChange={(e) => setInput(e.target.value)}
 onKeyDown={(e) => {
 if (e.key === "Enter") {
 const v = parseInt(input, 10);
 if (!Number.isNaN(v)) {

```

```

 <button onClick={() => { setIdx(0); setPlaying(
 ld bg-slate-700 text-white disabled:opacity-0.5
 В начало
 </button>
 <button onClick={() => { setPlaying(false); setPlaying(
 -lg text-xs font-bold bg-slate-700 text-white
 ← Шаг
 </button>
 <button onClick={() => setPlaying(!playing)} className="px-3 py-1.5 rounded-lg text-xs font-medium
 er-600" : "bg-emerald-600"}`}>
 {playing ? " Пауза" : "▶ Пуск"}
 </button>
 <button onClick={() => { setPlaying(false); setPlaying(
 className="px-3 py-1.5 rounded-lg text-xs font-medium
 Шаг →
 </button>

 Шаг {idx + 1} из {steps.length} • поворотов

 {step?.caseLabel && (

 00/60">
 {step.caseLabel}

)}
 </div>
)}

 {/* Дерево */}
 <div className="bg-indigo-950/20 rounded-xl p-4 border border-indigo-950/50">
 {!steps && (
 <div className="absolute top-3 left-3 bg-indigo-950/50 text-white font-bold">
 Кликни по любому узлу – splay
 </div>
)}
 <svg viewBox={`0 0 ${autoW} ${autoH}`} className="w-full h-full">
 {edges.map((e) => {

```

```

 }}
 className="cursor-pointer"
 >
 <circle
 r={isAimNode ? 24 : 20}
 fill={stepColor(n.key)}
 stroke={strokeColor(n.key)}
 strokeWidth={isAimNode ? 4 : 2}
 className="transition-all duration-300"
 />
 <text y={4} fill="#ffffff" fontSize="13"
 >
 {n.key}
 </text>
</g>
);
}}
</svg>

```

```

{/* Лог шага + механика */}
<div className="w-full flex flex-col lg:flex-row" >
 <div className="flex-1 bg-slate-950/80 p-4 rounded" >
 <p className="font-semibold text-amber-400 mb-2" >
 {step?.kind === "aim" ? " Прицел:" : step?.kind === "search" ? " Спуск:" : "Ставка:"}
 </p>
 <p className="min-h-[40px] flex items-center" >
 {step?.kind === "aim" ? " Прицел на ребро, поворот, переехал" : " Спуск"}
 {steps && step && step.kind !== "search" && (
 <p className={`text-xs mt-2 ${inorderKeys(step.root).join(",")} === inorderKeys(step.root).join(",")} >
 {inorderKeys(step.root).join(",")} === inorderKeys(step.root).join(",")}
 сортировка) не изменился • узлов {inorderKeys(step.root).join(",")}
 </p>
)}
 </div>
 <div className="w-full lg:w-[300px] shrink-0" >
 <MechanismInset step={step} />
 </div>
</div>

```

```

 {step?.caseLabel === "Zig-Zig" && step.
7 : step?.caseLabel === "Zig-Zig" && l.line
 {step?.caseLabel === "Zig-Zag" && step.
10 : step?.caseLabel === "Zig-Zag" && l.li
 return (
 <div key={l.line} className={`py-1 px-2
ber-400" : "text-slate-400"}`}>
 <span className="text-slate-600 selec
 {l.text}
 </div>
 </div>
);
 }
 </div>
 </div>
</div>
);
};

export default SplayWalkViz;

```

ФАЙЛ 65/83: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';

```

```

interface Plate {
 id: string;
 name: string;
 emoji: string;
 color: string;
 isDirty: boolean;
 animState: 'in' | 'idle' | 'washing' | 'clean';
}

```

```

 }

 const preset = PLATE_PRESETS[counter % PLATE_PRESETS.length];
 counter++;

 const newPlate: Plate = {
 id: Date.now().toString(),
 ...preset,
 isDirty: true,
 animState: 'in',
 };

 setDirtyStack(prev => [...prev, newPlate]);
 addLog(`  PUSH: Положили ${newPlate.emoji} сверху с ${newPlate.id}`);

 setTimeout(() => {
 setDirtyStack(prev => prev.map(p => p.id !== newPlate.id));
 }, 500);
 }, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length === 0) {
 setShake(true);
 addLog('⚠ POP: Нечего мыть! Все тарелки чистые.');
 setTimeout(() => setShake(false), 400);
 return;
 }

 setIsWashing(true);
 const topPlate = dirtyStack[dirtyStack.length - 1];

 addLog(`  POP: Начинаем мыть верхнюю тарелку с ${topPlate.emoji}`);

 // Step 1: Start washing animation (sponge and soap)
 setDirtyStack(prev => prev.map(p => p.id !== topPlate.id));
 setShowSoap(true);

```

```

<div className="bg-blue-950/40 border border-blue-950"
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-blue-400 text-2xl">
 Ты складываешь грязные тарелки друг на друга.
 А когда приходит время мыть, ты берёшь именно эту.
 Попробуй вытащить нижнюю — всё рухнет!
 </h3>
 </div>
</div>

```

```

{/* УПРАВЛЕНИЕ */}
<div className="flex items-center gap-3 flex-wrap"
 <button
 onClick={pushPlate}
 disabled={isWashing}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/95 disabled:cursor-not-allowed text-white transition-all cursor-pointer flex items-center justify-center"
 >
 Положить грязную (PUSH)
 </button>

 <button
 onClick={popPlate}
 disabled={isWashing || dirtyStack.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/95 disabled:cursor-not-allowed text-white transition-all cursor-pointer flex items-center justify-center"
 >
 Помыть верхнюю (POP)
 </button>

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white cursor-pointer border border-slate-700"
 >
 Сбросить (RESET)
 </button>
</div>

```

```

 <p className="text-white font-black text-1">
 <p className="text-slate-500 text-xs mt-1">
 </div>
) : (
 <div className={`flex-1 flex flex-col justify-between`>

 {/* Указатель TOP */}
 <div
 className="absolute right-8 flex items-center"
 style={{ bottom: `${24 + topIndex * 44}px` }}
 >

er gap-1">
 <ArrowUp className="w-3.5 h-3.5" />
 ВЕРШИНА (TOP)

 </div>

 {/* Тарелки (отрисовываем в обратном порядке) */}
 {dirtyStack.slice().reverse().map((plate, idx) => {
 const isTop = idx === topIndex;
 return (
 <div
 key={plate.id}
 className={`
 w-full max-w-[280px] h-10 rounded-
shadow-md select-none transition-all duration-300
 ${plate.color}
 ${plate.animState === 'in' ? 'animate-in' : ''}
 ${plate.animState === 'washing' ? 'animate-washing' : ''}
 ${isTop ? 'ring-4 ring-blue-500/50' : ''}
 `}
 >
 {plate.name}

 {isTop && (

```

```

 className="w-full h-8 rounded-full flex items-center justify-center gap-2 opacity-50"
 >
 {plateName}
 {plateType}
 </div>
)))
 </div>
})
</div>

{/* Столешница сушилки */}
<div className="w-full h-4 bg-gradient-to-r from-slate-900 to-slate-800" />
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-800 p-4" >
 <div className="text-[10px] font-mono text-slate-400" >
 {log.map((entry, idx) => (
 <div
 key={idx}
 className={`text-xs font-mono flex items-center justify-between gap-2`}
 >
 {idx === 0 ? Действие:
 {entry}
 </div>
))}
 </div>
</div>
);
};

```



```

 steps.push({ i, j, pi: [...pi], found: true
 }
}
return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string)
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const z = new Array(n).fill(0);
 const steps: any[] = [];

 let l = 0, r = 0;
 steps.push({ i: 0, l, r, z: [...z], desc: `Начало.` });

 for (let i = 1; i < n; i++) {
 steps.push({ i, l, r, z: [...z], desc: `Шаг i=$
 if (i <= r) {
 steps.push({ i, l, r, z: [...z], desc: `Вни
 !` });
 z[i] = Math.min(r - i + 1, z[i - l]);
 steps.push({ i, l, r, z: [...z], desc: `Пре
 }. Копипаста сработала.` });
 }

 steps.push({ i, l, r, zTarget: z[i], zSource: i
 while (i + z[i] < n && s[z[i]] === s[i + z[i]])
 steps.push({ i, l, r, zTarget: z[i], zSource
 падают! Окно растёт.` });
 z[i]++;
 }

 if (i + z[i] < n) {
 steps.push({ i, l, r, zTarget: z[i], zSource
 [i]]}' разные. Стоп.` });
 } else {

```

```

 setStepIdx(0);
 setAutoPlay(false);
 }, [s, mode]);

// Timer loop for autoPlay
useEffect(() => {
 if (autoPlay) {
 timer.current = setInterval(() => {
 setStepIdx(prev => {
 if (prev >= steps.length - 1) { set
 return prev + 1;
 });
 }, 1200);
 }
 return () => { if (timer.current) clearInterval
}, [autoPlay, steps.length]);

const playPause = () => setAutoPlay(!autoPlay);
const nextStep = () => { setAutoPlay(false); setSte
const prevStep = () => { setAutoPlay(false); setSte
const reset = () => { setAutoPlay(false); setStepId

const step = steps[stepIdx] || steps[0];

return (
 <div className="space-y-4">
 <div className="flex items-center justify-b
 <div className="flex bg-slate-900 borde
 <button onClick={() => setMode("kmp
 className={`px-3 py-1.5 text-xs
e" : "text-slate-400 hover:text-slate-300"}
 КМП (π-ФУНКЦИЯ)
 </button>
 <button onClick={() => setMode("z")
 className={`px-3 py-1.5 text-xs
" : "text-slate-400 hover:text-slate-300"}`
 Z-ФУНКЦИЯ
 </button>

```

```

 if (index >= step.l && index <= step.r) {
 bgColor = "bg-emerald-900";
 borderColor = "border-emerald-900";
 }
 if (step.zTarget === index) {
 borderColor = "border-amber-400";
 bgColor = step.match === "L" ? "bg-rose-500" : "bg-slate-700";
 }
 }
 return (
 <div key={index} className={
 /* L/R markers for Z */
 {mode === "z" && step.l <= index && index <= step.r
 ? "absolute left-0 right-0 top-0 bottom-0 border border-amber-400"
 : ""}
 >
 {mode === "z" && step.l <= index && index <= step.r
 ? <div className="absolute left-0 right-0 top-0 bottom-0 border border-amber-400" />
 : ""}

 {/* Index */}
 <span className={`text-${mode === "z" ? "rose-500" : "slate-700"} font-mono text-lg transition-colors`}
 >{index}

 {/* Char Box */}
 <div className={`w-8 h-8 flex items-center justify-center text-center text-sm font-mono text-lg transition-colors`}
 >{char}</div>

 {/* Value array box */}
 <div className="text-[10px] font-mono font-bold">
 {mode === "kmp" ? step.kmp[index] : ""}
 </div>

 {/* KMP fingers */}
 {mode === "kmp" && step.kmp[index] > 0
 ? <div className="text-[10px] font-mono font-bold">{step.kmp[index]}</div>
 : ""}
 </div>
);
}

```

```

 ulse z-0 bg-emerald-500/20 shadow-[0_0_15px_0px]
 }}
 {step.found && mode === 'step' ? (
 <div className="absolute top-0 right-0 z-0 bg-emerald-500/20 shadow-[0_0_15px_0px]
 rounded-lg disabled:opacity-30 disabled:hover:opacity-100
 <Undo strokeWidth={3} size={16} />
 </div>
) : (
 <div className="absolute top-0 right-0 z-0 bg-emerald-500/20 shadow-[0_0_15px_0px]
 rounded-lg disabled:opacity-30 disabled:hover:opacity-100
 <PlayPause className="text-white" />
 </div>
)
 </div>
 </div>
 </div>

```

```

<div className="flex flex-col sm:flex-row gap-4" style={{width: '100%'}}>
 <div className="flex bg-slate-900 border-1 border-slate-800 rounded-lg p-4" style={{width: '100%'}}>
 <button onClick={prevStep} disabled={step === 0} className="text-white rounded-lg disabled:opacity-30 disabled:hover:opacity-100" style={{width: '50%'}}>
 <Undo strokeWidth={3} size={16} />
 </button>
 <button onClick={playPause} className="text-white rounded-lg disabled:opacity-30 disabled:hover:opacity-100" style={{width: '50%'}}>
 {autoplay ? <Pause size={16} /> : <Play size={16} />}
 </button>
 <button onClick={nextStep} disabled={step === steps.length - 1} className="text-white rounded-lg disabled:opacity-30 disabled:hover:opacity-100" style={{width: '50%'}}>
 <SkipForward strokeWidth={3} size={16} />
 </button>
 <button onClick={reset} className="text-white rounded-lg disabled:opacity-30 disabled:hover:opacity-100" style={{width: '50%'}}>
 <RefreshCw strokeWidth={3} size={16} />
 </button>
 </div>

```

```

 <div className="flex flex-col gap-4" style={{width: '100%'}}>
 <div className="flex-1 bg-slate-900/50 border-1 border-slate-800 rounded-lg p-4" style={{width: '100%'}}>
 <div className="absolute top-0 right-0 z-0 bg-emerald-500/20 shadow-[0_0_15px_0px]
 rounded-lg disabled:opacity-30 disabled:hover:opacity-100

 </div>
 </div>
 </div>

```

```
while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
if (i + z[i] - 1 > r) {
 l = i;
 r = i + z[i] - 1;
}
}
}

</pre>
</div>
</div>
);
}
```

---

ФАЙЛ 67/83: src/components/TopologicalSortViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useEffect } from "react";
import {
 Play,
 Pause,
 RotateCcw,
 Clock,
} from "lucide-react";

interface TNode {
 id: number;
 x: number;
 y: number;
 label: string;
 name: string;
}

interface TEdge {
 u: number;
 v: number;
}
```

```

const recordStep = (desc: string, currentNode: number) => {
 listSteps.push({
 desc,
 visited: new Set(visited),
 fullyProcessed: new Set(fullyProcessed),
 currentNode,
 topoList: [...topoList],
 dfsStack: [...dfsStack],
 });
};

recordStep(
 "Начало топологической сортировки. Используем кла",
 null,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
dagEdges.forEach((e) => adj[e.u].push(e.v));

const dfs = (u: number) => {
 visited.add(u);
 dfsStack.push(u);
 recordStep(
 `DFS: зашли в вершину ${u} ("${dagNodes[u].name}
 u,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs(to);
 } else if (!fullyProcessed.has(to)) {
 // Explaining potential cycle detected (not in
 recordStep(
 `⚠ Внимание: Рёбра в серую вершину ${to} о
 u,
);
 }
 }
};

```

```

 });

 useEffect(() => {
 let timer: NodeJS.Timeout;
 if (isPlaying && currentStepIdx < steps.length - 1) {
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1600);
 } else {
 setIsPlaying(false);
 }
 return () => clearInterval(timer);
 }, [isPlaying, currentStepIdx, steps.length]);

 const togglePlay = () => setIsPlaying(!isPlaying);
 const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
 };

 const getTopoListString = () => {
 return [...step.topoList]
 .reverse()
 .map((id) => dagNodes[id].name)
 .join(" → ");
 };

 return (
 <div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Clock className="w-5 h-5 text-indigo-400" />
 Топологическая сортировка (Зависимости обучен
 </h3>

 <div className="flex items-center gap-2 bg-slate
 <button
 onClick={togglePlay}

```

```

 id="dag-arrow-active"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#8
 </marker>
</defs>

{/* Edges */}
{dagEdges.map((edge, idx) => {
 const u = dagNodes.find((n) => n.id === e
 const v = dagNodes.find((n) => n.id === e

 const isActive =
 (step.currentNode === edge.u || step.cu
 step.visited.has(edge.v));

 return (
 <line
 key={`edge-${idx}`}
 x1={u.x}
 y1={u.y}
 x2={v.x}
 y2={v.y}
 stroke={isActive ? "#818cf8" : "#3341
 strokeWidth={isActive ? "3" : "2"}
 markerEnd={`url(#${isActive ? "dag-ar
 className="transition-all duration-300
 />
);
}})

{/* Nodes */}
{dagNodes.map((node) => {

```



```

 y={node.y - 4}
 textAnchor="middle"
 fill="white"
 fontSize="11px"
 fontWeight="bold"
 className="pointer-events-none"
 >
 ({node.label}) {node.name.split(" "
</text>
<text
 x={node.x}
 y={node.y + 12}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="9px"
 className="pointer-events-none"
 >
 {node.name.split(" ").slice(1).join
</text>
</g>
);
 }}}
</svg>
</div>

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5
 00 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-400">
 Статус сортировки
 </h4>

 <div className="bg-slate-900 border border-slate-800">
 <p className="text-xs text-indigo-300 min-h-[100px]">
 {step.desc}
 </p>
 </div>
</div>

```

```

 <div className="mt-4 pt-3 border-t border-slate-200">

 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-700 text-white"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx >= steps.length - 1}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-700 text-white"
 >
 Вперед
 </button>
 </div>
 </div>
 </div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 68/83: src/components/TreapBuildViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useEffect, useMemo, useState } from "react";
```

```
/**
```

```

 { key: 1, grade: -4 },
];

 /** Порядок ввода (как в черновике) – намеренно НЕ отсортирован
 export const INPUT_ORDER: Pair[] = [
 { key: 3, grade: 5 },
 { key: 45, grade: 8 },
 { key: 3, grade: 6 },
 { key: 1, grade: -4 },
 { key: 7, grade: 2 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 9, grade: 3 },
 { key: 12, grade: 0 },
];
 // (3,5) дубль ключа/приоритета – уберём: ввод без повт
 export const INPUT_CLEAN: Pair[] = INPUT_ORDER.filter((p) => {

 const sortedByY = (ps: Pair[]): Pair[] => [...ps].sort((a, b) => {

 /* ----- чистые алгоритмы (экспортируются для тестов) -----

 export function insert(root: TNode | null, p: Pair): TNode | null {
 if (!root) return { id: pairId(p.key, p.grade), key: p.key, grade: p.grade };
 if (p.key <= root.key) root.left = insert(root.left, p);
 else root.right = insert(root.right, p);
 return root;
 }

 export function buildTree(ps: Pair[]): TNode | null {
 let root: TNode | null = null;
 for (const p of sortedByY(ps)) root = insert(root, p);
 return root;
 }

 export interface SplitResult {
 l: TNode | null;
 r: TNode | null;
 }

```

```

 a.right = go(a.right, b);
 return a;
 }
 trace.push({ aId: a.id, bId: b.id, chosenId: b.id,
 `${a.key}, ${a.grade}` с его левым поддеревом.`
 });
 b.left = go(a, b.left);
 return b;
};
return { root: go(a, b), trace };
}

```

```

export function eraseNode(root: TNode | null, key: number): TNode | null {
 const trace: { atId: NodeId | null; note: string }[] = [];
 const go = (t: TNode | null): TNode | null => {
 if (!t) return null;
 if (key < t.key) {
 trace.push({ atId: t.id, note: `${key} < ${t.key}` });
 t.left = go(t.left);
 return t;
 }
 if (key > t.key) {
 trace.push({ atId: t.id, note: `${key} > ${t.key}` });
 t.right = go(t.right);
 return t;
 }
 trace.push({ atId: t.id, note: `Нашли (${t.key}), ${t.grade}` });
 const m = mergeTree(t.left, t.right);
 trace.push(...m.trace.map((s) => ({ atId: s.chosenId, note: s.note })));
 return m.root;
 };
 return { root: go(root), trace };
}

```

```

const cloneTree = (t: TNode | null): TNode | null =>
 t ? { id: t.id, key: t.key, grade: t.grade, left: cloneTree(t.left), right: cloneTree(t.right) } : null;

```

```

/* ----- валидация treap (для чек-листа и стресс-теста) -----

```

```

 if (n.left) { es.push({ from: n.id, to: n.left.id })
 if (n.right) { es.push({ from: n.id, to: n.right.id })
 };
 walk(root);
 return es;
};

```

```

interface TreeViewProps {
 root: TNode | null;
 width?: number;
 height?: number;
 nodeColor?: (id: NodeId) => string;
 nodeStroke?: (id: NodeId) => string;
 edgeColor?: (from: NodeId, to: NodeId) => string;
 edgeDash?: (from: NodeId, to: NodeId) => boolean;
 pulseId?: NodeId | null;
 subtitle?: string;
}

```

```

const TreeView: React.FC<TreeViewProps> = ({ root, width, height, ...props }) => {
 const pos = useMemo(() => layoutTree(root), [root]);
 const nodes = useMemo(() => [...pos.entries()].map(([id, node]) => node), [pos]);
 const edges = useMemo(() => treeEdges(root), [root]);
 const autoW = Math.max(420, ...nodes.map((n) => n.px));
 const autoH = Math.max(260, ...nodes.map((n) => n.py));
 const w = width ?? autoW;
 const h = height ?? autoH;
 const byId = (id: NodeId) => nodes.find((n) => n.id === id);
 return (
 <div>
 <svg viewBox={`0 0 ${w} ${h}`} className="w-full h-full">
 {edges.map((e) => {
 const a = byId(e.from);
 const b = byId(e.to);
 if (!a || !b) return null;
 return (
 <line
 key={`te-${e.from}-${e.to}`}

```

```

interface PlaneProps {
 points: Pair[];
 drawnEdges: { a: Pair; b: Pair }[];
 pending: Pair | null;
 interactive?: boolean;
 selected?: Pair | null;
 cursor?: { x: number; y: number } | null;
 badEdge?: { a: Pair; b: Pair } | null;
 onPointDown?: (p: Pair) => void;
 onPointUp?: (p: Pair) => void;
 onMove?: (e: React.PointerEvent<SVGSVGElement>) => void;
}

const Plane: React.FC<PlaneProps> = ({ points, drawnEdges, ...props }) => {
 const byPair = (p: Pair) => ({ x: PX(p.key), y: PY(p.key) });
 return (
 <svg
 viewBox={`0 0 ${PL_W} ${PL_H}`}
 className={`w-full ${interactive ? "touch-none select-none" : ""}`}
 onPointerMove={onMove}
 style={interactive ? { cursor: "crosshair" } : undefined}
 >
 <defs>
 <pattern id="pl-grid" width="14" height="14" patternUnits="userSpaceOnUse">
 <path d="M 14 0 L 0 0 0 14" fill="none" stroke="black" stroke-width="1px"/>
 </pattern>
 </defs>
 <rect x="34" y="16" width={PL_W - 46} height={PL_H - 16} fill="white" stroke="black" stroke-width="1px"/>
 <line x1="34" y1={PY(0)} x2={PL_W - 12} y2={PY(0)} stroke="black" stroke-width="1px"/>
 <line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y2={PL_H - 16} stroke="black" stroke-width="1px"/>
 {[0, 10, 20, 30, 40].map((k) => {
 <text key={`tx-${k}`} x={PX(k)} y={PY(0) + 16} font-family="monospace" font-size="10" fill="black"/>
 })}
 {[{-4, 0, 4, 8}.map((g) => {
 <text key={`ty-${g}`} x={PX(1) - 14} y={PY(g) + 16} font-family="monospace" font-size="10" fill="black"/>
 })}
 <text x={PL_W - 14} y={PY(0) - 8} fill="#94a3b8" font-family="monospace" font-size="10">0</text>
 <text x={PX(1) - 26} y="14" fill="#94a3b8" font-family="monospace" font-size="10">1</text>
 </svg>
);
};

```

```

 {pending && (
 <g className="transition-all duration-500">
 <circle cx={PX(pending.key)} cy={PY(pending.g
 >
 <text x={PX(pending.key)} y={PY(pending.grade
 ly="monospace">{pending.key}</text>
 </g>
)}
 </svg>
);
};

```

/\* ===== ИГРА: соединение мышкой =====

export type GEdge = { a: Pair; b: Pair }; // a – родители

```

export const CANON_GAME: GEdge[] = (() => {
 const es: GEdge[] = [];
 const walk = (n: TNode | null) => {
 if (!n) return;
 if (n.left) { es.push({ a: { key: n.key, grade: n.g
 if (n.right) { es.push({ a: { key: n.key, grade: n.g
 };
 walk(buildTree(POINTS));
 return es;
})();

```

```

const sameP = (p: Pair, q: Pair) => p.key === q.key && p
const gpid = (p: Pair) => `${p.key}:${p.grade}`;

```

```

/** Цепочка родителей от точки вверх. */
const parentOf = (edges: GEdge[], id: string): GEdge |
 edges.find((e) => gpid(e.b) === id);
const reaches = (edges: GEdge[], from: Pair, target: Pa
 let cur: Pair | undefined = from;
 const seen = new Set<string>();
 while (cur) {

```

```

export function tryConnect(edges: GEdge[], a: Pair, b: Pair): boolean {
 if (sameP(a, b)) return { ok: false, reason: "Это одна и та же точка" };
 if (a.grade === b.grade)
 return { ok: false, reason: "«!» равны — «кто выше?»" };
 const parent = a.grade > b.grade ? a : b;
 const child = a.grade > b.grade ? b : a;
 if (parentOf(edges, gpId(child)))
 return { ok: false, reason: `У точки ${child.key} нет родителя по этой
 линии.` };
 if (reaches(edges, parent, child))
 return { ok: false, reason: "Цикл: линия замкнёт саму себя" };
 const next = [...edges, { a: parent, b: child }];
 if (!forestIsBst(next, POINTS))
 return { ok: false, reason: "Линия вниз по y допустима только в начале" };
 return { ok: true };
}

```

/\* ===== РЕЖИМ BUILD ===== \*/

```

type BuildPhase = "sort" | "connect" | "game";
type SortAlgo = "none" | "quicksort" | "counting";
type ConnectOrder = "y-desc" | "left-right";

const parsePairs = (raw: string): { pairs: Pair[]; errors: string[] } => {
 const pairs: Pair[] = [];
 const errors: string[] = [];
 const seen = new Set<string>();
 const chunks = raw.split(/[,;\n]+/).map((c) => c.trim());
 for (const chunk of chunks) {
 const nums = chunk.split(/[\sxx*]+/).filter((v) => !Number.isNaN(+v));
 if (nums.length !== 2 || nums.some((v) => !Number.isFinite(+v)))
 errors.push(`«${chunk}» — нужно ровно два числа: левое и правое`);
 continue;
 }
 const id = `${nums[0]}:${nums[1]}`;
 if (seen.has(id)) {
 errors.push(`точка (${nums[0]}; ${nums[1]}) дважды`);
 continue;
 }
 pairs.push({ key: id, grade: +nums[0], x: +nums[1], y: +nums[2] });
 seen.add(id);
}

```



```

while (grew) {
 grew = false;
 for (const e of edges) {
 if (childComp.has(pid(e.b)) && !childComp.has(pid(e.a)))
 childComp.add(pid(e.a));
 grew = true;
 }
}
if (childComp.has(pid(parent))) return { ok: false, reason: "уже есть" };
return {
 ok: true,
 edge: { a: parent, b: child },
 reason:
 x.grade > y.grade ? undefined : "линия в treap violates heap property"
};
}

export function checkGameState(
 points: Pair[],
 edges: GameEdge[],
): { heapOk: boolean; singleParent: boolean; connected: boolean } {
 const n = points.length;
 if (n < 2) return { heapOk: true, singleParent: true, connected: true };
 const heapOk = edges.every((e) => e.a.grade > e.b.grade);
 const parentCount = new Map<string, number>();
 for (const e of edges) parentCount.set(pid(e.b), (parentCount.get(pid(e.b)) || 0) + 1);
 const singleParent = [...parentCount.values()].every(v => v === 1);
 // связность: компоненты через union-find
 const comp = new Map<string, string>(points.map((p) => [p, p]));
 const find = (x: string): string => {
 while (comp.get(x) !== x) {
 comp.set(x, comp.get(x)!);
 x = comp.get(x)!;
 }
 return x;
 };
 for (const e of edges) {

```

```

 if (sorted[0].key <= p.key) left = sorted[0];
 else right = sorted[0];
 }
 if (left) walk(left);
 inorder.push(p.key);
 if (right) walk(right);
};
walk(roots[0]);
if (seen.size !== n) bstOk = false;
for (let i = 1; i < inorder.length; i++)
 if (inorder[i] < inorder[i - 1]) bstOk = false;
}
return { heapOk, singleParent, connected, countOk, bs
}

```

```

/** Дерево из проведённых линий – для показа результата
export function edgesToTree(edges: GameEdge[]): TNode |
 const parentOf = new Map<string, GameEdge>();
 for (const e of edges) parentOf.set(pid(e.b), e);
 const roots = edges.filter((e) => !parentOf.has(pid(e
 if (!roots.length && edges.length) return null;
 const nodes = new Map<string, TNode>();
 const make = (p: Pair): TNode => ({
 id: pid(p),
 key: p.key,
 grade: p.grade,
 left: null,
 right: null,
 });
 const rootPair = roots.length ? roots[0].a : edges.le
 if (!rootPair) return null;
 const build = (p: Pair): TNode => {
 const node = make(p);
 nodes.set(pid(p), node);
 const kids = edges.filter((e) => pid(e.a) === pid(p
 if (kids.length > 2) return node; // невалидно – по
 kids.sort((u, v) => u.key - v.key);
 const left = kids.filter((k) => k.key <= p.key); //

```

```

useEffect(() => {
 if (!playing || phase === "game") return;
 const t = setTimeout(() => {
 if (phase === "sort") {
 if (algo === "none") { setPlaying(false); return; }
 if (sortIdx < 3) setSortIdx(sortIdx + 1);
 else setPlaying(false);
 } else {
 const total = order === "y-desc" ? canonEdges.length;
 if (connIdx < total) setConnIdx(connIdx + 1);
 else setPlaying(false);
 }
 }, 900);
 return () => clearTimeout(t);
}, [playing, phase, sortIdx, connIdx, algo, order, canonEdges]);

const startConnect = (ord: ConnectOrder) => {
 setOrder(ord);
 setPhase("connect");
 setConnIdx(0);
 setPlaying(true);
};

const comparisons = algo === "quicksort" ? (n > 1 ? Math.floor(n / 2) : 0) : 0;
const connEdgesAll = order === "y-desc" ? canonEdges : canonEdges.reverse();
const connEdges = connEdgesAll.slice(0, connIdx);
const connDone = phase === "connect" && n >= 1 && connIdx >= connEdges.length;

// --- игра «соедини сам» ---
const [gameEdges, setGameEdges] = useState<GameEdge[]>([]);
const [sel, setSel] = useState<Pair | null>(null);
const [cursor, setCursor] = useState<{ x: number; y: number}>({ x: 0, y: 0 });
const [gameMsg, setGameMsg] = useState<string | null>(null);
const [badEdge, setBadEdge] = useState<GameEdge | null>(null);
const [hintUsed, setHintUsed] = useState(false);

const resetGame = () => {

```

```

 attemptConnect(sel, p);
 setSel(null);
 }
};
const pointUp = (p: Pair) => {
 if (phase !== "game" || !sel) return;
 if (sel.key === p.key && sel.grade === p.grade) return;
 attemptConnect(sel, p);
 setSel(null);
 setCursor(null);
};
const planeMove = (e: React.PointerEvent<SVGSVGElement>) => {
 if (!sel) return;
 const rect = (e.currentTarget as SVGSVGElement).getBoundingClientRect();
 setCursor({ x: ((e.clientX - rect.left) / rect.width) * 100 });
};

const undoEdge = () => {
 setGameEdges((prev) => prev.slice(0, -1));
 setGameMsg(null);
};
const showSolution = () => {
 setGameEdges(canonEdges.map((e) => ({ a: e.a, b: e.b })));
 setHintUsed(true);
 setGameMsg("Решение показано – собери сам в следующем наборе");
};

return (
 <div className="space-y-5">
 {/* Произвольный список */}
 <div className="bg-slate-900/60 rounded-xl p-4 border border-slate-700">
 <div className="flex flex-wrap items-center justify-center">
 <h4 className="text-sm font-bold text-slate-300">Задача 1
 <button onClick={() => { setRaw(DEMO_RAW); setRawIndex(0); }}
 py-1 rounded-lg text-xs font-bold bg-slate-700 text-slate-200
 демо-набор
 </button>
 </div>
 </div>
 </div>
);

```

```

 className={`px-3 py-1.5 rounded-lg text-white`} : "bg-slate-700 text-slate-400"
 >
 {a === "quicksort" ? "quicksort (сравнение)" : "сравнение"}
 </button>
)}}
 </div>
) : (
 <>

 <button
 onClick={() => startConnect("y-desc")}
 className={`px-3 py-1.5 rounded-lg text-white`} : "bg-emerald-600 text-white" : "bg-slate-700 text-slate-400"
 >
 по y ↓ (алгоритм)
 </button>
 <button
 onClick={() => startConnect("left-right")}
 className={`px-3 py-1.5 rounded-lg text-white`} : "bg-emerald-600 text-white" : "bg-slate-700 text-slate-400"
 >
 слева направо
 </button>
 <button
 onClick={startGame}
 disabled={n < 2}
 className={`px-3 py-1.5 rounded-lg text-white`} : "bg-indigo-500 text-white" : "bg-indigo-500/80 text-white"
 >
 соединить сам
 </button>
 </div>
)}
</div>
</div>
<Plane
 points={userPairs}

```

```

 : точки прибиты, дерево одно. А теперь
) : (
 <>Линий проведено: {connIdx} из {connEdgesA
 })
</div>
</div>

{/* Чек-лист инвариантов */}
<div className={`rounded-xl p-4 border transition
 er-slate-700 bg-slate-900/40`} `>
 <h4 className="text-sm font-bold text-slate-300">
 <ul className="space-y-1.5 text-sm">
 {phase === "game" ? (
 <>
 {verdict.heapOk ? " " : " "} каждое р
 {verdict.singleParent ? " " : " "} у
 {verdict.connected ? " " : " "} все т
 {verdict.countOk ? " " : " "} линий р
 {verdict.bstOk ? " " : " "} обход сле
 </>
) : (
 <>
 {connDone ? " " : " "} каждое ребро в
 {connDone ? " " : " "} обход слева-на
 {connDone ? " " : " "} линий ровно <b
 </>
)}

 {phase === "game" && (
 <div className="flex flex-wrap gap-2 mt-3">
 <button onClick={undoEdge} disabled={!gameE
 white disabled:opacity-30">↶ Убрать линию</
 <button onClick={resetGame} disabled={!gameE
 ate-700 text-white disabled:opacity-30"> С
 <button onClick={showSolution} disabled={ga
 te disabled:opacity-30"> Показать решение<
 </div>
)}
}
```

```

const rTree = split.r;

return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center gap-3">
 Разрез
 <select value={x0} onChange={(e) => { setX0(Num
 -700 rounded-lg px-3 py-1.5 text-sm font-mono
 {[2, 3, 6, 7, 9, 12, 45].map((k) => <option k
 </select>
 <button onClick={() => setStepIdx(Math.max(0, i
 bg-slate-700 text-white disabled:opacity-30">
 <button onClick={() => setStepIdx(Math.min(step
 font-bold bg-indigo-600 text-white disabled:op
 шаг {i
 </div>

 <div className="bg-slate-900/40 rounded-xl p-4 bo
 <h4 className="text-sm font-bold text-slate-300
 , розовые → R
 <TreeView
 root={fullTree}
 nodeColor={(id) => (inL.has(id) ? "#1e3a8a" :
 nodeStroke={(id) => (inL.has(id) ? "#60a5fa"
 subtitle={done ? "Каждый узел покрашен ровно
 />
 </div>

 <div className="text-xs text-slate-300 bg-slate-9
 {steps[idx]?.note}
 </div>

 {done && (
 <div className="grid grid-cols-1 md:grid-cols-2
 <div className="bg-sky-950/20 rounded-xl p-3 l
 <h5 className="text-sm font-bold text-sky-3
 <TreeView root={lTree} width={480} height={
 </div>

```

```

 <h5 className="text-sm font-bold text-sky-300">
 <TreeView root={l} width={480} height={220} n
 </div>
 <div className="bg-rose-950/20 rounded-xl p-3 b
 <h5 className="text-sm font-bold text-rose-300">
 <TreeView root={r} width={480} height={220} n
 </div>
 </div>

 <div className="text-xs text-slate-300 bg-slate-9
 {cur?.chosenId ? (
 <> {cur.note}</>
) : (
 <>Готово: {done ? "деревья склеены обратно в
 />
)}
 </div>

 {done && (
 <div className="bg-emerald-950/20 rounded-xl p-3
 <h5 className="text-sm font-bold text-emerald
 <TreeView root={merged.root} subtitle="merge(
 </div>
)}

 <div className="text-xs text-slate-400 bg-slate-9
 Грамотно про «слияние»: у корней сравни
 в (то, что сохраняет порядок x) отправляется
 </div>
 </div>
);
};

/* ===== РЕЖИМ ERASE ===== */

const EraseMode: React.FC = () => {
 const [key, setKey] = useState(9);
 const [stepIdx, setStepIdx] = useState(0);

```



```

 <div className="bg-slate-900/40 rounded-xl p-4" >
 <h4 className="text-sm font-bold text-slate-300" >
 <TreeView root={res.root} subtitle="Валидность" >
 <ul className="mt-2 space-y-1 text-sm" >
 обход слева-направо по-прежнему сортир
 каждое ребро вниз по у (куча целая) –

 </div>
 </div>
 </div>
 </div>
 <div className="text-xs text-slate-300 bg-slate-900/40 p-4" >
 {steps[idx]?.note}
 </div>
 <div className="text-xs text-slate-400 bg-slate-900/40 p-4" >
 Ответ на «убрать можно только самый верхний»
 ягивает merge двух детей. Удаляется любая
 </div>
</div>
);
};

/* ===== РЕЖИМ LAYERS (2k/2k+1) =====

const LayersMode: React.FC = () => {
 const [pick, setPick] = useState<"heap" | "treap">("h
 // Полное дерево из 7 узлов (куча): индексы 1..7
 const heapArr = [0, 98, 45, 90, 32, 21, 76, 12];
 const full = useMemo(() => buildTree(POINTS), []);
 // Честная слотовая разметка treap: индексы как в куче
 const slots = useMemo(() => {
 const arr: (string | null)[] = Array(16).fill(null);
 const walk = (n: TNode | null, i: number) => {
 if (!n || i >= 16) return;
 arr[i] = n.id;
 walk(n.left, 2 * i);
 walk(n.right, 2 * i + 1);
 };
 });
};

```



```

 </button>
 })
 <button onClick={() => { setStage(0); setBidx(0) }}>
 Сброс</button>
</div>

<div className="flex gap-1.5 overflow-x-auto">
 {(stage === 2 ? sortedByKey : shuffled).map((p, i) => {
 const highlight = stage === 1 && i === wrongM.length - 1;
 const inBs = stage === 2 && (() => { const s = sortedByKey(p);
 const isMid = stage === 2 && (() => { const s = sortedByKey(p);
 const isFound = stage === 2 && bidx >= bsSteps.length - 1;
 return (
 <div key={` ${p.key}: ${p.grade} - ${i} `} className="p-2 border-1
 position-all ${highlight ? "border-rose-500 border-2" : "border-
 r-amber-500 bg-amber-950/30" : inBs ? "border-rose-500 border-2"
 <div className="text-white font-bold">{p.key}</div>
 <div className="text-slate-400 text-[10px]>{p.grade}</div>
)
 });
 })}
</div>

<div className="text-xs text-slate-300 bg-slate-900 p-2">
 {stage === 0 && <>Массив пар не отсортирован и находится в беспорядке.</>}
 {stage === 1 && <> Бинарный поиск посмотрел на первую половину... в которой лежала цель. На месте не на
 реставил: массив как был в беспорядке, так и остался.
 {stage === 2 && bidx < bsSteps.length - 1 && <>
 {stage === 2 && bidx >= bsSteps.length - 1 && <>
 сравнения. Но чтобы поиск вообще заработал, нужно сделать
 сравнения на всех. «Отсортирую бинарным поиском»
 </div>

 <div className="text-xs text-slate-400 bg-slate-900 p-2">
 Есть ли разница, чем сортировать? Сравнение элементов по
 нная граница, каждое сравнение даёт 1 бит. Но для поиска

```

```

 }
 };

 const win = v.win;

 return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center gap-2">
 <button onClick={() => { setEdges(edges.slice(0,
 rounded-lg text-xs font-bold bg-slate-700 text-white
 <button onClick={() => { setEdges([]); setSel(n
 3 py-1.5 rounded-lg text-xs font-bold bg-slate-700
 <button onClick={() => { setEdges(CANON_GAME.map
); }} className="px-3 py-1.5 rounded-lg text-white
 линий:
 </div>

 <div className="bg-emerald-950/20 rounded-xl p-4">
 <svg viewBox={`0 0 ${PL_W} ${PL_H + 20}`} class="w-full h-full">
 <defs>
 <pattern id="gm-grid" width="14" height="14">
 <path d="M 14 0 L 0 0 0 14" fill="none" stroke="white" stroke-width="1"/>
 </pattern>
 </defs>
 <rect x="34" y="16" width={PL_W - 46} height={PL_H - 16}/>
 <line x1="34" y1={PY(0)} x2={PL_W - 12} y2={PY(0)}/>
 <line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y2={PY(1)}/>

 {edges.map((e, i) => (
 <line
 key={`ge-${i}`}
 x1={PX(e.a.key)} y1={PY(e.a.grade)}
 x2={PX(e.b.key)} y2={PY(e.b.grade)}
 stroke={i === edges.length - 1 ? "#34d399" : "white"}
 strokeWidth={i === edges.length - 1 ? 3.5 : 1}
 className="transition-all duration-300"
 />
))}
 </svg>
 </div>
 </div>
);
}

```

```

 <div className="text-xs text-slate-300 bg-slate-900" style={bg} />
 </div>
);
};

/* ===== КОРПУС ===== */

type Tab = "build" | "split" | "merge" | "erase" | "layers";

const TABS: { id: Tab; label: string }[] = [
 { id: "build", label: "Собрать" },
 { id: "split", label: "Split $\times$ " },
 { id: "merge", label: "Merge " },
 { id: "erase", label: "Erase " },
 { id: "layers", label: "Миф: $2k/2k+1$ " },
 { id: "search", label: "Поиск $\neq$ сортировка" },
 { id: "game", label: "Игра " },
];

export const TreapBuildViz = () => {
 const [tab, setTab] = useState<Tab>("build");

 return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border" style={bg} >
 <div className="flex flex-wrap items-center justify-center" >
 <div className="flex items-center gap-3" >
 <div className="p-2.5 bg-emerald-600 text-white" style={bg} >

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white" >
 <p className="text-sm text-emerald-300" >Пап
 </p>
 </div>
 </div>
 </div>

 <div className="flex flex-wrap gap-2 mb-5" >
 {TABS.map((t) => {

```

```
import { GraphTraversalViz } from "../GraphTraversalViz"
import { HeapViz } from "../HeapViz";
import { JohnsonViz } from "../JohnsonViz";
import { KosarajuViz } from "../KosarajuViz";
import { KruskalSimulator } from "../KruskalSimulator";
import MnemonicCards from "../MnemonicCards";
import { PlanarityDemo } from "../PlanarityDemo";
import { QueueViz } from "../QueueViz";
import { SalmonAutomatonWidget } from "../SalmonAutomaton";
import { SegmentTreeVisualizer } from "../SegmentTreeVisualizer";
import SplayPlayground from "../SplayPlayground";
import { StackViz } from "../StackViz";
import { StringAlgorithmsViz } from "../StringAlgorithmsViz";
import { TopologicalSortViz } from "../TopologicalSortViz";
import { WaterfallAnimationWidget } from "../WaterfallAnimation";
import ChapterImage from "../ChapterImage";
import { Simulator } from "../Simulator";
import { PrefixSumViz } from "../visualizers/PrefixSumVisualizer";
import { SparseTableViz } from "../visualizers/SparseTableVisualizer";
import { TreapBuildViz } from "../TreapBuildViz";
```

```

"dynamic-programming": {
 title: "Динамическое программирование",
 hint: "Таблица подзадач заполняется на глазах.",
 Component: DPVisualizer,
},
"splay-tree": {
 title: "Splay: механика → большое дерево → собери с",
 hint: "Три вкладки: покадровая механика поворотов в
 песочница «подними узел сам» с проверкой порядка.
 Component: SplayPlayground,
},
"splay-rotations": {
 title: "Splay: механика → большое дерево → собери с",
 hint: "Три вкладки: покадровая механика поворотов в
 песочница «подними узел сам» с проверкой порядка.
 Component: SplayPlayground,
},
"graph-dfs-bfs": { title: "DFS и BFS на графе", Component:
"top-sort": { title: "Топологическая сортировка", Component:
"scc-kosaraju": { title: "Компоненты сильной связности", Component:
"graph-articulation": { title: "Точки сочленения", Component:
"bridges-code": { title: "Мосты: DFS + tin/low", Component:
"euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component:
"planarity-euler-formula": { title: "Планарность: K5 и K3,3", Component:
dijkstra: {
 title: "Дейкстра",
 hint: "Жадно забираем ближайшую вершину и релаксируем",
 Component: () => (
 <>
 <ChapterImage vizType="dijkstra" />
 <DijkstraViz />
 </>
),
},
"bellman-ford": {
 title: "Форд–Беллман",
 Component: () => (
 <>

```

Универсальное пособие • полный исходный код

```

export const getViz = (id?: string): VizEntry | undefined
```

```

ФАЙЛ 70/83: src/components/WaterfallAnimationWidget.tsx
```

```
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО
```

```

import React, { useState } from 'react';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb }
```

```
// Структура заранее подготовленного красивого бора для
// Словарь: ["HE", "SHE", "HIS", "HERS"]
```

```
interface WNode {
 id: number;
 label: string; // путь от корня
 char: string;
 x: number; // Координаты для SVG водопада
 y: number;
 depth: number;
 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
}
```

```
const WATERFALL_NODES: { [key: number]: WNode } = {
 0: { id: 0, label: 'ROOT', char: 'Ø', x: 400, y: 60, depth: 0, fail: 0, term_link: 0, is_terminal: false, words: [] },
 // Ветка H -> E -> R -> S
 1: { id: 1, label: 'H', char: 'H', x: 280, y: 160, depth: 1, fail: 0, term_link: 0, is_terminal: false, words: [] },
 2: { id: 2, label: 'HE', char: 'E', x: 200, y: 260, depth: 2, fail: 0, term_link: 0, is_terminal: false, words: [] },
 3: { id: 3, label: 'HER', char: 'R', x: 150, y: 360, depth: 3, fail: 0, term_link: 0, is_terminal: false, words: [] },
 4: { id: 4, label: 'HERS', char: 'S', x: 100, y: 460, depth: 4, fail: 0, term_link: 0, is_terminal: false, words: [] },
 // Ветка H -> I -> S
 5: { id: 5, label: 'HI', char: 'I', x: 350, y: 260, depth: 2, fail: 0, term_link: 0, is_terminal: false, words: [] },
 6: { id: 6, label: 'HIS', char: 'S', x: 320, y: 360, depth: 3, fail: 0, term_link: 0, is_terminal: false, words: [] },
 // Ветка S -> H -> E
 7: { id: 7, label: 'S', char: 'S', x: 550, y: 160, depth: 1, fail: 0, term_link: 0, is_terminal: false, words: [] },
```



```

const handleTextChange = (e: React.ChangeEvent<HTMLInp
 const clean = e.target.value.toUpperCase().replace(
 setTextToScan(clean);
 resetSearchSimulation();
};

const resetSearchSimulation = () => {
 setCurrentIndex(0);
 setCurrentNode(0);
 setIsSearchDone(false);
 setSearchLog('Симуляция сброшена. Лосось вернулся н
 setJumpPath(null);
 setFoundMatches([]);
 setActiveSearchCodeLine(1);
};

const stepSearchSimulation = () => {
 if (currentIndex >= textToScan.length) {
 setIsSearchDone(true);
 setSearchLog('Мы проплыли весь текст! Лосось дово
 setActiveSearchCodeLine(4);
 return;
 }

 const char = textToScan[currentIndex];
 let curr = currentNode;
 let logMsg = `[Символ '${char}']`;

 let jumpType: 'normal' | 'fail' | null = null;
 void jumpType;
 let originalCurr = curr;

 if (TRIE_TRANSITIONS[curr][char] !== undefined) {
 const nextNode = TRIE_TRANSITIONS[curr][char];
 logMsg += `У пусла #${curr} (${WATERFALL_NODES[cu
 _NODES[nextNode].label}).`;
 setJumpPath({ from: curr, to: nextNode, type: 'no

```

```

 setActiveSearchCodeLine(4);
 }

 setCurrentNode(curr);
 setCurrentIndex(currentIndex + 1);
 setSearchLog(logMsg);
 if (currentMatches.length > 0) {
 setFoundMatches((prev) => [...prev, ...currentMat
 }
};

// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В
const TRAINING_STEPS_INFO = [
 {
 targetNodeId: 0,
 title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
 desc: 'Начало алгоритма BFS. Корень (ROOT) не обра
 рние вершины сразу инициализируются с fail = RO
 codeLine: 1,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'ø',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 1,
 title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
 desc: 'Мы обрабатываем первую вершину из очереди -
 просто не существует. fail[H] автоматически нап
 codeLine: 5,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 7,
 title: 'Шаг 2 (Depth 1): Вершина #7 (S)',

```

```

 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 3,
 title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
 desc: 'Переходим на Depth 3 к #3 (HER). Родитель :
 е подходят. fail[HER] = ROOT.',
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'R',
 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 6,
 title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
 desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель
 но ветка "S" → #7 (S) совпадает! fail[HIS] = #7',
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 9,
 title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) — КАК С
 desc: ' МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
 ба плывет в #1 (H) и ищет ветку "E". У вершины :
 codeLine: 4,
 scoutPos: 1,
 inspectedParentFail: 1,
 checkTargetChar: 'E',
 checkingBranchesFrom: 1,
 },
 {

```

```

 ? 'bg-indigo-600 text-white shadow-lg'
 : 'text-slate-400 hover:text-white'
 `} }
 >
 <GitBranch className="w-4 h-4" />
 Режим 1: Полное обучение (BFS от корня)
 </button>
 <button
 onClick={() => setActiveMode('search')}
 className={`flex items-center space-x-1.5 p-2
 ${activeMode === 'search'
 ? 'bg-blue-600 text-white shadow-lg'
 : 'text-slate-400 hover:text-white'}
 `}
 >
 <Play className="w-4 h-4" />
 Режим 2: Поиск в тексте
 </button>
</div>
</div>

```

```

{/* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */}
{activeMode === 'training' ? (
 /* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
 <div className="grid grid-cols-1 lg:grid-cols-3" >
 {/* Управление шагами обучения */}
 <div className="bg-slate-900/90 p-5 rounded-x" >
 <div>
 <h5 className="text-indigo-300 font-bold" >
 Полный обход в ширину (BFS)
 </h5>
 <p className="text-xs text-slate-300 mb-3" >
 Показаны все шаги очереди BFS (начиная с шага
 ar 8 (SHE → HE), где наглядно виден
 </p>
 <div className="space-y-1.5 mb-6 overflow" >
 {TRAINING_STEPS_INFO.map((step, idx) =>
 <button

```

```

>
<div>
 <h5 className="text-white font-bold mb-3">

 Код BFS построения fail

 Вершина: #{targetTrainingNode.id} ({t

 </h5>
 <div className="bg-slate-950 p-4 rounded-1
 <div className={`p-1.5 rounded flex item
-l-4 border-indigo-500 text-indigo-300 font
 1. let u = trie[r][char]; // Те
 {currentTrainInfo.codeLine === 1 && <
 тут}
 </div>
 <div className={`p-1.5 rounded flex item
-l-4 border-indigo-400 text-indigo-200 font
 2. let v = fail[r]; // Подъем к
ainInfo.inspectedParentFail].label}}
 {currentTrainInfo.codeLine === 2 && <
 к родителю}
 </div>
 <div className={`p-1.5 rounded flex item
l-4 border-amber-500 text-amber-300 font-bo
 3. while (v !== root && trie[v]
 {currentTrainInfo.codeLine === 3 && <
 т, возврат в корень}
 </div>
 <div className={`p-1.5 rounded flex item
r-l-4 border-emerald-500 text-emerald-300 f
 4. if (trie[v][char] !== undefin
 /span>
 {currentTrainInfo.codeLine === 4 && <
 ан IF!}
 </div>
 <div className={`p-1.5 rounded flex item

```

```

 />
 </div>

 <div>
 <div className="mb-4 flex flex-wrap gap-1" >

 {textToScan.split('').map((char, idx) =>
 <span
 key={idx}
 className={`w-7 h-8 flex items-center justify-center
 ${idx === currentIndex
 ? 'bg-blue-600 text-white shadow'
 : idx < currentIndex
 ? 'bg-emerald-950/60 text-emerald-400'
 : 'bg-slate-800 text-slate-500'}
 `}
 >
 {char}

)}}
 {currentIndex >= textToScan.length && (

 ГOTOBO ✓

)}
 </div>

 <button
 onClick={stepSearchSimulation}
 disabled={isSearchDone || textToScan.length === 0}
 className={`w-full py-2.5 rounded-lg font-medium
 ${isSearchDone || textToScan.length === 0
 ? 'bg-slate-700 text-slate-500 cursor-not-allowed'
 : 'bg-gradient-to-r from-blue-600 to-emerald-900/40 active:scale-[0.98]'}
 `}
 >
 {currentIndex === 0 ? 'Начать план' : 'Продолжить'}
 </button>
 </div>
 </div>
 </div>
</div>

```

```

 <div>
 <h5 className="text-slate-300 font-bold m-0">
 <AlertCircle className="w-3.5 h-3.5 text-slate-300"/>
 </h5>
 <div className="bg-slate-800 p-3.5 rounded-2xl items-center">
 {searchLog}
 </div>
 </div>
 </div>
 </div>
)}

```

```

 {/* SVG Анимация водопада */}
 <div className="bg-gradient-to-b from-blue-950 via-blue-900 to-transparent pointer-events-none hidden shadow-2xl">
 {/* Водопадные фоновые эффекты */}
 <div className="absolute inset-0 opacity-10 bg-gradient-to-b from-blue-900 to-transparent pointer-events-none"></div>
 <div className="absolute top-0 left-1/2 -translate-x-1/2">
 <div className="w-8 bg-blue-400 h-full animate-pulse"></div>
 <div className="w-12 bg-blue-300 h-full animate-pulse"></div>
 <div className="w-6 bg-blue-500 h-full animate-pulse"></div>
 </div>

 <div className="flex items-center justify-between p-4">
 <h5 className="text-white font-bold text-base">
 Ветвящийся синий водопад (Борис)
 </h5>
 <div className="flex flex-wrap gap-4 text-xs">

 </div>
 </div>
 </div>
 </div>

```

```

 // Подсветка в режиме поиска
 let isHighlighted = activeMode === 'search' || activeMode === 'normal';

 // Подсветка в режиме обучения (когда и
 let isTrainingExamined = activeMode === 'training' || activeMode === 'normal';
 let isMatchBranch = isTrainingExamined && !isHighlighted;

 return (
 <g key={`edge-${fromNode.id}-${toNode.id}`}
 {/* Линия течения воды */}
 <line
 x1={fromNode.x}
 y1={fromNode.y}
 x2={toNode.x}
 y2={toNode.y}
 stroke={isHighlighted ? '#3b82f6' : '#ccc'}
 strokeWidth={isHighlighted ? 2 : 1}
 className={isHighlighted ? 'highlighted' : ''}
 />
 {/* Символ перехода */}
 <circle cx={{fromNode.x + toNode.x} / 2} cy={{fromNode.y + toNode.y} / 2}
 r={10}
 fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5b') : 'white'}
 stroke={isHighlighted ? '#3b82f6' : '#ccc'}
 strokeWidth={isHighlighted ? 2 : 1}
 />
 <text
 x={{(fromNode.x + toNode.x) / 2}}
 y={{(fromNode.y + toNode.y) / 2} + 15}
 fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5b') : 'white'}
 fontSize="12"
 fontWeight="bold"
 textAnchor="middle"
 />
 {char}
 </text>

 {/* ВИЗУАЛЬНЫЕ МЕТКИ ПРОВЕРКИ IF В РЕЖИМЕ ОБУЧЕНИЯ */}
 {isTrainingExamined && (
 <g transform={`translate(${(fromNode.x + toNode.x) / 2}, ${
 (fromNode.y + toNode.y) / 2})`}
 <rect
 x={-10}
 y={-10}
 width={20}
 height={20}
 fill={isMatchBranch ? '#10b981' : '#f43f5b'}
 stroke={isHighlighted ? '#3b82f6' : '#ccc'}
 strokeWidth={isHighlighted ? 2 : 1}
 />
 </g>
)}
)
)
}

```



```

 stroke={isHighlighted ? '#f43f5e' : '#2563eb'}
 strokeWidth={isHighlighted || isJustEstablished ? 2 : 1}
 strokeDasharray="6,6"
 className={isHighlighted || isJustEstablished ? 'highlighted' : ''}
 opacity={isHighlighted || isJustEstablished ? 1 : 0.5}
 />
 {isJustEstablished && (
 <text x={cx} y={cy - 10} fill="#10b981">
 fail[{node.label}] = {targetNode.label}
 </text>
)}
 </g>
);
}
}
}

```

```

{/* Рендер вершин (бассейнов водопада) */}
Object.values(WATERFALL_NODES).map((node) => {
 const isCurrent = activeFishPosNode.id === node.id;
 const isTargetChild = activeMode === 'target' && node.id === targetNode.id;
 const hasWords = node.words.length > 0;

```

```

 return (
 <g key={`node-${node.id}`} className={isCurrent ? 'current' : ''}>
 {/* Внешнее свечение для активной вершины */}
 {isCurrent && (
 <circle cx={node.x} cy={node.y} r={15} fill="none" stroke="#f43f5e" />
)}
 {isTargetChild && (
 <circle cx={node.x} cy={node.y} r={15} fill="none" stroke="#2563eb" />
)}

```

```

 {/* Основной круг вершины */}
 <circle
 cx={node.x}
 cy={node.y}
 r={isCurrent || isTargetChild ? 24 : 18}
 fill={isCurrent ? '#2563eb' : isTargetChild ? '#f43f5e' : '#10b981'}
 />

```

```

 }}}

 { /* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ */
 <g>
 { /* Круг-подложка под рыбу */
 <circle
 cx={activeFishPosNode.x + 15}
 cy={activeFishPosNode.y - 25}
 r="18"
 fill={activeMode === 'training' ? '#8b5'
 stroke="#ffffff"
 strokeWidth="2"
 style={{ transition: 'cx 0.8s cubic-bez
 className="shadow-lg"
 />
 { /* Сама рыба */
 <text
 x={activeFishPosNode.x + 15}
 y={activeFishPosNode.y - 19}
 fontSize="18"
 textAnchor="middle"
 style={{ transition: 'x 0.8s cubic-bezi
 className="animate-float select-none"
 >
 {activeMode === 'training' ? ' ♂ ' :
 </text>
 </g>

 </svg>
</div>
</div>

{ /* Список найденных совпадений (только в режиме
{activeMode === 'search' && {
 <div className="mt-6 bg-slate-900/50 p-5 rounded
 <h5 className="text-white font-bold mb-3 text
 Улов лосося (Найденные слова
 </h5>

```

```
/**
 * Префиксные суммы: 1D и 2D.
 *
 * Главная фишка — наведение на число:
 * * в 1D наводим на P[i] — подсвечивается кусок массива
 * * в 1D наводим на a[i] — видно, в какие префиксы он входит
 * * в 2D наводим на ячейку — подсвечивается прямоугольник
 * а при выбранном запросе показывается формула включения
 */
```

```
const A1 = [3, 1, 4, 1, 5, 9, 2, 6];
```

```
const A2 = [
 [1, 2, 3, 4],
 [5, 6, 7, 8],
 [9, 1, 2, 3],
 [4, 5, 6, 7],
];
```

```
const cellBase =
 "flex items-center justify-center rounded-md font-mono text-sm" +
 "w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] text-center";
```

```
export const PrefixSumViz: React.FC = () => {
 const [mode, setMode] = useState<"1d" | "2d">("1d");
 return (
 <div className="w-full bg-slate-950 rounded-2xl border border-slate-800">
 <div className="flex gap-2 mb-4 overflow-x-auto">
 {(
 [
 ["1d", "1. Одномерные суммы"],
 ["2d", "2. Двумерные суммы"],
] as const
).map(([key, label]) => (
 <button
 key={key}
 onClick={() => setMode(key)}
 />
))}
 </div>
 </div>
);
}
```

```

<div className="flex gap-1.5 mb-1 pl-[52px]">
 {A1.map(({_, i}) => (
 <div key={i} className={` ${cellBase} !h-5
 {i}
 </div>
))}
</div>

```

```

{/* массив a */}
<div className="flex gap-1.5 items-center mb-1">
 <div className="w-[46px] shrink-0 text-right">
 {A1.map(({v, i}) => {
 const inCover = covered && i >= covered.f
 const inRange = i >= range.l && i <= range.r
 return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind:
 onMouseLeave={() => setHover(null)}
 onClick={() => setRange((r) => (i < r
 className={` ${cellBase} cursor-pointe
 inCover
 ? "bg-indigo-500 text-white ring-1
 : inRange
 ? "bg-indigo-900/70 text-indigo
 : "bg-slate-800 text-slate-300
 }`}
 title={`a[${i}] = ${v}`}
 >
 {v}
 </div>
);
 })}
 </div>

```

```

{/* массив P */}
<div className="flex gap-1.5 items-center">
 <div className="w-[46px] shrink-0 text-right">

```

```

 <b className="text-emerald-400">
 P[{hover.i}] = {pref[hover.i]}
 {" "}
 – это сумма всего, что набежало от начала

 a[0..{hover.i - 1}] = {A1.slice(0, hove

 </p>
)
) : hover?.kind === "a" ? {
 <p className="text-slate-300">
 <b className="text-indigo-400">
 a[{hover.i}] = {A1[hover.i]}
 {" "}
 входит во все префиксы, начиная с{" "}

 подвинуть границу запроса.

 </p>
) : {
 <p className="text-slate-500">
 Наведите курсор на любое число: у <b classN
 у <b className="text-indigo-400">a – ку
 </p>
 }
}
</div>

{/* Запрос */}
<div className="bg-slate-900 border border-indigo
 <div className="text-xs text-slate-400 mb-2">
 Запрос суммы на отрезке (кликайте по массиву
 </div>
 <p className="font-mono text-sm sm:text-base te
 sum(a[{range.l}..{range.r}]) = P[{range.r + 1
 {pref[rang
 {pref[range.l
 {
 </p>
 <p className="text-[11px] text-slate-500 mt-1">

```

```

return (
 <div className="space-y-5">
 <div className="grid gap-5 lg:grid-cols-2">
 /* Исходная матрица */
 <div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 <div className="inline-block">
 {A2.map((row, i) => {
 <div key={i} className="flex gap-1.5 mb-1">
 {row.map((v, j) => {
 const inRect = i >= rect.r1 && i <= r1
 return (
 <div
 key={j}
 onClick={() =>
 setRect((r) =>
 i < r.r1 || j < r.c1 ? { r1:
)
 }
 className={` ${cellBase} cursor-po
 inRect ? "bg-indigo-500 text-wh
 }`}
 >
 {v}
 </div>
)
 })
 </div>
 })
 </div>
 </div>
 </div>
 </div>
)

 /* Матрица префиксных сумм */
 <div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 <div className="inline-block">
 {P.map((row, i) => {

```

```

 {hover ? (
 <p className="text-slate-300">
 <b className="text-amber-400">
 $P[\{hover.i\}][\{hover.j\}] = \{P[hover.i][hov$
 {" "}
 – сумма всего прямоугольника от левого верх
 </p>
) : (
 <p className="text-slate-500">Наведите курсор
)}
 </div>

 <div className="bg-slate-900 border border-indigo
 <p className="font-mono text-sm sm:text-base te
 сумма = A<
 C + <s
 {
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Вычли два «хвоста» и вернули дважды вычитенный
 </p>
 </div>
 </div>
);
};

```

ФАЙЛ 72/83: src/components/visualizers/SparseTableViz.t  
 КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 709 | РАЗМ

```

import React, { useState } from "react";
import { motion, AnimatePresence } from "motion/react";
import {
 Play,
 RotateCcw,
 ChevronRight,

```

```

for (let r = 0; r < 8; r++) {
 for (let c = 0; c < 8; c++) {
 ST2D[r][c][0][0] = val2D[r][c];
 }
}

for (let kx = 0; kx <= 3; kx++) {
 for (let ky = 0; ky <= 3; ky++) {
 if (kx === 0 && ky === 0) continue;
 for (let r = 0; r + (1 << kx) <= 8; r++) {
 for (let c = 0; c + (1 << ky) <= 8; c++) {
 if (kx > 0) {
 ST2D[r][c][kx][ky] = Math.min(
 ST2D[r][c][kx-1][ky],
 ST2D[r + (1 << (kx-1))][c][kx-1][ky]
);
 } else {
 ST2D[r][c][kx][ky] = Math.min(
 ST2D[r][c][kx][ky-1],
 ST2D[r][c + (1 << (ky-1))][kx][ky-1]
);
 }
 }
 }
 }
}

export const SparseTableViz: React.FC = () => {
 const [tab, setTab] = useState<"1d" | "2d_build" | "2d"> "1d";

 return (
 <div className="w-full bg-slate-950 p-3 sm:p-4 md:p-5">
 <div className="flex gap-2 sm:gap-4 mb-5 border-b border-slate-800">
 <button
 onClick={() => setTab("1d")}
 className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white : "bg-slate-800 text-slate-400"}`
 />
 </div>
 </div>
);
}

```



```

 for (let i = 0; i + (1 << j) <= N; i++) {
 computeSteps.push({ i, j });
 }
 }

 const maxStep = computeSteps.length;
 const covered = hover ? { from: hover.i, to: hover.i + 1 } : null;

 return (
 <div className="flex flex-col gap-5">
 <div className="flex items-center justify-between">
 <div>
 <h3 className="text-lg font-bold text-indigo-600">
 Построение Sparse Table (Min)
 </h3>
 <p className="text-sm text-slate-400">
 Формула: $ST[i][j] = \min(ST[i][j-1], ST[i + 2^{j-1}][j-1])$
 </p>
 </div>
 </div>
 <div className="flex gap-2">
 <button
 onClick={() => setStep(Math.max(0, step - 1))}
 className="p-2 bg-slate-800 hover:bg-slate-700 disabled:{step === 0}"
 >
 <ChevronLeft size={20} />
 </button>
 <button
 onClick={() => setStep(Math.min(maxStep, step + 1))}
 className="p-2 bg-indigo-600 hover:bg-indigo-700 disabled:{step === maxStep}"
 >
 <ChevronRight size={20} />
 </button>
 <button
 onClick={() => setStep(0)}
 className="p-2 bg-slate-800 hover:bg-slate-700"
 >

```

```


 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор (или тапните) на любое чи
 </p>
)}
 </div>
</div>

<div className="overflow-x-auto pb-2">
 <div className="inline-block min-w-full bg-slate-
 <div className="grid grid-cols-[auto_repeat(4
 {/* Headers */}
 <div className="text-slate-500 font-mono text-
 i \ j
 </div>
 <div className="text-center font-bold text-slate-500">
 rap">
 2^0 (L=1)
 </div>
 <div className="text-center font-bold text-slate-500">
 rap">
 2^1 (L=2)
 </div>
 <div className="text-center font-bold text-slate-500">
 rap">
 2^2 (L=4)
 </div>
 <div className="text-center font-bold text-slate-500">
 rap">
 2^3 (L=8)
 </div>

 {/* Matrix */}
 {Array.from({ length: N }).map((_, i) => {
 <React.Fragment key={i}>
 <div className="text-slate-500 font-mono

```

```

 <div className="w-[clamp(30px,9vw,17px)]"
) : {
 <motion.div
 onMouseEnter={() => isVisible}
 onMouseLeave={() => setHover(false)}
 onClick={() => isVisible && setSrc1()}
 initial={{ opacity: 0, scale: 0.8 }}
 animate={{
 opacity: isVisible ? 1 : 0,
 scale: isVisible ? 1 : 0.8,
 }}
 className={`w-[clamp(30px,9vw,17px)] font-bold transition-colors
 hover && hover.i === i && hover.i === i && hover.i === i
 ? "bg-sky-500 text-white"
 : isActive
 ? "bg-indigo-500 text-white"
 : isSrc1
 ? "bg-emerald-500 text-white"
 : isSrc2
 ? "bg-amber-500 text-white"
 : isVisible
 ? "bg-slate-800 text-white"
 : "bg-transparent text-white"
 }`}
 >
 {stID[i][j]}
 </motion.div>
)}

```

```

 /* Arrows visualization */
 {(isSrc1 || isSrc2) && (
 <motion.svg
 initial={{ opacity: 0 }}
 animate={{ opacity: 1 }}
 className="absolute pointer-events-none"
 style={{
 width: 100,
 }}
 >
 <rect
 x={x}
 y={y}
 width={width}
 height={height}
 fill="white"
 stroke="black"
 stroke-width="1"
 />
 </svg>
)}

```

```

 ,
 <span className="text-amber-400 font-bo
 ST[{c.i + half}][{c.j - 1}]

) → min(
 {st1

 {st1D[c.i + half][c.j - 1]}

) ={" "}
 <span className="text-indigo-400 font-bo
 {st1D[c.i][c.j]}

 </p>
);
 })()
) : (
 <p className="text-slate-500 italic">
 Нажмите далее, чтобы увидеть, как блоки сво
 </p>
 })
</div>
</div>
);
};

```

```

const Viz2DBuild: React.FC = () => {
 const [k, setK] = useState(1);
 const [hover, setHover] = useState<{ r: number; c: num
 const [locked, setLocked] = useState<{ r: number; c: r

 const L = 1 << k;
 const activeCell = locked || hover;

 return (
 <div className="flex flex-col xl:flex-row gap-6">
 <div className="flex-1 min-w-0 bg-slate-900 p-3 s
 <h3 className="text-xl font-bold text-indigo-40

```

```

<div className={`grid gap-[2px] p-
go-900/40 shadow-[0_0_25px_rgba(99,102,241,
eColumns: `repeat(${stepSize}, max-content)
{Array.from({length: stepSize * st
 const r = Math.floor(idx / step
 const c = idx % stepSize;

 let highlightClass = isCurr ? '
/50 cursor-pointer' : 'bg-slate-800/50 text
 let zIndex = 'z-0';

 const isActive = !!activeCell &
ll.c + L) && ((r - activeCell.r) % (1 << st

 if (isActive) {
 if (isCurr) {
 highlightClass = `bg-indigo
{locked && locked.r === r && locked.c === c
 zIndex = 'z-10';
 } else {
 const prevL = 1 << (k - 1);
 const isTop = r < activeCel
 const isLeft = c < activeCe

 if (isTop && isLeft) {
 highlightClass = 'bg-ros
md:scale-[1.08]';
 } else if (isTop && !isLeft
 highlightClass = 'bg-blur
] md:scale-[1.08]';
 } else if (!isTop && isLeft
 highlightClass = 'bg-eme
9,0.3)] md:scale-[1.08]';
 } else {
 highlightClass = 'bg-amb
3)] md:scale-[1.08]';
 }
 zIndex = 'z-10';

```



```

 }}({})
 </div>

 <div className="pt-3 mt-3 border-top"
 > <span className="text-white"
0">{ST2D[activeCell.r][activeCell.c][k][k]}
 </div>
 <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
 </div>
) : {
 <div className="bg-slate-950/40 border
mt-2 flex items-center justify-center anim
 Наведите курсор на матрицу {L}x{L}
 </div>
)}
 </div>
</div>
</div>
</div>
);
};

```

```

const Viz2DQuery: React.FC = () => {
 const [r1, setR1] = useState(1);
 const [c1, setC1] = useState(1);
 const [r2, setR2] = useState(5);
 const [c2, setC2] = useState(6);

 const H = r2 - r1 + 1;
 const W = c2 - c1 + 1;
 const kx = Math.floor(Math.log2(H));
 const ky = Math.floor(Math.log2(W));
 const Lx = 1 << kx;
 const Ly = 1 << ky;

 const matRows = 8 - Lx + 1;

```

```

 /* Показываем 4 угла в самой матрице
 <div className="absolute pointer-events-none shadow-[0_0_10px_#f43f5e]" style={{ top: `calc(${(Ly / 8)} * 100}% - 16px)`, height: `calc(${(Ly / 8)} * 100}% - 16px)`, width: `calc(${(Ly / 8)} * 100}% - 16px)`, left: `calc(${(Lx / 8)} * 100}% - 16px)`}>
 <div className="absolute pointer-events-none shadow-[0_0_10px_#3b82f6]" style={{ top: `calc(${(Ly / 8)} * 100}% - 16px)`, height: `calc(${(Ly / 8)} * 100}% - 16px)`, width: `calc(${(Ly / 8)} * 100}% - 16px)`, left: `calc(${(Lx / 8)} * 100}% - 16px)`}>
 <div className="absolute pointer-events-none shadow-[0_0_10px_#10b981]" style={{ top: `calc(${(Ly / 8)} * 100}% - 16px)`, height: `calc(${(Ly / 8)} * 100}% - 16px)`, width: `calc(${(Ly / 8)} * 100}% - 16px)`, left: `calc(${(Lx / 8)} * 100}% - 16px)`}>
 <div className="absolute pointer-events-none shadow-[0_0_10px_#f59e0b]" style={{ top: `calc(${(Ly / 8)} * 100}% - 16px)`, height: `calc(${(Ly / 8)} * 100}% - 16px)`, width: `calc(${(Ly / 8)} * 100}% - 16px)`, left: `calc(${(Lx / 8)} * 100}% - 16px)`}>
 </div>

 <div className="bg-slate-950 p-4 rounded-2xl shadow-[inset_0_2px_10px_rgba(0,0,0,0.5)]">
 <label className="flex items-center justify-between gap-2">
 Left
 <div className="flex gap-2">
 <input type="number" min={0} max={100} value={0} />
 <input type="number" min={0} max={100} value={0} />
 </div>
 </label>
 <label className="flex items-center justify-between gap-2">
 Top
 <div className="flex gap-2">
 <input type="number" min={0} max={100} value={0} />
 <input type="number" min={0} max={100} value={0} />
 </div>
 </label>
 </div>
 </div>

```



```

r1][<span className="text-white bg-b

```

```


 px, чтобы нижний край уперся в r2)


```

```


 nded">r2 - Lx + 1][<span className="

```

```


 и влево от c2)


```

```


 ">r2 - Lx + 1][<span className="text

```

```

 {'}'}));

```

```

</div>

```

```

<div className="w-full flex justify-center"

```

```

 <div className="flex flex-col items-center"

```

```

 <h4 className="text-slate-400 font-bold"

```

```

 Откуда берутся эти 4 числа: матрица ST
 border-slate-700 shadow-sm">ST[][][kx]][ky]

```

```

 </h4>

```

```

 <div className="grid gap-[2px] p-2.5"
 Columns: `repeat(${matCols}, 1fr)`}}>

```

```

 {Array.from({length: matRows * matCols},

```

```

 const r = Math.floor(idx / matCols);

```

```

 const c = idx % matCols;

```

```

 let isTL = r === r1 && c === c1;

```

```

 let isTR = r === r1 && c === c2;

```

```

 let isBL = r === r2 - Lx + 1 && c === c1;

```

```

 let isBR = r === r2 - Lx + 1 && c === c2;

```

```

 let color = "bg-slate-800 text-white";

```

```

 let txt = ST2D[r][c][kx][ky].toLocaleString();

```

```

 let badge = null;

```

```

 if (isTL) { color = "bg-rose-500 text-slate-800 border-2"; badge = "TL"; }

```

```

 else if (isTR) { color = "bg-lime-500 text-slate-800 border-2"; badge = "TR"; }

```

```
}
```

---

## РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

---

---

ФАЙЛ 73/83: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРOK: 137 | РАЗМЕР

---

```
 </div>
 </main>
</div>

<script>
 function setMode(mode) {
 document.getElementById('btn-mode-normal').cla
 ? 'px-3 py-1 rounded text-sm font-bold l
 : 'px-3 py-1 rounded text-sm font-bold

 document.getElementById('btn-mode-wtf').cla
 ? 'px-3 py-1 rounded text-sm font-bold l
 : 'px-3 py-1 rounded text-sm font-bold

 const aside = document.querySelector('aside
 const headerMain = document.querySelector('
 const questionsContainer = document.getElem
 const wtfContainer = document.getElementById

 if (mode === 'wtf') {
 document.body.classList.add('wtf-mode')
 if (aside) aside.style.display = 'none'
 if (headerMain) headerMain.style.display
 if (questionsContainer) questionsContain
 if (wtfContainer) wtfContainer.classLis
```

```
// Initialize theme and mode
setTheme('dark');

// Setup intersection observer for TOC highlight
document.addEventListener('DOMContentLoaded', () => {
 const mobileMenuBtn = document.getElementById('mobile-menu');
 const mobileDrawer = document.getElementById('mobile-drawer');
 const closeDrawerBtn = document.getElementById('close-drawer');
 const drawerOverlay = document.getElementById('drawer-overlay');

 function toggleDrawer() {
 const isClosed = mobileDrawer.classList.contains('closed');
 if (isClosed) {
 mobileDrawer.classList.remove('closed');
 if (drawerOverlay) {
 drawerOverlay.classList.remove('hidden');
 // Timeout allows display:block to take effect
 setTimeout(() => drawerOverlay.classList.remove('hidden'), 10);
 }
 document.body.style.overflow = "hidden";
 } else {
 mobileDrawer.classList.add('closed');
 if (drawerOverlay) {
 drawerOverlay.classList.add('hidden');
 setTimeout(() => drawerOverlay.classList.add('hidden'), 10);
 }
 document.body.style.overflow = "";
 }
 }

 if (mobileMenuBtn && mobileDrawer) {
 mobileMenuBtn.addEventListener("click", toggleDrawer);
 }
 if (closeDrawerBtn && mobileDrawer) {
 closeDrawerBtn.addEventListener("click", toggleDrawer);
 }
 if (drawerOverlay) {
 drawerOverlay.addEventListener("click", toggleDrawer);
 }
});
```

```
 </script>
</body>
</html>
```

---

ФАЙЛ 74/83: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 1169 | РАЗМЕР: 10000 Б

---

```
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width,
 <title>Пособие по алгебре: От пиццы к предикатам</title>
 <script src="https://cdn.tailwindcss.com"></script>
 <script src="https://polyfill.io/v3/polyfill.min.js"></script>
 <script id="MathJax-script" async src="https://cdn.jsdelivr.net/npm/mathjax@3.2.2/es5/tex-mml-chtml.js"></script>
 <script>
 window.MathJax = {
 tex: {
 inlineMath: [['$', '$'], ['\\(', '\\)']],
 displayMath: [['$$', '$$'], ['\\[', '\\]']],
 },
 };
 </script>
 <style>
 @import url('https://fonts.googleapis.com/css2?family=Fira+Code&family=Arial+Black&display=block');
 @reference "./src/index.css";
 .math-font { font-family: 'Fira Code', monospace; }
 .uno-card {
 width: 70px; height: 105px; border-radius: 10px;
 box-shadow: 0 4px 8px rgba(0,0,0,0.5); position: relative;
 align-items: center; justify-content: center; text-align: center;
 color: white; overflow: hidden; flex-shrink: 0;
 text-shadow: 2px 2px 0 #000, -1px -1px 0 #000;
 font-family: 'Arial Black', Impact, sans-serif;
 }
 </style>
</head>
<body>
```

```

 html { font-size: 13px; }

 .uno-card { width: 40px; height: 60px; font-
 .uno-card::before, .uno-card::after { font-
 .uno-card::before { top: 1px; left: 2px; }
 .uno-card::after { bottom: 1px; right: 2px; }

 .uno-card.mini { width: 28px !important; he
 us: 4px; }
 .uno-card.mini::before, .uno-card.mini::after

 .uno-equation .text-2xl { font-size: 1.2rem
 .uno-equation.gap-4 { gap: 0.5rem !important

 /* Allow horizontal scroll for equations in
 .formula-scroll { flex-wrap: nowrap !importa
 px; }
}
.uno-inner {
 position: absolute; width: 110%; height: 75%
 border-radius: 50%; transform: rotate(-30deg
 box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
}
.uno-val { position: relative; z-index: 2; }
.card-red { background-color: #e52521; } .card-
.card-blue { background-color: #004dcf; } .card
.card-green { background-color: #339e35; } .car
.card-yellow { background-color: #ffc400; } .ca
.card-black { background-color: #222; } .card-b
.card-black::before, .card-black::after { text-
.info-block { @apply bg-slate-800/80 border-l-4
.info-title { @apply font-bold mb-2 flex items-

/* Визуализация (Аналогии) */
.analogy-block { @apply bg-slate-900 border-2 b
 ems-center gap-6; }
.analogy-badge { @apply absolute -top-3 left-4 l
 king-widest border border-slate-500 shadow-md

```

```
body.theme-notebook #top-controls { background-color: #f0f0f0; }

/* CREEPY BLINKING */
@keyframes creepy-blink {
 0%, 95%, 98%, 100% { transform: scaleY(1); }
 96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 4s infinite;
}
.creepy-eye-alt {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 5.2s infinite;
 animation-delay: 1.5s;
}
.creepy-emoji {
 display: inline-block;
 transform-origin: center;
 animation: creepy-blink 3.8s infinite;
}
</style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead">

 <div class="sticky top-0 z-50 w-full bg-slate-950/90
 center items-center gap-4 sm:gap-8 transition-colors">
 <div class="flex items-center gap-3">
 ТЕМА
 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2
 <button onclick="setMode('normal')" id="btn-normal">Нормальный
 <button onclick="setMode('wtf')" id="btn-wtf">WTF
 </div>
 </div>
 </div>
```

```

<!-- ЛЕГО БЛОКИ -->
<g id="lego-yellow">
 <rect x="0" y="0" width="40" height="20"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<g id="lego-red">
 <rect x="0" y="0" width="40" height="20"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<g id="lego-rem">
 <rect x="0" y="0" width="40" height="10"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<!-- МЯСОРУБКА -->
<g id="meat-grinder">
 <path d="M 10 40 L 15 20 Q 25 15 35 20
 <rect x="5" y="40" width="40" height="20"
 <!-- Ручка -->
 <path d="M 45 50 Q 60 50 60 30 L 75 30"
 <circle cx="75" cy="30" r="4" fill="#333"
 <!-- Выход -->
 <path d="M 5 45 L -5 45 L -5 55 L 5 55"
 <circle cx="25" cy="50" r="15" fill="#fff"
</g>
<g id="lego-green">
 <rect x="0" y="10" width="20" height="10"
 <rect x="3" y="5" width="14" height="5"
</g>
<g id="lego-white">
 <rect x="0" y="0" width="40" height="20"
 <rect x="6" y="-5" width="8" height="6"

```

```
<details class="group" open>
 <summary class="list-none cursor-pointer text-teal-500 pl-3 font-bold text-teal-300">

 1. Алгебраические структуры

 </summary>
 <div class="pl-6 space-y-2 text-teal-500">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-indigo-500 pl-3 font-bold text-indigo-300">

 2. Комплексные числа

 </summary>
 <div class="pl-6 space-y-2 text-indigo-500">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-teal-500 pl-3 font-bold text-teal-300">

 3. Тригонометрическая форма

 </summary>
 <div class="pl-6 space-y-2 text-teal-500">
```



```
<details class="group">
 <summary class="list-none cursor-pointer text-lime-500 pl-3 font-bold text-lime-300">
 6. Делимость, НОД и Евклид

</summary>
<div class="pl-6 space-y-2 text-lime-500">

</div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-pink-500 pl-3 font-bold text-pink-300">
 7. Взаимно простые числа

</summary>
<div class="pl-6 space-y-2 text-pink-500">

</div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-violet-500 pl-3 font-bold text-violet-300">
 8. Простые числа
```

```


a>

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-cyan-500 pl-3 font-bold text-cyan-400">
 11. НОД и НОК многочленов

 </summary>
 <div class="pl-6 space-y-2 text-cyan-500">

 Единственность НОД

 Пирали

 вращается!)
 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-blue-500 pl-3 font-bold text-blue-400">
 12. Взаимно простые многочлены

 </summary>
 <div class="pl-6 space-y-2 text-blue-500">

 деление)

```

```
15. Производная. Критерии

 </summary>
 <div class="pl-6 mt-2 space-y-2">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-red-500 pl-3 font-bold text-red-400">
 16. Неприводимые многочлены

 </summary>
 <div class="pl-6 mt-2 space-y-2">

 </div>
</details>

<div class="mt-4 mb-2 text-xs font-sans">


```

```

document.querySelectorAll('.group-link')
 .forEach(link => {
 link.addEventListener('click', () => {
 isScrolling = true;
 setTimeout(() => isScrolling = false, 1000);
 });
 });

const observer = new IntersectionObserver((entries) => {
 if (isScrolling) return;

 let visibleId = null;
 entries.forEach(entry => {
 if (entry.isIntersecting) {
 visibleId = entry.target.id;
 }
 });

 // Highlight active link
 document.querySelectorAll('a')
 .forEach(a => {
 a.style.fontWeight = 'normal';
 if (a.href.endsWith(visibleId)) {
 a.style.fontWeight = 'bold';
 }
 });

 // Open the parent details
 const correspondingLink = document.querySelector(`a[href="#${visibleId}"]`);
 if (correspondingLink) {
 const details = correspondingLink.closest('details');
 if (details && !details.open) {
 details.open = true;
 document.querySelector(details.querySelector('div'))
 .addEventListener('click', () => {
 if (otherLink) {
 otherLink.click();
 }
 });
 }
 }
});

```

```
header.style.marginBottom = '10px';

const subHeader = document.createElement('h3');
subHeader.innerText = 'Из-за переезда веб-студии в новой вкладке (через меню)';
subHeader.style.marginBottom = '10px';
subHeader.style.color = '#f87171';
subHeader.style.fontSize = '0.9em';

const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';

const btnContainer = document.createElement('div');
btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop = '10px';

const copyBtn = document.createElement('button');
copyBtn.innerText = 'Копировать';
copyBtn.style.padding = '10px';
copyBtn.style.backgroundColor = '#f87171';
copyBtn.style.color = '#fff';
copyBtn.style.border = 'none';
copyBtn.style.borderRadius = '5px';
copyBtn.style.cursor = 'pointer';
copyBtn.onclick = () => {
 if (navigator.clipboard) {
 navigator.clipboard.writeText(contentRaw);
 copyBtn.innerText = 'Скопировано';
 } else {
 textarea.select();
 document.execCommand('copy');
 copyBtn.innerText = 'Скопировано';
 }
}
```

```

 if (el.nextElementSibling &
 caption = el.nextElement
 }
 el.textContent = '\n\n[Илл
));
 clone.querySelectorAll('.img-cap

let text = clone.innerText;
text = text.replace(/\n{3,}/g,

if (window !== window.top) {
 fallbackModal(text, 'Markdon
 return;
}
const blob = new Blob([text], {
const url = URL.createObjectURL
const link = document.createElem
link.href = url;
link.download = 'vyisshaya_alge
document.body.appendChild(link)
link.click();
document.body.removeChild(link)
}

function setTheme(theme) {

 const body = document.body;
 body.classList.remove('bg-slate

);

document.getElementById('theme-
document.getElementById('theme-
document.getElementById('theme-

let oldStyle = document.getElem
if (oldStyle) oldStyle.remove()
const style = document.createEl
style.id = 'theme-style';

```

```

 body.bg-white .text-white {
 color: white;
 }

 /* Ensure active links are visible
 body.bg-white aside a:active {
 color: white;
 }
 body.bg-white aside a:active {
 color: white;
 }
 body.bg-white aside a:active {
 color: white;
 }
 body.bg-white aside a:active {
 color: white;
 }
 body.bg-white aside a:active {
 color: white;
 }
 body.bg-white aside a:active {
 color: white;
 }
 body.bg-white aside a:active {
 color: white;
 }

 body.bg-white table tr {
 background-color: #f9f9f9;
 }
 body.bg-white table tr {
 background-color: #f9f9f9;
 }
 body.bg-white .text-teal {
 color: #008080;
 }
 body.bg-white .text-rose {
 color: #e91e63;
 }
 body.bg-white .text-eme {
 color: #008000;
 }
 body.bg-white .text-cyan {
 color: #00bfff;
 }
 body.bg-white .text-amber {
 color: #ffc107;
 }
 body.bg-white .text-fuch {
 color: #ff00ff;
 }
 body.bg-white .text-lime {
 color: #90ee90;
 }
 body.bg-white .text-blue {
 color: #4169e1;
 }
 body.bg-white .text-green {
 color: #32cd32;
 }
 body.bg-white .text-ind {
 color: #800080;
 }

 `;
} else if (theme === 'terminal') {
 body.classList.add('bg-black');
 document.getElementById('theme')
 .style.innerHTML = `
 body.bg-black .bg-slate-950 {
 background-color: #000000;
 }
 body.bg-black section h1 {
 color: #4ade80 !important;
 border-color: #22c55e;
 font-family: monospace;
 }
 `;
}

```

```

 <div>
 <div class="text-xs text-slate-
 <div class="flex items-center g
 <button id="theme-dark" onc
justify-center text-amber-300 hover:text-wh
line-amber-500" title="Темная тема"> </butt
 <button id="theme-light" on
r justify-center text-amber-50 hover:text-w
">* </button>
 <button id="theme-terminal"
-center justify-center text-green-400 hover
терминала"> </button>
 </div>
 </div>
 </div>
 </aside>

 <!-- Основной контент -->
 <main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p
 -base min-w-0">
 <header class="text-center mb-12 border-b b
 <h1 class="text-4xl md:text-5xl font-bl
4">
 ВЫСШАЯ АЛГЕБРА
 </h1>
 <p class="text-slate-400 italic text-lg

 <!-- Print Table of Contents -->
 <div class="hidden print:block mt-8 tex
fter: always;">
 <h2 class="font-bold text-xl mb-4">
 <ul class="text-base space-y-2">
 1. Ал
 2. Кор
 3. Тр
 4. Ст

```



```

 <div class="text-xs text-yellow
 <div class="font-bold text-slate

 <a href="#question-9" onclick="setM
-4 border-orange-500 hover:bg-slate-700 tra
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <a href="#question-2" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans.
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

 <!-- Row 2: Базовые операции / Евклид
 <a href="#question-6" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

 <a href="#question-10" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <a href="#question-3" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans.
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

 <!-- Row 3: НОД и НОК / Алгоритм Ев
 <a href="#question-7" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

 <a href="#question-12" onclick="setM

```

```

<div class="hidden md:block"></div>
<a href="#question-14" onclick="setl
-l-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

<div class="hidden md:block"></div>

<!-- Row 7: Производная -->
<div class="hidden md:block"></div>
<a href="#question-15" onclick="setl
-l-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

<div class="hidden md:block"></div>

<!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
<div class="col-span-1 md:col-span-1
b-2 border-b border-emerald-500/30 pb-2">Кл

 <a href="#question-17" onclick="setl
-l-4 border-teal-500 hover:bg-slate-700 tra
 <div class="text-xs text-teal-500
 <div class="font-bold text-slate

 <a href="#question-18" onclick="setl
-l-4 border-blue-500 hover:bg-slate-700 tra
 <div class="text-xs text-blue-500
 <div class="font-bold text-slate

 <a href="#question-19" onclick="setl
-l-4 border-emerald-500 hover:bg-slate-700
 <div class="text-xs text-emerald
 <div class="font-bold text-slate

 <a href="#question-20" onclick="setl

```

</div>

```
<div class="prose prose-invert max-w-3xl">
 <p class="text-lg">
 Хватит тупить. Доказательств
 ь для того, чтобы жать на кнопки. Вот жес
 </p>
```

```
 <h2 class="text-2xl font-bold">Меметика</h2>
```

```
 <!-- Прямое -->
 <div class="bg-slate-800/80 p-6">
 <h3 class="text-xl text-blue-400">Прямое
 <p class="text-sm text-slate-400">
 <ul class="list-disc pl-5">

 k$).

 ово.

 </div>
```

```
 <!-- От противного -->
 <div class="bg-slate-800/80 p-6">
 <h3 class="text-xl text-fuchsia-400">От противного
 <p class="text-sm text-slate-400">
 <ul class="list-disc pl-5">

 i>

 т.д.).

 что так и задумано: "Ой, противоречие, теор

 </div>
```

```

 </div>
```

```
 <h2 class="text-2xl font-bold b
оритмы экзекуции)</h2>
```

```
 <div class="grid grid-cols-1 md
 <div class="bg-slate-800 p-
 <h4 class="text-red-400
 <p class="text-sm text-
 <p class="text-xs font-
со следующим. Если в конце 0 – пациент мертв
 </div>
```

```
 <div class="bg-slate-800 p-
 <h4 class="text-red-400
 <p class="text-sm text-
 <p class="text-xs font-
c. Пока получается 0 – корень жив. Как то
корня минус один.</p>
 </div>
```

```
 <div class="bg-slate-800 p-
 <h4 class="text-red-400
 <p class="text-sm text-
 <p class="text-xs font-
ициентов (они делятся на p), кроме старше
зложить.</p>
 </div>
```

```
 <div class="bg-slate-800 p-
 <h4 class="text-red-400
 <p class="text-sm text-
 <p class="text-xs font-
. Чаще всего после этого Фейс-контроль p
 </div>
 </div>
```

```
 <li class="bg-slate-800/50"

 <div>
 <h4 class="text-cyan"
 <p class="text-sm t
авь их драться. Умножение всегда врывается
 <code class="text-x
 </div>

</div>
</section>
</div>

<div id="questions-container">
<!-- ===== ВОПРОС 1 =====
```

---

## РАЗДЕЛ: УТИЛИТЫ

---

---

ФАЙЛ 75/83: src/utils/cn.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

---

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
 return twMerge(clsx(inputs));
}
```

---

ФАЙЛ 76/83: src/utils/exportHtml.ts

```
const EXTRA_CSS = `
/* — правки для автономного файла — */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family:
a.term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; padding:
.export-head { border-bottom: 1px solid #1e293b; padding:
.export-note { background: #0f172a; border: 1px solid #
 border-radius: .5rem; padding: .85rem 1rem; font-size:
.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid
.export-gloss h2 { color: #fff; font-size: 1.25rem; font
.export-term { background: #0f172a; border: 1px solid #
.export-term h3 { color: #c7d2fe; font-size: .95rem; font
.export-term p { margin: 0 0 .3rem; font-size: .85rem;
.export-term .formal { color: #94a3b8; font-size: .8rem;
.export-term .cx { color: #6ee7b7; font-size: .78rem; font
.export-foot { margin-top: 3rem; border-top: 1px solid #
@media print {
 body { background: #fff; color: #0f172a; }
 .export-note, details { break-inside: avoid; }
 details { display: block; }
 details > div, details > p { display: block !important; }
 pre { white-space: pre-wrap; }
}
`;
```

```
const escapeHtml = (s: string) =>
 s.replace(/&/g, "&").replace(/</g, "<").replace(/>/g, ">");
```

```
/**
```

```
 * Размечает термины и превращает их в якорные ссылки на
```

```
 * Возвращает готовый HTML главы и список найденных терминов
```

```
 */
```

```
function prepareBody(html: string): { body: string; terms: string[] } {
 const host = document.createElement("div");
 host.innerHTML = html;
```

```

 })
 .join("\n");

 return `
```

```

 gloss: glossarySection(termIds),
 origin: window.location.origin,
 });
 triggerDownload(html, safeName(chapter.title, chapter
}

```

```

/** Выгрузить всё пособие одним файлом со сквозным оглавлением
export async function downloadBookHtml(chapters: Chapter[]) {

```

```

 const css = await collectCss();
 const allTerms: string[] = [];
 const bodies: string[] = [];

```

```

 const toc = chapters
 .map((c) => `<li style="margin:.2rem 0"><a href="#${c.id}">${c.title}</a></li>`)
 .join("\n");

```

```

 chapters.forEach((c) => {
 const { body, termIds } = prepareBody(c.content);
 termIds.forEach((t) => {
 if (!allTerms.includes(t)) allTerms.push(t);
 });
 bodies.push(`<div id="ch-${escapeHtml(c.id)}" style="border: 1px solid #0f172a; padding: 10px; margin-bottom: 10px;">
 <p style="text-align:right;font-size:.72rem;margin:0 0 10px 0">
 <h3>${c.title}</h3>
 <ol style="list-style-type: none; padding-left: 0">
 ${c.content}

 </div>`);
 });

```

```

 const body = `<nav id="toc" style="background:#0f172a; color:#fff; padding: 10px; margin-bottom: 10px;">
 <h2 style="color:#fff;font-size:1.05rem;font-weight:800;margin:0">Оглавление
 <ol style="margin:0;padding-left:1.1rem;font-size:.85rem;">
 ${toc}

 </nav>
 ${bodies.join("\n")}`;

```

```

 const html = wrap({
 title: "Универсальное пособие по алгоритмам",
 subtitle: `Все темы, ${chapters.length} шт.`,
 css,
 body,
 gloss: glossarySection(allTerms),
 });

```



```

 outfile: bundle,
 logLevel: "error",
 });

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/c
const registryBody = registry.slice(registry.indexOf("V
const vizIds = new Set([...registryBody.matchAll(/^\\s{2

const rows = chapters.map((c) => {
 const html = c.content ?? "";
 const analogies = (html.match(/Аналогия/gi) ?? []).length;
 return {
 id: c.id,
 title: c.title,
 num: Number.parseInt(c.title.match(/^((\\d+)\\.\\/)?.[1
 analogies,
 hasAnalogy: analogies > 0,
 inlineSvg: /<svg[\\s>]/i.test(html),
 image: /<img[\\s>]/i.test(html),
 interactive: vizIds.has(c.id),
 chars: html.length,
 };
});

const has2D = (r) => r.interactive || r.inlineSvg || r.
const gapBoth = rows.filter((r) => !r.hasAnalogy && !ha
const gapAnalogy = rows.filter((r) => !r.hasAnalogy && l
const gapViz = rows.filter((r) => r.hasAnalogy && !has2D
const orphanViz = [...vizIds].filter((id) => !chapters.

const nums = rows.map((r) => r.num).filter(Boolean);
const missingNums = Array.from({ length: 24 }, (_, i) =>

const flag = (b) => (b ? " " : "-");

```

```
const ROOT = process.cwd();
await build({
 entryPoints: ["src/data/content.ts", "src/data/glossary.ts"],
 bundle: true, format: "esm", platform: "node",
 outdir: "tmp/audit-terms", outExtension: { ".js": ".mjs" },
 external: ["react", "react-dom", "motion", "lucide-react"],
});

const { chapters } = await import(path.join(ROOT, "tmp/audit-terms.mjs"));
const { GLOSSARY_LABELS, GLOSSARY } = await import(path.join(ROOT, "tmp/audit-terms.mjs"));

const esc = (s) => s.replace(/[\.*\+\?\^\$\{\}\(\)|[\]\[\]]/g, "\\$&");
const re = new RegExp(
 `(?<![\\p{L}\\p{N}_-])(?=${GLOSSARY_LABELS.map((l) => esc(l.label))})` +
 "giu"
);

const map = new Map(GLOSSARY_LABELS.map((l) => [l.label, l]));

// те же лимиты, что и в src/hooks/useTermHints.ts
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const bad = [];
const used = new Set();
console.log("тем | подсв. | глава");
for (const c of chapters) {
 const text = c.content
 .replace(/<(script|style|code|pre)[\s\S]*?<\/[\s\S]>/gi, "")
 .replace(/<[^\>]+>/g, " ");

 const perTerm = new Map();
 const perSurface = new Map();
 let highlights = 0;
 const ids = new Set();

 for (const m of text.matchAll(re)) {
 const surface = m[1].toLowerCase();
 const id = map.get(surface);
```

```
// A4 в пунктах PDF
const A4 = { width: 595.28, height: 841.89 };

async function main() {
 if (!fs.existsSync(PAGES_DIR)) {
 throw new Error(`Нет ${path.relative(ROOT, PAGES_DIR)}`);
 }

 const pages = fs
 .readdirSync(PAGES_DIR)
 .filter((f) => f.endsWith(".png"))
 .sort();

 if (pages.length === 0) throw new Error("Не найдено ни одной страницы");

 ensureDir(OUT_DIR);

 const doc = new PDFDocument({
 size: [A4.width, A4.height],
 margin: 0,
 autoFirstPage: false,
 info: {
 Title: "Универсальное пособие – полный исходный код",
 Author: "CI: rebuild-pdf",
 Subject: "Экспорт всего кода проекта (код -> txt)",
 CreationDate: new Date(),
 },
 });

 const stream = fs.createWriteStream(PDF_PATH);
 doc.pipe(stream);

 for (const file of pages) {
 doc.addPage({ size: [A4.width, A4.height], margin: 0 });
 doc.image(path.join(PAGES_DIR, file), 0, 0, { width: A4.width });
 }

 doc.end();
}
```

## Универсальное пособие • полный исходный код

```

 ".css", ".scss", ".html", ".json", ".md", ".yml", ".y
});
```

```
/** Явные исключения: генерируемое, бинарное и просто ш
const EXCLUDE_FILES = new Set(["package-lock.json", "bu
const EXCLUDE_DIRS = ["node_modules", "dist", "build",
```

```
/** Человекочитаемые категории для оглавления. */
```

```
const CATEGORIES = [
 [/^src\/data\/tickets\/\//, "Билеты (учебный контент)"],
 [/^src\/data\/\//, "Контент и данные пособия"],
 [/^src\/components\/visualizers\/\//, "Визуализаторы (в.
 [/^src\/components\/\//, "Интерактивные компоненты и сим
 [/^src\/layout\/\//, "HTML-разметка (layout)"],
 [/^src\/utils\/\//, "Утилиты"],
 [/^src\/\//, "Ядро приложения"],
 [/^scripts\/\//, "Скрипты сборки и экспорта"],
 [/^\.github\/\//, "CI / автоматизация"],
 [/^[^\/]+\$/, "Конфигурация проекта"],
];
```

```
const categoryOf = (rel) => CATEGORIES.find(([re]) => re
```

```
/** Порядок разделов в документе. */
```

```
const ORDER = [
 "Конфигурация проекта",
 "Ядро приложения",
 "Контент и данные пособия",
 "Билеты (учебный контент)",
 "Интерактивные компоненты и симуляторы",
 "Визуализаторы (вложенные)",
 "HTML-разметка (layout)",
 "Утилиты",
 "Скрипты сборки и экспорта",
 "CI / автоматизация",
 "Прочее",
];
```

```

const seen = new Set();
for (const rel of files.filter((f) => f.startsWith("s
 const src = fs.readFileSync(path.join(ROOT, rel), "
 const re = /id:\s*"([^"]+)\s*,\s*\n?\s*title:\s*"
 let m;
 while ((m = re.exec(src)) !== null) {
 const [, id, title] = m;
 if (seen.has(id)) continue;
 seen.add(id);
 chapters.push({ id, title, file: rel });
 }
}
return chapters.sort((a, b) => {
 const na = parseInt(a.title.match(/(\d+)/)?.[0] ??
 const nb = parseInt(b.title.match(/(\d+)/)?.[0] ??
 return na - nb;
}));
});
}

function main() {
 const files = listFiles();
 const chapters = extractChapters(files);
 ensureDir(OUT_DIR);

 const out = [];
 const push = (s = "") => out.push(s);

 const stamp = new Date().toLocaleString("ru-RU", { ti
 const totalBytes = files.reduce((s, f) => s + fs.stat

 push(HR);
 push(" УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТ
 push(" ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФА
 push(HR);
 push();
 push(`Дата генерации:                ${stamp}`);
 push(`Файлов исходного кода:         ${files.length}`);
 push(`Суммарный объем кода: ${humanSize(totalByt

```

```

 const lines = content.split("\n");
 push(SUBHR);
 push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
 push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} |`);
 push(SUBHR);
 push();
 push(content.replace(/\n+$/, ""));
 push();
 push();
 });

 push(HR);
 push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сгенерировано`);
 push(HR);

 const txt = out.join("\n") + "\n";
 fs.writeFileSync(TXT_PATH, txt, "utf8");

 console.log("[1/3] код -> txt");
 console.log(`    файлов:  ${files.length}`);
 console.log(`    глав:    ${chapters.length}`);
 console.log(`    строк:   ${txt.split("\n").length}`);
 console.log(`    выход:   ${path.relative(ROOT, TXT_PATH)}`);
}

main();

```

---

ФАЙЛ 81/83: scripts/export/common.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 66 | РАЗМЕР: 1.1 КБ

---

```

/**
 * Общие настройки и утилиты пайплайна экспорта.
 *
 * код -> full_code_all_pages.txt (collect-code.mjs)
 * txt -> page-XXXX.png (render-pages.mjs)

```

```
}

export function humanSize(bytes) {
 if (bytes < 1024) return `${bytes} B`;
 if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(2)} KB`;
 return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
}

/** ImageMagick 7 (`magick`) или 6 (`convert`). */
export function imageMagickCmd() {
 for (const cmd of ["magick", "convert"]) {
 try {
 execFileSync(cmd, ["-version"], { stdio: "ignore" });
 return cmd;
 } catch {
 /* пробуем следующий */
 }
 }
 throw new Error(
 "ImageMagick не найден. Установите его: sudo apt-get install imagemagick"
);
}
```

---

ФАЙЛ 82/83: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМЕР: 1.5 KB

---

```
/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full_code_all_pages.txt на страницы формата
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu)
 *
 * Результат: tmp/export-pages/page-0001.png, page-0002.png, ...
 */
import fs from "node:fs";
```

```

 throw new Error(`Нет ${path.relative(ROOT, TXT_PATH)}
 }

 const im = imageMagickCmd();
 const pages = paginate(fs.readFileSync(TXT_PATH, "utf8"));

 fs.rmSync(PAGES_DIR, { recursive: true, force: true });
 ensureDir(PAGES_DIR);

 const total = pages.length;
 const jobs = pages.map((lines, idx) => {
 const num = String(idx + 1).padStart(4, "0");
 const header = `Универсальное пособие • полный исходный код
 Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / ${total}`)
 `стр. ${idx + 1} / ${total}`;
 const body = [header, "-".repeat(PAGE.cols), ...lines];
 const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
 const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
 fs.writeFileSync(txtFile, body + "\n", "utf8");
 return { txtFile, pngFile, num };
 });

 const args = (job) => [
 "-density", String(PAGE.density),
 "-page", PAGE.size,
 "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : "DejaVuSansMono",
 "-pointsize", String(PAGE.pointsize),
 `text:${job.txtFile}`,
 "-background", "white",
 "-flatten",
 "-colorspace", "Gray",
 ...(PAGE.monochrome ? ["-monochrome"] : []),
 "-strip",
 "-define", "png:compression-level=9",
 job.pngFile,
];

 const concurrency = Math.max(2, Math.min(os.cpus().length, 10));

```



ФАЙЛ 83/83: `.github/workflows/rebuild-pdf.yml`

КАТЕГОРИЯ: CI / автоматизация | СТРОК: 128 | РАЗМЕР: 4.0 КБ

name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

```
#
исходный код → full_code_all_pages.txt (script)
#
|
|→ page-0001.png ... page-NNNN.png (images)
#
|
|→ full_code_all_page.pdf (PDF)
```

# Готовые TXT и PDF коммитятся обратно в ветку (их отдаёт GitHub Actions)

# промежуточные PNG сохраняются как artifacts запуска.

on:

push:

branches:

- "\*\*"

paths-ignore:

# чтобы коммит самого workflow не запускал бесконечно

- "public/export/\*\*"

- "\*\*/\*.md"

workflow\_dispatch:

inputs:

density:

description: "DPI растеризации страниц (по умолчанию 300)"

required: false

default: "150"

concurrency:

group: rebuild-pdf-\${{ github.ref }}

cancel-in-progress: true

permissions:

contents: write

- run: npm run export:txt
- name: Step 2 – txt → png  
env:  
EXPORT\_DENSITY: \${github.event.inputs.density}  
run: npm run export:png
- name: Step 3 – png → pdf  
run: npm run export:pdf
- name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true
- name: Summary  
run: |  
{  
echo "### Экспорт пересобран"  
echo ""  
echo "| Артефакт | Размер |"  
echo "| --- | --- |"  
echo "| \public/export/full\_code\_all\_pages  
echo "| \public/export/full\_code\_all\_pages  
echo "| PNG-страниц | \$(ls tmp/export-pages.  
} >> "\$GITHUB\_STEP\_SUMMARY"
- name: Upload artifacts (PDF + TXT)  
uses: actions/upload-artifact@v4  
with:  
name: full-code-export  
path: |  
public/export/full\_code\_all\_pages.pdf  
public/export/full\_code\_all\_pages.txt  
if-no-files-found: error  
retention-days: 30
- name: Upload artifacts (PNG-страницы)  
uses: actions/upload-artifact@v4